Track C: Microsoft Fabric

Generated from the same content files as the app. Each module lists its roadmap months. Solutions are in the appendix.

Contents

Track C: Microsoft Fabric 3

C0 Start here: foundations for Fabric 4

C0-L1 Data platform vocabulary, files, tables, lakes and warehouses 4

C0-L2 Power BI basics, reports, semantic models and measures 7

C0-L3 Two engines, one table, Spark notebooks and the SQL analytics endpoint 11

C1 Fabric architecture 15

C1-L1 Tenant, capacity, workspace and item 15

C1-L2 OneLake, one lake per tenant 19

C1-L3 Workspace roles, granted to groups 23

C2 Lakehouse, Warehouse and SQL analytics endpoint 28

C2-L1 Lakehouse Tables and Files, and schemas 28

C2-L2 Warehouse T-SQL, and where it differs from SQL Server 32

C2-L3 Lakehouse, Warehouse or SQL analytics endpoint 36

C3 Unifying datasets 42

C3-L1 OneLake shortcuts, and the delete that goes through 42

C3-L2 Mirroring operational databases into OneLake 46

C3-L3 Mirroring Azure Databricks Unity Catalog into Fabric 50

C4 Ingestion and orchestration 55

C4-L1 Choosing how data gets into Fabric 55

C4-L2 Incremental copy with a watermark, or with CDC 59

C4-L3 Pipelines that fail fast, retry sensibly and alert 63

C5 Spark notebooks in Fabric 68

C5-L1 From Databricks notebooks to Fabric notebooks 68

C5-L2 Environments, resource profiles and V-Order 72

C5-L3 Delta tables shared between Databricks and Fabric 76

C6 Semantic models and Direct Lake 81

C6-L1 Star schemas for semantic models 81

C6-L2 DAX measures that state their filter context 85

C6-L3 Direct Lake, framing and the fallback you did not plan 89

C7 Real-Time Intelligence 94

C7-L1 Eventstream, events in, routed out 94
C7-L2 Eventhouse and KQL for time-series questions 98
C7-L3 Activator, rules that act on events 102
C8 Copilot and AI in Fabric 107
C8-L1 Copilot in Fabric, where it runs and what it costs 107
C8-L2 Data agents, questions in plain language 111
C8-L3 Verify what Copilot writes 116
C9 Governance and security 121
C9-L1 Domains and endorsement, finding the right data 121
C9-L2 OneLake security, roles down to rows and columns 125
C9-L3 Sensitivity labels, Purview and lineage 130
C9-L4 Audit evidence, Purview scans, label coverage and the audit log 134
C10 DevOps and cost 139
C10-L1 Git integration for Fabric workspaces 139
C10-L2 Deployment pipelines, dev to test to production 143
C10-L3 Capacity, smoothing and throttling 147
Track C best-practice checklist 152

Appendix: solutions and answer keys 153

Track C: Microsoft Fabric

Unify data, analytics and Copilot in Fabric, and verify what Copilot produces.

C0 Start here: foundations for Fabric

Roadmap: Prerequisite for all months. Data platform vocabulary, Power BI semantic models, and the Spark and SQL engines side by side.

C0-L1 Data platform vocabulary, files, tables, lakes and warehouses

Concept

Fabric lessons assume a shared vocabulary. Here it is, platform first, then how Fabric maps onto it.

Files and formats.

- **CSV** is plain text. It stores no types, so a reader guesses them, and a postcode `01234` can come back as the number `1234`.
- **Parquet** stores data by column, with each column's type, and compresses it. Reading two columns of a wide table reads only those two.
- **Delta** is a folder of Parquet files plus a **transaction log**, `_delta_log`. The log turns files into a table: each write is a numbered version, readers never see a half-finished write, and you can read an older version.

Places for data.

- A **data lake** holds files of any shape, cheaply. Structure is applied when you read.
- A **data warehouse** holds tables with a fixed schema, queried with SQL.
- A **lakehouse** keeps tables in an open format, such as Delta, in a lake, so a Spark notebook and a SQL engine read the same data.

Storage and compute are separate. Data sits once in storage. Engines, such as Spark, SQL and Power BI, bring compute to it when they run.

How data moves. Batch processing handles a delivery at a time, such as a nightly file. **Streaming** handles events as they arrive, seconds apart. With **ELT**, you load raw data first, then transform it inside the platform. Many teams name the stages **bronze** (raw, as received), **silver** (cleaned) and **gold** (aggregated for reports).

In Fabric, all of this lives in **OneLake**, one logical lake for the whole organisation (GA). Lakehouse tables are Delta tables (GA). Spark reads them,

and the **SQL analytics endpoint** (GA) queries the same tables with T-SQL. Lessons C1-L2 and C2-L1 go deeper.

Why it matters

A team exported its gold tables to CSV for a reporting tool. Store codes such as `00042` became `42`, and the join to the store list silently matched nothing for 300 stores. Reading the Delta table directly, with its types intact, avoided both the copy and the broken join.

Worked example: weak code

```
  1 # Export gold to CSV, then read it back for a report
(1) gold.to_csv("gold_stores.csv", index=False)
(2) report = pd.read_csv("gold_stores.csv")
```

1. **Line 2.** A second copy of the data, in a format that stores no types.
2. **Line 3.** The reader guesses types: store code 00042 comes back as 42.

Worked example: strong code

```
  1 # Read the one Delta table that every engine shares
(1) report = spark.read.table("gold.store_sales")
```

1. **Line 2.** No copy, and the column types travel with the table.

Keep data once, in a typed table format such as Delta, and let each engine read it, instead of exporting copies to untyped files.

Recap card

- **CSV, Parquet and Delta** CSV: text, no types. Parquet: columnar, typed, compressed. Delta: Parquet files plus a transaction log with versions.
- **Lake, warehouse, lakehouse** Lake: files, structure on read. Warehouse: typed tables for SQL. Lakehouse: open tables in a lake, read by Spark and SQL.
- **ELT and medallion stages** Load raw first, transform inside the platform: bronze (as received), silver (clean), gold (aggregated).
- **Fabric's names for these** OneLake is the one lake (GA). Lakehouse tables are Delta (GA). The SQL analytics endpoint queries them with T-SQL (GA).

Practice

Exercise 1. Multiple choice (level 1)

What makes a folder of Parquet files a Delta table?

- a) A transaction log that records every write as a numbered version
- b) A different file extension
- c) Storing the data as CSV
- d) Running a SQL engine on top

Why?

- r1) The log tells readers which files form each version, so writes are all-or-nothing and old versions stay readable.
- r2) The log compresses the data.

Exercise 2. Predict the output (level 2)

What does this print? (pandas 3.0 with pyarrow)

What you need to know.

The code uses pandas, Python's table library, and the `io` module:

- `pd.DataFrame({"postcode": ["01234"]})` builds a one-column table.
- `df.to_csv(index=False)` writes it as CSV text. `pd.read_csv(...)` reads CSV back.
- `df.to_parquet(buffer)` writes it as Parquet. `pd.read_parquet(buffer)` reads Parquet back.
- `io.StringIO(text)` and `io.BytesIO()` are in-memory stand-ins for a text file and a binary file. `buffer.seek(0)` rewinds to the start before reading.
- `["postcode"].iloc[0]` takes the first value of the column.

```
import io

import pandas as pd

df = pd.DataFrame({"postcode": ["01234"]})
csv_text = df.to_csv(index=False)
buffer = io.BytesIO()
df.to_parquet(buffer)
buffer.seek(0)
print(pd.read_csv(io.StringIO(csv_text))["postcode"].iloc[0],
    pd.read_parquet(buffer)["postcode"].iloc[0])
```

Exercise 3. Fill the gap (level 2)

Complete the definitions.

```
A Delta table is a folder of _____(1) files plus a transaction log.
A _____(2) keeps open-format tables in a lake, so Spark and SQL read the same data.
With _____(3), you load raw data first and transform it inside the platform.
```

Exercise 4. Put in order (level 2)

Put the stages of a typical ELT flow for a nightly delivery in order.

- Clean, rename and type it into a silver table.
- A semantic model and its reports read the gold tables.
- Load it into a bronze table, as received, with audit columns.
- The delivery file lands in the lake, unchanged.
- Aggregate it into gold tables for reporting.

Exercise 5. Architecture design (level 3)

Pick the best fit for each need.

Options: Files in a data lake, Delta tables in a lakehouse, Streaming ingestion, A nightly CSV export.

1. Keep supplier PDFs and raw JSON exports cheaply, structure unknown
2. A table that a Spark notebook and a SQL report must both read, with types intact
3. Vehicle readings that must reach a dashboard within seconds

Sources

- A CSV round trip turns the text 01234 into 1234 in pandas 3.0, while a Parquet round trip keeps 01234. (verified: Run on pandas 3.0.2 with pyarrow 22.0.0 and pandas 3.0.6 with pyarrow 26.0.0.)
- OneLake, Lakehouse (Delta tables) and the SQL analytics endpoint are GA. (search-only: VERSIONS.md Fabric feature status, checked on Microsoft Learn in September 2026.)

C0-L2 Power BI basics, reports, semantic models and measures

Concept

Power BI separates three layers. Keep them apart in your head.

- **Data** lives in tables at the source, such as Delta tables in a lakehouse.

- A **semantic model** describes that data for analysis: which tables it uses, how they relate, and which calculations exist.
- A **report** holds pages of **visuals**, such as charts and tables, that query the model.

Tables and relationships. A model usually has a **fact table**, one row per event or response, holding numbers and keys, and **dimension tables** that describe those keys, such as region or brand. A relationship links each dimension to the fact table, one-to-many, so a filter on a region narrows the facts to that region. Lesson C6-L1 builds this properly as a star schema.

Columns and measures. A **calculated column** is worked out once per row and stored. A **measure** is worked out when a visual asks, for whatever the visual currently shows. That difference matters for ratios:

```
1 Weighted reach % = DIVIDE ( SUM ( fact[reach] ), SUM ( fact[base] ) )
```

This measure divides two totals, so it is right at every level: a region, a wave or everything. An average of per-row percentages weights a row with 2 respondents the same as a row with 2,000.

Filter context. Each visual cell evaluates a measure under the filters that apply to it: slicers, the row and column headers, and page filters. The same measure gives North's figure in North's row and the total in the total row. Lesson C6-L2 shows how to change that context on purpose.

`DIVIDE` returns blank when the denominator is 0 or blank.

Storage modes. Import copies the data into the model and refreshes it on a schedule. **DirectQuery** sends each query to the source. **Direct Lake** (GA in Fabric) reads Delta tables in OneLake directly, without an import copy (lesson C6-L3).

Why it matters

A dashboard showed "average reach %" as the average of a calculated column, reach divided by base for each region-wave row. Small regions counted as much as large ones, and the national figure read 70% where the true weighted reach was 83%. A measure dividing the summed reach by the summed base gives 83% at every level, including the total row.

Worked example: weak code

```
❶ Calculated column: fact[reach_pct] = fact[reach] / fact[base]
❷ Visual value: Average of fact[reach_pct]
```

1. **Line 1.** Worked out once per row and stored, before any filter applies.

2. **Line 2.** Averages the row percentages, so a tiny region counts as much as a large one.

Worked example: strong code

```
   1 Weighted reach % =
(1) DIVIDE ( SUM ( fact[reach] ), SUM ( fact[base] ) )
```

1. **Line 2.** A measure: two totals divided, evaluated for whatever each visual cell shows. DIVIDE returns blank for a zero base.

Put ratios in measures that divide totals, and let filter context decide what each total covers.

Recap card

- **Data, semantic model, report** Data sits in source tables. The model describes tables, relationships and measures. Reports hold visuals that query the model.
- **Calculated column against measure** A column is worked out per row and stored. A measure is worked out when a visual asks, in that visual's filter context.
- **Ratios in Power BI** A measure dividing totals: DIVIDE (SUM (reach), SUM (base)). Never average per-row percentages.
- **Import, DirectQuery, Direct Lake** Import copies and refreshes. DirectQuery queries the source each time. Direct Lake (GA) reads OneLake Delta tables directly.

Practice

Exercise 1. Multiple choice (level 1)

Which layer do you change to add a new calculation that every report can use?

- a) The semantic model, by adding a measure
- b) One report's visual
- c) The source Delta table, by adding a column for each possible filter
- d) The report's page filter

Why?

- r1) Reports query the model, so a measure defined there is available to all of them.
- r2) The model is where the data is stored.

Exercise 2. Predict the output (level 2)

The fact table has two rows: reach 1 with base 2, and reach 9 with base 10. What do the two total cells show, rounded to two decimals? Answer as two numbers: the average of the calculated column, then the measure.

```
Calculated column: fact[reach_pct] = fact[reach] / fact[base]
Total cell 1: Average of fact[reach_pct]
Total cell 2: DIVIDE ( SUM ( fact[reach] ), SUM ( fact[base] ) )
```

Exercise 3. Fill the gap (level 2)

Write the weighted reach measure so a zero base gives a blank.

```
Weighted reach % =
_____(1) ( _____(2) ( fact[reach] ), SUM ( fact[base] ) )
```

Exercise 4. Architecture design (level 2)

A survey model has these tables. Classify each one.

Options: Fact table, Dimension table.

1. fact_response: one row per respondent per wave, with weight, reach and keys
2. dim_region: one row per region code, with the region's name
3. dim_wave: one row per wave, with its fieldwork dates

Exercise 5. Put in order (level 3)

A report shows weighted reach for North. Put the path from data to that number in order.

- The measure divides the summed reach by the summed base for those rows.
- Gold Delta tables hold the responses in OneLake.
- The semantic model uses those tables and relates dim_region to the fact table.
- A slicer on the report page selects North.
- The filter flows from dim_region to the fact rows for North.

Sources

- DIVIDE returns BLANK for a zero or BLANK denominator unless an alternate result is passed. (verified: https://learn.microsoft.com/en-us/dax/best-practices/dax-divide-function-operator (as cited in C6-L2))
- Direct Lake is GA in Fabric; Import and DirectQuery are Power BI storage modes. (search-only: VERSIONS.md Fabric feature status (fundamentals/

direct-lake-overview, 2026-09-02) and Power BI storage mode documentation.)

C0-L3 Two engines, one table, Spark notebooks and the SQL analytics endpoint

Concept

A Fabric lakehouse (GA) stores its tables once, as Delta in OneLake. Two engines read them, and you will use both.

Spark, in a notebook. Fabric notebooks run PySpark and give you a `spark` session. Runtime 2.0 (GA) runs Spark 4.1, and the examples here behave the same on the Spark 4.0 used in Track A. You work with DataFrames:

```
from pyspark.sql import functions as F

df = spark.read.table("respondents")
totals = df.filter(F.col("final_weight") >
    0).groupBy("region").agg(F.sum("final_weight").alias("total"))
totals.show()
```

`filter`, `groupBy`, `agg`, `select` and `orderBy` are **transformations**: they describe a new DataFrame and run nothing. **Actions** run the plan: `show()` prints rows, `count()` counts them and `collect()` returns them to Python as `Row` objects. `groupBy("region").count()` is a shortcut that adds a `count` column. In tests, `spark.createDataFrame(rows, "region STRING, w DOUBLE")` builds a small DataFrame. `spark.sql("SELECT ...")` runs Spark SQL in the same notebook.

Spark also **writes**: `df.write.mode("append").saveAsTable("respondents")` adds rows to a managed table, and `mode("overwrite")` replaces them.

T-SQL, on the SQL analytics endpoint. Every lakehouse gets a SQL analytics endpoint (GA). It answers T-SQL queries over the same tables, so analysts and reports can query with familiar SQL:

```
SELECT region, SUM(final_weight) AS total
FROM dbo.respondents
WHERE final_weight > 0
GROUP BY region;
```

The endpoint is **read-only for table data**. You can create views on it, but `INSERT`, `UPDATE` and `DELETE` against lakehouse tables fail there. Write with Spark, or use a Warehouse when you need T-SQL writes (lesson C2-L3).

Pick by job: Spark for loading and transforming, the SQL endpoint for querying and reporting.

Why it matters

A team planned its nightly load as T-SQL `INSERT` statements on the SQL analytics endpoint, because the analysts knew SQL. The first run failed on every statement, since the endpoint does not write to lakehouse tables. Moving the load to a Spark notebook, and keeping T-SQL for the reports, matched each engine to what it does.

Worked example: weak code

```
1 -- On the SQL analytics endpoint
1 INSERT INTO dbo.respondents (respondent_id, final_weight)
3 SELECT respondent_id, final_weight FROM dbo.staging_w14;
```

1. **Line 2.** The endpoint is read-only for lakehouse table data, so this fails.

Worked example: strong code

```
1 -- Load with Spark, in a notebook:
1 --
    spark.read.table("staging_w14").write.mode("append").saveAsTable("respo
    ndents")
3 -- Query with T-SQL, on the SQL analytics endpoint:
2 SELECT region, SUM(final_weight) AS total
5 FROM dbo.respondents
6 GROUP BY region;
```

1. **Line 2.** Spark writes the Delta table.
2. **Line 4.** The endpoint reads the same table with T-SQL, without a copy.

One Delta table, two engines: write and transform with Spark, query with T-SQL on the SQL analytics endpoint.

Recap card

- **Two engines over one lakehouse table** Spark in notebooks reads and writes. The SQL analytics endpoint queries with T-SQL and is read-only for table data.
- **Append rows to a lakehouse table from Spark** df.write.mode("append").saveAsTable("respondents"). mode("overwrite") replaces the rows.
- **Spark transformations against actions** filter, groupBy, agg, select describe a new DataFrame. show, count, collect run the plan.
- **Where to put T-SQL writes** In a Warehouse. The lakehouse's SQL analytics endpoint does not write table data.

Practice

Exercise 1. Multiple choice (level 1)

You need to append tonight's delivery to the lakehouse table `respondents`. Where do you write the code?

- a) In a Spark notebook, with `df.write.mode("append").saveAsTable("respondents")`
- b) On the SQL analytics endpoint, with `INSERT INTO dbo.respondents ...`
- c) In a Power BI report
- d) Anywhere: every engine can write lakehouse tables

Why?

- r1) Spark writes lakehouse tables. The SQL analytics endpoint is read-only for table data.
- r2) Spark is faster than SQL.

Exercise 2. Predict the output (level 2)

What does this print? (Spark 4.0 and later)

```
from pyspark.sql import functions as F

df = spark.createDataFrame([("N", 2.0), ("S", 0.0), ("S", 3.0)], "region STRING, w DOUBLE")
print(df.filter(F.col("w") > 0).groupBy("region").count().orderBy("region").collect())
```

Exercise 3. Fill the gap (level 2)

Read the staging table and add its rows to `respondents`, keeping the rows already there.

```
spark.read._____(1)
    ("staging_w14").write.mode("_____(2)").saveAsTable("respondents")
```

Exercise 4. Write the function (level 3)

On the SQL analytics endpoint, write a query that returns each `region` with `total_weight`, the sum of `final_weight`, from `dbo.respondents`. Count only rows with a positive weight, and sort by region.

Shape of the answer. The reference answer follows this outline. Replace each ... with your own code.

```
SELECT ...
FROM ...
WHERE ...
GROUP BY ...
```

```
5 ORDER BY ...
```

Starter code:

```
1 SELECT *
2 FROM dbo.respondents;
```

Exercise 5. Spot the bug (level 3)

This script runs on a lakehouse's SQL analytics endpoint and fails. Tap the line that cannot work there.

```
1 SELECT COUNT(*) AS rows_before FROM dbo.respondents;
2 INSERT INTO dbo.respondents (respondent_id, final_weight) VALUES ('R9',
    1.0);
3 SELECT COUNT(*) AS rows_after FROM dbo.respondents;
```

Write down the line number of the bug.

Sources

- The E2 code prints [Row(region='N', count=1), Row(region='S', count=1)]; spark.sql runs SQL in the same session. (verified: Run on PySpark 4.0.4 locally. Not run on Fabric Runtime 2.0 (Spark 4.1).)
- The SQL analytics endpoint (GA) is read-only for lakehouse table data and supports views; Lakehouse is GA; Fabric Runtime 2.0 (GA) runs Spark 4.1. (search-only: VERSIONS.md Fabric feature status (data-engineering/lakehouse-sql-analytics-endpoint, runtime), as taught in C2-L3 and C5-L2.)

C1 Fabric architecture

Roadmap: Months 16 to 18: MLOps and deployment; Current work. Workspaces, capacities, items and OneLake as one logical lake.

C1-L1 Tenant, capacity, workspace and item

Concept

Everything in Fabric sits in one hierarchy. Learn it first, because cost, access and deployment all follow it.

- **Tenant.** Your organisation's Microsoft Entra ID tenant. Tenant settings, set by Fabric administrators, apply to everything below.
- **Capacity.** A pool of compute you buy. Every Fabric workload, from Spark to Power BI, draws from the capacity its workspace is assigned to.
- **Workspace.** A container where a team creates and shares items. Access is granted per workspace, through roles (lesson C1-L3).
- **Item.** A lakehouse, warehouse, notebook, pipeline, semantic model, report, eventhouse and so on. Items hold or process the data.

Capacities come as SKUs. F SKUs are the recommended Fabric capacities. The number is the compute in **capacity units** (CUs): an F2 has 2 CUs, an F64 has 64, up to F2048. A trial capacity has 64 CUs. Microsoft 365 P SKUs also run Fabric if Fabric is enabled on them.

Size affects licensing, not only speed. On **F64 or larger** (or a P capacity), users with a free Fabric license can view Power BI content. On smaller F SKUs, each viewer needs a Pro, Premium Per User or individual trial licence.

Pausing a capacity stops compute billing while it is paused. It is not free of consequences: when you pause, any smoothed and overage usage still owed is added to your Azure bill at once, and items on that capacity stop working until you resume.

Two practical rules follow from the hierarchy:

- Put dev, test and production in **separate workspaces**, and assign production to a capacity that experiments cannot exhaust.
- Before you design anything, find out which capacity SKU you have. It decides what users need to pay for and how much work runs at once.

Facts in this lesson come from Microsoft Learn's licensing and capacity pages, checked September 2026.

Why it matters

A team built a Power BI report for 300 frontline managers on an F32 capacity, assuming that managers with free licences could view it, as they had on the old Premium P1. Launch day produced 300 "You need a license" messages. The fix was either 300 Pro licences or a move to F64. Neither was in the budget, and the launch slipped a quarter.

Worked example: weak code

```
Workspace: Analytics (all projects)
Capacity: F32 shared with the data science sandbox
Viewers: 300 managers with free licences
Environments: dev and prod items in the same workspace
```

1. **Line 2.** Production shares compute with experiments. One heavy notebook throttles the reports.
2. **Line 3.** Below F64, every viewer needs a Pro, PPU or trial licence.
3. **Line 4.** No separation: a change to a dev item can break production.

Worked example: strong code

```
Workspace: sales_reporting_prod   -> capacity: F64 (production only)
Workspace: sales_reporting_test   -> capacity: F8
Workspace: sales_reporting_dev    -> capacity: F8
Viewers: 300 managers with free licences, reading the prod workspace's app
Sandbox workspaces                -> capacity: F4, paused out of hours
```

1. **Line 1.** F64 lets free-licence users view Power BI content, and production has its own compute.
2. **Line 5.** Experiments cannot starve production, and pausing saves money when nobody works.

The capacity decides who can view, how much runs at once and what it costs. Separate environments into workspaces, and give production its own capacity.

Recap card

- **Fabric hierarchy** Tenant, capacity, workspace, item.
- **What F64 means** An F SKU with 64 capacity units. F SKUs run from F2 to F2048.
- **Free-licence viewers of Power BI content** Need an F64 or larger (or P) capacity. Smaller F SKUs need Pro, PPU or trial licences for viewers.
- **Pausing a capacity** Stops compute billing from then on, bills remaining smoothed and overage usage at once, and stops its items.

Practice

Exercise 1. Multiple choice (level 1)

What does the number in the SKU name F32 tell you?

- a) 32 users can use the capacity.
- b) The capacity has 32 capacity units (CUs) of compute.
- c) 32 GB of OneLake storage is included.
- d) 32 workspaces can be assigned to it.

Why?

- r1) For F SKUs the number equals the capacity units: F2 is 2 CUs, F64 is 64.
- r2) SKU numbers are marketing names with no meaning.

Exercise 2. Put in order (level 2)

Order the Fabric hierarchy from the broadest to the most specific.

- Workspace
- Capacity
- Tenant
- Item, such as a lakehouse

Exercise 3. Architecture design (level 2)

A report will serve 200 viewers who have only free Fabric licences. Choose the set-up.

Options: F64 or larger, F16, cheaper, same features.

1. Capacity for the production workspace
2. What else runs on that capacity

Exercise 4. Write the function (level 3)

Write `cu_hours(sku: str, hours: float) -> float` for a cost estimate. The SKU is a string such as `"F64"`. Return the capacity units multiplied by the hours running. Accept only the F SKUs F2, F4, F8, F16, F32, F64, F128, F256, F512, F1024 and F2048, and raise `ValueError` for any other SKU or for negative hours.

Your code must pass these checks:

- Multiply the SKU's CUs by hours: F64 for 24 hours is 1,536 CU hours.

- Only the listed F SKUs are valid. F3, P1, lower-case f64, F4096 and 64 must raise ValueError.
- Negative hours must raise ValueError.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```python
# Module constant.
F_SKUS = ...

# Capacity-unit hours for running an F SKU capacity.
def cu_hours(sku: str, hours: float) -> float: ...
```

Starter code:

```python
def cu_hours(sku, hours):
    return int(sku[1:]) * hours
```

Exercise 5. Multiple choice (level 4)

A team pauses its F64 capacity at 18:00 to save money after a heavy afternoon of Spark jobs. What happens?

- a) Nothing is billed for the afternoon's work, because the capacity is paused.
- b) Smoothed and overage usage still owed is added to the Azure bill at once, and items on the capacity stop working until it resumes.
- c) The capacity keeps running reports but stops Spark.
- d) OneLake data on the capacity is deleted after 24 hours.

Why?

- r1) Fabric smooths usage over time. Pausing settles what is still owed and makes the capacity's items unavailable.
- r2) Paused capacities are billed at double rate.

Sources

- F SKUs from F2 to F2048, where the number equals capacity units; a trial capacity has 64 CUs. (verified: https://learn.microsoft.com/en-us/fabric/enterprise/licenses (updated 2026-06-15))
- F64+ or P capacities support viewing Power BI content by users with a free Fabric licence; smaller F SKUs require Pro, PPU or trial licences for viewers. (verified: https://learn.microsoft.com/en-us/fabric/enterprise/licenses (updated 2026-06-15))
- When a capacity is paused, remaining cumulative overages and smoothed operations are added to the Azure bill. (verified: https://

learn.microsoft.com/en-us/fabric/enterprise/pause-resume (updated 2026-04-20): pausing can prevent content from being available until resumed)

C1-L2 OneLake, one lake per tenant

Concept

Every Fabric tenant has exactly **one OneLake**. You cannot create a second one or delete it. It is built on Azure Data Lake Storage and stores tables in open formats: **Delta Parquet** or **Iceberg**. Every Fabric engine (Spark, the SQL engines, Power BI's Direct Lake, KQL) reads the same files, so one copy of the data serves them all.

OneLake mirrors the Fabric hierarchy: a folder per **workspace**, and inside it a folder per **item**, named with its type. A lakehouse holds two areas: `Tables` for managed Delta tables and `Files` for anything else.

You address data with an ADLS-style path:

```
1 https://onelake.dfs.fabric.microsoft.com/<workspace>/<item>.<itemtype>/
    <path>
2 abfss://<workspace>@onelake.dfs.fabric.microsoft.com/<item>.<itemtype>/
    <path>
```

For example `abfss://sales_prod@onelake.dfs.fabric.microsoft.com/lh_sales.lakehouse/Tables/orders`. The item type is lower case: `.lakehouse`, `.warehouse`.

Names change. **GUIDs do not.** OneLake accepts the workspace and item GUIDs in place of names, and they stay fixed when someone renames the workspace or item. With GUIDs you give both the workspace and the item as GUIDs, and you drop the item type. You find them in the item's URL in the Fabric portal.

Use GUIDs in pipelines, jobs and code that other people will run. Two more reasons from Microsoft Learn:

- The `abfs` driver URI does not allow special characters such as spaces in the workspace name. GUIDs avoid the problem.
- The global endpoint `onelake.dfs.fabric.microsoft.com` can route through another region. If data residency matters, use your workspace's regional endpoint, `https://<region>-onelake.dfs.fabric.microsoft.com`.

Because tools that speak ADLS APIs can read OneLake, Azure Databricks, Azure Storage Explorer or Python's Azure SDK can read Fabric data with the right permissions.

Why it matters

A Databricks job read a Fabric lakehouse through `abfss://Sales Analytics@onelake.dfs.fabric.microsoft.com/...`. It failed on the first run, because the workspace name held a space. The engineer renamed the workspace to `SalesAnalytics`. The job worked, and four Power BI dataflows and two pipelines that referred to the old name broke that night. Using the workspace and item GUIDs would have avoided both failures.

Worked example: weak code

```
1 path = "abfss://Sales Analytics@onelake.dfs.fabric.microsoft.com/Sales LH.Lakehouse/Tables/orders"
2 df = spark.read.format("delta").load(path)
```

1. **Line 1.** Spaces break the abfs URI, the item type is not lower case, and a rename breaks the path.

Worked example: strong code

```
1  ONELAKE_HOST = "onelake.dfs.fabric.microsoft.com"
2
3
4  def onelake_table_path(workspace_id: str, item_id: str, table: str) -> str:
5      """abfss path to a lakehouse table, by GUID, stable across renames."""
1      return f"abfss://{workspace_id}@{ONELAKE_HOST}/{item_id}/Tables/{table}"
7
8
2  path = onelake_table_path(CONFIG["workspace_id"], CONFIG["lakehouse_id"], "orders")
10 df = spark.read.format("delta").load(path)
```

1. **Line 6.** With GUIDs, both workspace and item are IDs and the item type is dropped.
2. **Line 9.** IDs come from config, not from code (lesson A1-L3).

One lake, one copy, many engines. Address data by GUID in anything automated.

Recap card

- **How many OneLakes per tenant?** Exactly one. You cannot create or delete OneLakes.

- **OneLake table formats** Delta Parquet or Iceberg, open formats every Fabric engine reads.
- **OneLake path by name** abfss://<workspace>@onelake.dfs.fabric.microsoft.com/<item>.<itemtype>/Tables/<table>.
- **Why GUIDs in automation** They survive renames, avoid special-character problems in abfs URIs, and drop the item type.

Practice

Exercise 1. Multiple choice (level 1)

Your company has three business units in one Fabric tenant. How many OneLakes are there?

- a) One per business unit
- b) One for the whole tenant
- c) One per capacity
- d) One per workspace

Why?

- r1) Every tenant has a single OneLake. Workspaces organise it; they do not create new lakes.
- r2) OneLake licences are sold per tenant.

Exercise 2. Fill the gap (level 2)

Complete the path to table `orders` in lakehouse `lh_sales`, workspace `sales_prod`.

```
abfss://_____(1)@onelake.dfs.fabric.microsoft.com/lh_sales._____(2)/
    _____(3)/orders
```

Exercise 3. Spot the bug (level 3)

This path fails with an invalid URI error. Tap the line with the problem and rewrite the path so it survives renames too.

```
WORKSPACE = "Sales Analytics"
LAKEHOUSE = "lh_sales"
path = f"abfss://{WORKSPACE}@onelake.dfs.fabric.microsoft.com/
    {LAKEHOUSE}.lakehouse/Tables/orders"
```

Write down the line number of the bug. Then write the fix.

Exercise 4. Write the function (level 3)

Write `onelake_path(workspace: str, item: str, item_type: str, relative: str) -> str` returning `abfss://<workspace>@onelake.dfs.fabric.microsoft.com/<item>.<item_type>/<relative>`.

- `item_type` must be `"lakehouse"` or `"warehouse"`. Otherwise raise `ValueError`.
- Raise `ValueError` if `workspace` or `item` contains a space.
- Strip leading slashes from `relative`.

Your code must pass these checks:

- Build the path in the given format, without a double slash.
- Only lower-case lakehouse or warehouse are allowed.
- Names with spaces break the abfs URI. Raise ValueError.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```python
# Module constant.
ONELAKE_HOST = ...

# Module constant.
ITEM_TYPES = ...

# abfss path to data in a OneLake item, addressed by name.
def onelake_path(
    workspace: str,
    item: str,
    item_type: str,
    relative: str,
) -> str: ...
```

Starter code:

```python
def onelake_path(workspace, item, item_type, relative):
    return f"abfss://{workspace}@onelake.dfs.fabric.microsoft.com/{item}.{item_type}/{relative}"
```

Exercise 5. Architecture design (level 4)

An Azure Databricks job in the UK will read a Fabric lakehouse in a UK South workspace every night. Data must stay in the UK. Choose the set-up.

Options: GUIDs from config, Display names in the code.

1. How the job refers to workspace and lakehouse
2. Endpoint
3. Data movement

Sources

- Every tenant has a single OneLake, built on ADLS, storing tables in Delta Parquet or Iceberg; you cannot create multiple OneLakes. (verified: https://learn.microsoft.com/en-us/fabric/onelake/onelake-overview (updated 2026-03-09))
- OneLake URI formats by name and by GUID; GUIDs do not change on rename; abfs URIs do not allow special characters such as spaces in the workspace name; the global endpoint may route data out of region, and regional endpoints avoid this. (verified: https://learn.microsoft.com/en-us/fabric/onelake/onelake-access-api (updated 2026-05-18))

C1-L3 Workspace roles, granted to groups

Concept

A workspace has four roles: **Admin**, **Member**, **Contributor** and **Viewer**. Each role includes the rights of the ones below it, with a few rights reserved for the top.

From Microsoft Learn's role table (checked September 2026):

Right	Admin	Member	Contributor	Viewer
Update or delete the workspace, add or remove admins	yes			
Connect the workspace to Git, create a workspace identity	yes			
Add members or lower roles, allow resharing	yes	yes		
Create and change items, run notebooks and pipelines	yes	yes	yes	
Read lakehouse and warehouse data through Spark and OneLake APIs (ReadAll)	yes	yes	yes	
Read lakehouse and warehouse data with T-SQL (ReadData)	yes	yes	yes	yes

Two consequences surprise people coming from Databricks:

- A **Viewer** can query a lakehouse through its SQL analytics endpoint, but cannot read the same tables from a Spark notebook or through OneLake APIs.

- A **Contributor** can change and delete every item in the workspace, including production lakehouse tables. Contributor in production is powerful.

Rules that keep access manageable:

- **Assign roles to Microsoft Entra security groups**, not to individuals (as in lesson A4-L2). People join and leave groups. Role assignments stay the same.
- **Least privilege per environment.** Engineers are Contributors in dev. In production, deployment runs under an automation identity, and people are Viewers unless they are on call.
- **Keep at least two admins**, as groups or named people, so a departure does not lock the workspace.
- Workspace roles are coarse. For finer control inside an item, use item sharing, SQL permissions, or OneLake security (lesson C9-L2).

Why it matters

A contractor was added to a production workspace as Contributor "to look at a notebook". Two weeks later they ran a clean-up notebook against the wrong lakehouse and dropped three tables that reports depended on. Viewer would have let them read the notebook and query the data with T-SQL, and would not have let them change anything.

Worked example: weak code

```
1 workspace: sales_prod
2 roles:
1   Admin: [sam@contoso.com]
2   Contributor: [grp-sales-analysts, contractor@partner.com]
5   Viewer: []
```

1. **Line 3.** One named admin: if Sam leaves, nobody can manage the workspace.
2. **Line 4.** Write access to production for people who need to read, including an external individual.

Worked example: strong code

```
1 workspace: sales_prod
2 roles:
1   Admin: [grp-fabric-platform-admins]
4   Member: [grp-sales-data-leads]
2   Contributor: [sp-sales-deployment]
3   Viewer: [grp-sales-analysts, grp-sales-contractors]
```

1. **Line 3.** A group of admins: no single point of failure.

2. **Line 5.** Only the deployment identity writes to production.
3. **Line 6.** Readers query with T-SQL and reports, and cannot change items.

Roles go to groups, write access in production goes to automation, and people read.

Recap card

- **Fabric workspace roles** Admin, Member, Contributor, Viewer.
- **Admin-only rights** Update or delete the workspace, manage admins, connect Git, create a workspace identity.
- **Viewer and lakehouse data** Reads with T-SQL (ReadData). Not through Spark or OneLake APIs (ReadAll starts at Contributor).
- **Who writes to production?** An automation identity as Contributor. People get Viewer, through groups.

Practice

Exercise 1. Multiple choice (level 1)

Which workspace role can connect a workspace to a Git repository?

- a) Admin only
- b) Admin and Member
- c) Any role except Viewer
- d) Any role

Why?

- r1) Connecting Git, like deleting the workspace or creating a workspace identity, is reserved for Admins.
- r2) Git is a paid feature for Admins.

Exercise 2. Multiple choice (level 2)

An analyst with the Viewer role runs `spark.read.table("lh_sales.orders")` in a notebook in that workspace. What happens?

- a) It works: Viewers read everything.
- b) It fails: Viewers read lakehouse data with T-SQL, not through Spark or OneLake APIs.
- c) It works, but returns only the first 1,000 rows.
- d) It fails because Viewers cannot see notebooks at all.

Why?

- r1) Viewer includes ReadData (T-SQL) but not ReadAll (Spark and OneLake APIs).
- r2) Spark is disabled for all users in production workspaces.

Exercise 3. Architecture design (level 2)

Assign roles in the production workspace `sales_prod`.

Options: Admin, Member, Contributor, Viewer.

1. Fabric platform team (group)
2. Deployment service principal
3. Sales analysts (group)

Exercise 4. Write the function (level 3)

Write `min_role(action: str) -> str` returning the lowest workspace role that allows the action. Use these actions from the role table:

- `"connect_git"`, `"delete_workspace"`: Admin
- `"add_members"`: Member
- `"run_notebook"`, `"spark_read"`: Contributor
- `"tsql_read"`: Viewer

Raise `KeyError` for any other action.

Your code must pass these checks:

- Map each action to the lowest role in the table. Spark reads need Contributor. T-SQL reads need Viewer.
- Unknown actions must raise KeyError.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```python
# Module constant.
MIN_ROLE = ...

# Lowest Fabric workspace role that allows an action.
def min_role(action: str) -> str: ...
```

Starter code:

```python
def min_role(action):
    return "Viewer"
```

Exercise 5. Spot the bug (level 4)

This role file for the production workspace breaks two rules of the lesson. Tap the line with the more dangerous one, then rewrite the file.

```
1 workspace: sales_prod
2 roles:
3   Admin: [grp-fabric-platform-admins]
4   Contributor: [sp-sales-deployment, grp-sales-analysts]
5   Viewer: [jo.bloggs@contoso.com]
```

Write down the line number of the bug. Then write the fix.

Sources

- The four workspace roles and their rights, including Admin-only Git connection and Viewer having ReadData but not ReadAll. (verified: https://learn.microsoft.com/en-us/fabric/fundamentals/roles-workspaces (updated 2026-04-24))

C2 Lakehouse, Warehouse and SQL analytics endpoint

Roadmap: Months 16 to 18: MLOps and deployment; Current work. When to use each.

C2-L1 Lakehouse Tables and Files, and schemas

Concept

A lakehouse has two top-level areas in OneLake:

- **Tables** holds **managed Delta tables**. Fabric registers them, the SQL analytics endpoint exposes them, and Direct Lake semantic models read them.
- **Files** holds anything else: CSV, JSON, Parquet, images, raw deliveries.

Only Delta tables in Tables reach the SQL engines. A Parquet or CSV file is invisible to the SQL analytics endpoint, wherever it sits. If you put a folder that is not a valid Delta table under Tables, the lakehouse explorer shows it in an **Unidentified** area. A broken shortcut also lands there.

So the flow for a delivery is: land the raw file in **Files** (your bronze copy, as received), then read it with Spark and write a Delta table to **Tables** with `saveAsTable`.

Schemas. New lakehouses created in the portal have schemas enabled by default. Every schema-enabled lakehouse has a `dbo` schema that you cannot rename or remove. A table written without a schema goes to `dbo`. Create schemas by domain (`sales`, `hr`) to keep a growing lakehouse navigable.

In Spark SQL you reference tables with a **four-part name**: `workspace.lakehouse.schema.table`. Parts you leave out come from defaults: a bare table name means the `dbo` schema of the notebook's default lakehouse. For code that others run, name at least the lakehouse and schema, so it does not depend on which lakehouse happens to be attached.

One change to note if you learned Fabric earlier: since 5 September 2025, Fabric no longer creates a default semantic model automatically with each new lakehouse. Create semantic models on purpose (lesson C6-L1).

Why it matters

An engineer wrote cleaned survey data as Parquet into `Files/clean/respondents/`, the way they had written to cloud storage before. The Power BI team could not find the table in the SQL analytics endpoint, so they imported the Parquet into a separate dataflow, a second copy with its own refresh. Writing a Delta table to Tables with `saveAsTable` would have exposed it to every engine with no copy.

Worked example: weak code

```
❶ raw = spark.read.option("header", True).csv("Files/landing/
    respondents_w14.csv")
❷ raw.write.mode("overwrite").parquet("Files/clean/respondents")
```

1. **Line 1.** Every column read as a string, types unchecked.
2. **Line 2.** Parquet under Files: invisible to the SQL analytics endpoint and Direct Lake.

Worked example: strong code

```
 1 from pyspark.sql import DataFrame, SparkSession
 2 from pyspark.sql import types as T
 3
 4 RESPONDENT_SCHEMA = T.StructType([
 5     T.StructField("respondent_id", T.StringType(), False),
 6     T.StructField("wave", T.IntegerType(), False),
 7     T.StructField("weight", T.DoubleType(), True),
 8 ])
 9
10
11 def load_delivery(spark: SparkSession, path: str, table: str) ->
    DataFrame:
12     """Read a CSV delivery from Files and append it to a managed Delta
    table in Tables."""
❶      df = spark.read.option("header",
    True).schema(RESPONDENT_SCHEMA).csv(path)
❷      df.write.mode("append").saveAsTable(table)
15     return df
16
17
❸ load_delivery(spark, "Files/landing/respondents_w14.csv",
    "lh_survey.sales.respondents")
```

1. **Line 13.** An explicit schema: types are decided, not inferred.
2. **Line 14.** A managed Delta table in Tables: visible to Spark, T-SQL and Direct Lake.
3. **Line 18.** Lakehouse and schema named, so the code does not depend on the attached default.

Raw files in Files, Delta tables in Tables, and schema-qualified names in shared code.

Recap card

- **Lakehouse Tables vs Files** Tables: managed Delta tables, visible to SQL and Direct Lake. Files: raw and non-Delta data.
- **Non-Delta folder under Tables** Shown in the Unidentified area, not queryable.
- **Lakehouse schemas** On by default for new lakehouses. dbo is the fixed default schema.
- **Spark four-part name** workspace.lakehouse.schema.table. A bare name means dbo in the default lakehouse.

Practice

Exercise 1. Multiple choice (level 1)

A Parquet file sits at `Files/exports/orders.parquet` in a lakehouse. What can the SQL analytics endpoint do with it?

- a) Query it as a table named orders.
- b) Nothing. Only Delta tables in Tables appear in the endpoint.
- c) Query it after a metadata refresh.
- d) Convert it to Delta automatically.

Why?

- r1) The endpoint exposes Delta tables only. Parquet and CSV cannot be queried through it.
- r2) Files is a private folder only the owner can see.

Exercise 2. Multiple choice (level 2)

In a notebook attached to schema-enabled lakehouse `lh_survey`, you run `spark.read.table("respondents")`. Which table does Spark read?

- a) lh_survey.dbo.respondents
- b) The first schema that has a respondents table
- c) It raises an error: schemas require a four-part name
- d) respondents in every schema, unioned

Why?

- r1) A bare table name resolves to the dbo schema of the default lakehouse.

- r2) dbo is the only schema Spark can read.

Exercise 3. Fill the gap (level 2)

Join two tables in different workspaces in one Spark SQL query.

```
SELECT e.name, d.dept_name
FROM ops_ws.hr_lakehouse._____(1).employees AS e
JOIN global_ws.corporate_lakehouse.company.departments AS d
  ON e.deptno = d.deptno
```

Exercise 4. Write the function (level 3)

Write `load_delivery(spark, path, table)`. Read the CSV at `path` with a header and the given `RESPONDENT_SCHEMA`, append it to the managed table `table` with `saveAsTable`, and return the DataFrame you read. Do not write Parquet to Files.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```
# Module constant.
RESPONDENT_SCHEMA = ...

# Read a CSV delivery and append it to a managed Delta table.
def load_delivery(
    spark: SparkSession,
    path: str,
    table: str,
) -> DataFrame: ...
```

Starter code:

```
from pyspark.sql import types as T

RESPONDENT_SCHEMA = T.StructType([
    T.StructField("respondent_id", T.StringType(), False),
    T.StructField("wave", T.IntegerType(), False),
    T.StructField("weight", T.DoubleType(), True),
])


def load_delivery(spark, path, table):
    df = spark.read.option("header", True).csv(path)
    df.write.mode("overwrite").parquet("Files/clean/respondents")
    return df
```

Exercise 5. Architecture design (level 4)

Lay out a lakehouse for a survey programme with raw CSV deliveries, cleaned respondent data and reporting tables.

Options: Files, Tables, dbo schema, Tables, a named schema per domain.

1. Raw CSV deliveries, kept as received
2. Cleaned respondent-level data
3. Reporting tables read by Power BI

Sources

- Lakehouse Tables holds managed Delta tables and Files holds unstructured or non-Delta data; only Delta tables appear in the SQL analytics endpoint; default semantic models are no longer created automatically since 5 September 2025. (verified: https://learn.microsoft.com/en-us/fabric/data-engineering/lakehouse-overview (updated 2026-05-07))
- Non-Delta folders under Tables and broken shortcuts appear in the Unidentified area. (verified: https://learn.microsoft.com/en-us/fabric/data-engineering/navigate-lakehouse-explorer (updated 2026-05-07))
- Schemas are enabled by default for new lakehouses; dbo cannot be renamed or removed; Spark uses workspace.lakehouse.schema.table and resolves omitted parts from defaults. (verified: https://learn.microsoft.com/en-us/fabric/data-engineering/lakehouse-schemas (updated 2026-02-24))

C2-L2 Warehouse T-SQL, and where it differs from SQL Server

Concept

Fabric Warehouse speaks T-SQL, and most SQL Server habits carry over: `CREATE TABLE`, `INSERT`, `UPDATE`, `DELETE`, stored procedures, views, `TRUNCATE TABLE`, identity columns and explicit transactions. `MERGE` is generally available. Its tables are Delta tables in OneLake, so Spark and Power BI read them too.

The differences cause real bugs. Microsoft Learn, checked September 2026:

Keys are not enforced. `PRIMARY KEY` is supported only as `NONCLUSTERED NOT ENFORCED`. `UNIQUE` likewise, and `FOREIGN KEY` only `NOT ENFORCED`. The engine uses them as hints for query plans. It does **not** stop duplicates or orphans. Deduplicate and check keys yourself (lesson A3-L1 for MERGE sources, lesson A6-L1 for contracts).

Some data types are missing, with documented alternatives:

Not supported	Use
`nchar`, `nvarchar`	`char`, `varchar` (UTF-8 collation)
`money`, `smallmoney`	`decimal`
`datetime`, `smalldatetime`	`datetime2` (up to 6 fractional digits)
`tinyint`	`smallint`
`text`, `ntext`, `json`	`varchar`

`varchar(max)` is limited to 16 MB per value.

Some features are absent or in preview. Triggers are not supported. `ALTER COLUMN` and nested CTEs are in preview. Check the T-SQL surface area page before you port a procedure that relies on them.

Porting a SQL Server script is usually a type-mapping job plus one design change: replace constraint enforcement with explicit checks.

Why it matters

A team ported a customer dimension from SQL Server, where `PRIMARY KEY` had rejected duplicate loads for years. In Fabric the same key was created `NOT ENFORCED`, so a pipeline that ran twice on the same file loaded every customer twice. Sales per customer halved in the joined reports for a week before anyone traced it. A duplicate check after each load would have failed the run on the first rerun.

Worked example: weak code

```
1 CREATE TABLE dbo.dim_customer (
2     customer_id INT PRIMARY KEY,
3     name NVARCHAR(200),
4     credit_limit MONEY,
5     created_at DATETIME
6 );
```

1. **Line 2.** An enforced clustered primary key is not supported in Fabric Warehouse.
2. **Line 3.** nvarchar is not supported. Use varchar with the UTF-8 collation.
3. **Line 4.** money is not supported. Use decimal.
4. **Line 5.** datetime is not supported. Use datetime2.

Worked example: strong code

```
1 CREATE TABLE dbo.dim_customer (
2     customer_id INT NOT NULL,
3     name VARCHAR(200),
4     credit_limit DECIMAL(19, 4),
5     created_at DATETIME2(6)
```

```
);
ALTER TABLE dbo.dim_customer
    ADD CONSTRAINT pk_dim_customer PRIMARY KEY NONCLUSTERED (customer_id)
    NOT ENFORCED;

-- The key is a hint, so check it after every load.
SELECT customer_id, COUNT(*) AS n
FROM dbo.dim_customer
GROUP BY customer_id
HAVING COUNT(*) > 1;
```

1. **Line 4.** decimal replaces money, with the scale stated.
2. **Line 8.** The only supported form of primary key: a hint for the optimiser, not a guard.
3. **Line 14.** The duplicate check the key no longer performs. Any row returned fails the load.

Map the types, declare keys as NOT ENFORCED hints, and check uniqueness yourself after every load.

Recap card

- **Primary keys in Fabric Warehouse** Only PRIMARY KEY NONCLUSTERED ... NOT ENFORCED. Duplicates are not blocked.
- **nvarchar, money, datetime in Fabric Warehouse** Not supported. Use varchar, decimal, datetime2.
- **MERGE and TRUNCATE in Fabric Warehouse** Both supported. MERGE is generally available.
- **Triggers in Fabric Warehouse** Not supported. Put the logic in the loading procedure.

Practice

Exercise 1. Multiple choice (level 1)

A Fabric Warehouse table has `PRIMARY KEY NONCLUSTERED (customer_id) NOT ENFORCED`. You insert two rows with `customer_id = 7`. What happens?

- a) The second insert fails with a key violation.
- b) Both rows are stored. The key is not enforced.
- c) The second row replaces the first.
- d) The table is locked until you remove a duplicate.

Why?

- r1) NOT ENFORCED keys are optimiser hints. The engine does not check them.

- r2) Fabric deduplicates later in a background job.

Exercise 2. Fill the gap (level 2)

Add the only supported form of primary key to `dbo.dim_customer`.

```
ALTER TABLE dbo.dim_customer
    ADD CONSTRAINT pk_dim_customer PRIMARY KEY _____(1) (customer_id)
    _____(2);
```

Exercise 3. Spot the bug (level 3)

This table definition, ported from SQL Server, fails in Fabric Warehouse. Tap one failing line, then rewrite the definition with supported types.

```
CREATE TABLE dbo.fact_payment (
    payment_id BIGINT NOT NULL,
    customer_id INT NOT NULL,
    amount MONEY NOT NULL,
    note NVARCHAR(500),
    paid_at DATETIME2(6) NOT NULL
);
```

Write down the line number of the bug. Then write the fix.

Exercise 4. Write the function (level 3)

Write a query that returns every `customer_id` appearing more than once in `dbo.dim_customer`, with its count as `n`. The load job fails if it returns any row.

Shape of the answer. The reference answer follows this outline. Replace each ... with your own code.

```
SELECT ...
FROM ...
GROUP BY ...
HAVING ...
```

Starter code:

```
SELECT DISTINCT customer_id FROM dbo.dim_customer;
```

Exercise 5. Architecture design (level 4)

You port a SQL Server data mart to Fabric Warehouse. Decide how to handle each feature.

Options: NOT ENFORCED keys plus an explicit duplicate check after each load, NOT ENFORCED keys, trusting the source.

1. Primary keys that used to block duplicates
2. An audit trigger on the orders table
3. nvarchar and money columns

Sources

- Identity columns, TRUNCATE TABLE and MERGE (GA) are supported; triggers are not; ALTER COLUMN and nested CTEs are in preview. (verified: https://learn.microsoft.com/en-us/fabric/data-warehouse/tsql-surface-area (updated 2026-08-26))
- PRIMARY KEY and UNIQUE are supported only as NONCLUSTERED NOT ENFORCED, FOREIGN KEY only as NOT ENFORCED. (verified: https://learn.microsoft.com/en-us/fabric/data-warehouse/table-constraints (updated 2026-09-08))
- Unsupported types and alternatives: nchar/nvarchar to char/varchar, money to decimal, datetime to datetime2, tinyint to smallint; varchar(max) limited to 16 MB; datetime2 limited to 6 fractional digits. (verified: https://learn.microsoft.com/en-us/fabric/data-warehouse/data-types (updated 2026-06-11))

C2-L3 Lakehouse, Warehouse or SQL analytics endpoint

Concept

Fabric gives you three ways to hold and query tables. All three store data as Delta tables in OneLake. They differ in who writes the data and in which language.

Lakehouse. You write with Spark: notebooks, Spark job definitions, Dataflows Gen2 or pipelines. It holds files and Delta tables side by side. Choose it for raw files, PySpark transformations and data science. This is the closest match to your Databricks work.

SQL analytics endpoint. Every lakehouse gets one automatically. It gives a **read-only** T-SQL surface over the lakehouse's Delta tables. You can `SELECT`, and you can create views, functions and stored procedures. You cannot `INSERT`, `UPDATE` or `DELETE` table data through it. It picks up new or changed tables through a metadata sync. Microsoft documents the usual lag as under one minute, but it varies.

Warehouse. You write with T-SQL. It supports full DML and DDL, and multi-table transactions. Choose it when a team works in T-SQL, loads data with stored procedures, or needs several tables to change together or not at all.

One rule decides most cases: **who writes the data, and in which language?** Spark writers go to a lakehouse. T-SQL writers go to a warehouse. T-SQL readers of lakehouse data use the SQL analytics endpoint.

You do not have to copy data between them. A warehouse query reads lakehouse tables in the same workspace with a three-part name: `lakehouse_name.schema.table`. Direct Lake semantic models read from either.

Check the status of each feature you rely on in VERSIONS.md before you build. Fabric changes monthly.

Why it matters

A team moves a finance reconciliation from SQL Server to Fabric. They land every table in a lakehouse, because the Databricks team already uses lakehouses. The month-end stored procedure updates three tables. On the SQL analytics endpoint it fails on its first `UPDATE`, because the endpoint is read-only.

Under deadline, someone rewrites it as three separate Spark notebook writes. On one month-end the second notebook fails. Table one holds the new month and table three holds the old one, and the reconciliation report shows a balance that does not tie. A warehouse transaction would have rolled back all three tables together.

Worked example: weak code

```
1 -- Connected to the SQL analytics endpoint of lakehouse lh_finance
2 CREATE PROCEDURE dbo.close_month @period CHAR(7) AS
3 BEGIN
2     UPDATE dbo.ledger SET status = 'closed' WHERE period = @period;
5     INSERT INTO dbo.balances
6     SELECT account_id, @period, SUM(amount) FROM dbo.ledger
7     WHERE period = @period GROUP BY account_id;
3     DELETE FROM dbo.open_items WHERE period = @period;
9 END;
```

1. **Line 1.** The endpoint gives read-only access to lakehouse tables. Every write below fails.
2. **Line 4.** UPDATE, INSERT and DELETE on lakehouse tables are not supported through the endpoint.
3. **Line 8.** Three tables must change together. Without a transaction, a failure halfway leaves them inconsistent.

Worked example: strong code

```
1 -- Connected to warehouse wh_finance
2 CREATE PROCEDURE dbo.close_month @period CHAR(7) AS
3 BEGIN
2     BEGIN TRANSACTION;
5     UPDATE dbo.ledger SET status = 'closed' WHERE period = @period;
3     INSERT INTO dbo.balances (account_id, period, balance)
```

```
    SELECT account_id, @period, SUM(amount) FROM dbo.ledger
    WHERE period = @period GROUP BY account_id;
    DELETE FROM dbo.open_items WHERE period = @period;
    COMMIT TRANSACTION;
END;
```

1. **Line 1.** The T-SQL writer works in a warehouse, which supports full DML.
2. **Line 4.** All three changes commit together or not at all. Warehouses support multi-table transactions.
3. **Line 6.** Name the target columns. An INSERT without a column list breaks when someone adds a column.

The data and the logic are the same. The item type decides whether the procedure runs at all, and whether a failure leaves the tables consistent.

Recap card

- **Lakehouse, Warehouse or endpoint: the deciding question** Who writes the data, and in which language? Spark writes to a lakehouse, T-SQL writes to a warehouse.
- **What can you do on the SQL analytics endpoint?** SELECT and create views, functions and procedures. No INSERT, UPDATE or DELETE on table data.
- **When do you need a Warehouse over a Lakehouse?** T-SQL writes, stored procedure loads, and multi-table transactions.
- **Read a lakehouse table from a warehouse in the same workspace** Use a three-part name: lakehouse.schema.table. No copy.
- **A new lakehouse table is missing from the endpoint** Metadata sync lag. Usually under a minute. Refresh or retry before dependent steps.

Practice

Exercise 1. Multiple choice (level 1)

Which statement about a lakehouse's SQL analytics endpoint is true?

- a) You can run UPDATE on lakehouse tables through it.
- b) You can create a view over lakehouse tables in it.
- c) You create it by hand after you create the lakehouse.
- d) It stores its own copy of the lakehouse tables.

Why?

- r1) Views are T-SQL objects that read data. The endpoint allows objects that read, not statements that change table data.

- r2) Views are stored inside the Delta tables.

Exercise 2. Architecture design (level 2)

Pick the Fabric item for each workload.

Options: Lakehouse, Warehouse, SQL analytics endpoint.

1. Land raw JSON survey deliveries and clean them with PySpark
2. Finance loads budgets with T-SQL stored procedures that change several tables together
3. Analysts query the cleaned lakehouse tables with T-SQL and save a few views

Exercise 3. Spot the bug (level 3)

This script runs against the SQL analytics endpoint of `lh_research`. It fails partway. Tap the failing line, then rewrite the script so it runs on the endpoint.

```
1 -- Connected to the SQL analytics endpoint of lh_research
2 CREATE VIEW dbo.v_wave_reach AS
3 SELECT wave_id, SUM(weighted_reach) / SUM(weighted_base) AS reach_pct
4 FROM dbo.nat_metrics
5 GROUP BY wave_id;
6 GO
7 UPDATE dbo.nat_metrics SET weighted_reach = 0 WHERE wave_id = 'W00';
```

Write down the line number of the bug. Then write the fix.

Exercise 4. Write the function (level 3)

On the SQL analytics endpoint, write a view `dbo.v_wave_reach` with one row per `wave_id` and a column `reach_pct`: the sum of `weighted_reach` divided by the sum of `weighted_base` from `dbo.nat_metrics`. A wave with a zero base must return NULL, not a divide-by-zero error.

What you need to know.

`NULLIF(a, b)` returns NULL when `a` equals `b`, and `a` otherwise. Any arithmetic with NULL gives NULL, so dividing by `NULLIF(x, 0)` turns a zero divisor into a NULL result instead of an error:

```
1 SELECT 10 / NULLIF(0, 0);   -- NULL
2 SELECT 10 / NULLIF(5, 0);   -- 2
```

`CREATE VIEW name AS SELECT ...` saves a query under a name. Reading the view runs the query.

Shape of the answer. The reference answer follows this outline. Replace each ... with your own code.

```
CREATE VIEW ...
SELECT
    ...
FROM ...
GROUP BY ...
```

Starter code:

```
CREATE VIEW dbo.v_wave_reach AS
```

Exercise 5. Fill the gap (level 4)

You are connected to warehouse `wh_finance`. Lakehouse `lh_research` sits in the same workspace. Complete the query so it reads the lakehouse table without copying it.

```
INSERT INTO dbo.wave_summary (wave_id, weighted_base)
SELECT wave_id, SUM(weighted_base)
FROM _____(1).dbo.nat_metrics
GROUP BY wave_id;
```

Exercise 6. Multiple choice (level 5)

A pipeline runs a Spark notebook that creates a new gold table in `lh_research`. The next pipeline step immediately queries the table through the SQL analytics endpoint and fails with "Invalid object name". An hour later the same query works. What is the most likely cause?

- a) The notebook wrote the table in a format the endpoint cannot read.
- b) The endpoint had not yet synced the new table's metadata.
- c) The endpoint needs a manual refresh every time a table changes.
- d) The pipeline's identity lacks permission on new tables for one hour.

Why?

- r1) The endpoint discovers new Delta tables through an automatic sync that usually takes under a minute but has no fixed limit.
- r2) The endpoint copies data from the lakehouse on a schedule.

Practical: Lakehouse, endpoint and warehouse side by side

Platform: Fabric trial workspace. Run this in a Fabric trial workspace. You create a lakehouse and a warehouse, write one table with Spark, read it through the endpoint, and see a write fail there.

1. In your trial workspace, create a lakehouse named `lh_research`.
2. Open a new notebook attached to `lh_research`. Run:
   `spark.createDataFrame([("W01", 120.0, 400.0), ("W02", 0.0, 0.0)],`

`"wave_id STRING, weighted_reach DOUBLE, weighted_base DOUBLE").write.mode("overwrite").saveAsTable("nat_metrics")`.

3. Switch the lakehouse view to its SQL analytics endpoint. Wait for `nat_metrics` to appear. Note how long it takes.
4. Open a new SQL query on the endpoint and create the `dbo.v_wave_reach` view from exercise 4. Query it. Wave W02 returns NULL.
5. In the same query window, run `UPDATE dbo.nat_metrics SET weighted_reach = 0 WHERE wave_id = 'W01';` and read the error message.
6. Create a warehouse named `wh_finance`. In a new query, run `SELECT * FROM lh_research.dbo.nat_metrics;`. It returns the rows without a copy.
7. Write down what you saw in each step. Compare it with the recap card.

Sources

- The SQL analytics endpoint is read-only for Delta table data. No INSERT, UPDATE or DELETE. (verified: https://learn.microsoft.com/en-us/fabric/data-engineering/lakehouse-sql-analytics-endpoint (updated 2026-05-19))
- You can create views, functions and stored procedures in the SQL analytics endpoint. (verified: https://learn.microsoft.com/en-us/fabric/data-engineering/lakehouse-sql-analytics-endpoint (updated 2026-05-19))
- Fabric Warehouse supports explicit multi-table transactions with BEGIN TRAN, COMMIT TRAN and ROLLBACK TRAN. (verified: https://learn.microsoft.com/en-us/fabric/data-warehouse/transactions (updated 2026-06-03))
- Metadata sync runs in the background, and you can refresh it from the portal, the REST API or sys.sp_dw_refresh_ext_table. (verified: https://learn.microsoft.com/en-us/fabric/data-engineering/sql-analytics-endpoint-metadata-sync (updated 2026-05-29))
- A warehouse reads lakehouse tables in the same workspace with three-part names, and transactions can span them. (verified: https://learn.microsoft.com/en-us/fabric/data-warehouse/query-warehouse and .../transactions)
- Under normal conditions the lag between a lakehouse and its SQL analytics endpoint is under one minute; the background sync stops after 15 minutes without query activity. (verified: https://learn.microsoft.com/en-us/fabric/data-engineering/sql-analytics-endpoint-performance (updated 2026-08-07))

C3 Unifying datasets

Roadmap: Months 16 to 18: MLOps and deployment; Current work. Shortcuts, mirroring (including Azure Databricks Unity Catalog) and avoiding duplicate copies.

C3-L1 OneLake shortcuts, and the delete that goes through

Concept

A **shortcut** is a folder in a lakehouse (or KQL database) that points to data somewhere else. It behaves like a symbolic link. The target can be inside OneLake, such as a table in another workspace, or outside it: ADLS Gen2, Azure Blob Storage, Amazon S3 and S3-compatible storage, Google Cloud Storage, Dataverse, OneDrive or SharePoint.

Shortcuts let every Fabric engine read the data **without copying it**. OneLake handles the connection and credentials, so each workload does not need its own.

Where you create the shortcut matters:

- Under **Tables**, point it at Delta data. The lakehouse recognises the folder as a table, and the SQL analytics endpoint and Direct Lake can use it.
- Under **Files**, it can point at data in any format.

Deletes are the trap. Microsoft Learn states the rule plainly:

- Deleting the **shortcut itself** deletes only the shortcut. The target data stays.
- Deleting a **file or folder inside** the shortcut deletes it **in the target**, if your identity has write permission there.

So a clean-up notebook that runs `rm -r` on `Files/partner_feed/2025/` removes the partner's 2025 files from their storage account, not a local copy.

Other points from Learn (checked September 2026):

- Moving, renaming or deleting the target breaks the shortcut. Broken shortcuts appear in the Unidentified area.
- For external shortcuts you can turn on **caching**. OneLake keeps files it has read for 1 to 28 days, which cuts cross-cloud egress costs.

- With Direct Lake over SQL or T-SQL in delegated identity mode, the calling user's identity is not passed through to the target. Check which identity reaches the source before relying on its permissions.

Why it matters

A lakehouse held a shortcut `Files/finance_share` to a folder in the finance team's storage account. An analyst ran a notebook to free up space: it deleted every subfolder older than two years under `Files/`. Their identity had write access to the finance account for another project. Seven years of finance exports disappeared from the source. The storage account had no soft delete, and the files were gone.

Worked example: weak code

```
1 for path in notebookutils.fs.ls("Files/"):
2     if path.name.startswith("2023"):
2         notebookutils.fs.rm(path.path, True)
```

1. **Line 1.** Walks every folder under Files, shortcuts included.
2. **Line 3.** Recursive delete inside a shortcut deletes the target data, if the identity can write there.

Worked example: strong code

```
1 OWNED_PREFIXES = ("Files/landing/", "Files/tmp/")
2
3
4 def safe_to_delete(path: str, shortcut_roots: set[str]) -> bool:
5     """True only for paths this lakehouse owns, never under a shortcut."""
2     if any(path == root or path.startswith(root.rstrip("/") + "/") for
      root in shortcut_roots):
7         return False
8     return path.startswith(OWNED_PREFIXES)
```

1. **Line 1.** An allowlist of folders this lakehouse owns.
2. **Line 6.** Anything under a shortcut is refused, whatever its name.

Shortcuts save copies. Treat every path under a shortcut as someone else's data.

Recap card

- **What a shortcut is** A folder that points to data in OneLake or external storage, without copying it.
- **Delete the shortcut vs delete inside it** Deleting the shortcut keeps the data. Deleting files inside it deletes them in the target, if you can write there.

- **Shortcut as a table** Create it under Tables, pointing at Delta data.
- **External shortcut caching** OneLake caches read files for 1 to 28 days to cut egress.

Practice

Exercise 1. Multiple choice (level 1)

`Files/partner_feed` is a shortcut to an ADLS Gen2 folder. You delete the shortcut `Files/partner_feed` itself. What happens to the data in ADLS?

- a) It is deleted too.
- b) It stays. Deleting a shortcut deletes only the shortcut object.
- c) It is moved to the lakehouse recycle bin.
- d) It is copied into OneLake before the shortcut goes.

Why?

- r1) Shortcuts do not cascade deletes. The shortcut is an independent object.
- r2) ADLS data can never be deleted from Fabric.

Exercise 2. Multiple choice (level 2)

Your identity has write access to the target storage. You delete `Files/partner_feed/2023/` where `partner_feed` is a shortcut. What happens?

- a) Only the lakehouse view of 2023 is hidden.
- b) The 2023 folder is deleted in the target storage account.
- c) The delete fails: shortcuts are read-only.
- d) The shortcut breaks, but the data stays.

Why?

- r1) Deleting a file or folder within a shortcut deletes it in the target when you have permission there.
- r2) Fabric syncs deletes back every night.

Exercise 3. Fill the gap (level 2)

Complete the rule for where to create a shortcut so the SQL analytics endpoint sees it as a table.

```
1 Create the shortcut under _____(1), pointing at data in _____(2) format.
```

Exercise 4. Write the function (level 3)

Write `safe_to_delete(path: str, shortcut_roots: set[str]) -> bool` for a clean-up job. Return `False` if `path` is a shortcut root or lies under one. Otherwise return `True` only if it starts with `"Files/landing/"` or `"Files/tmp/"`. Treat `"Files/partner"` and `"Files/partner_old/x"` as different folders.

Your code must pass these checks:

- Anything at or under a shortcut root must be refused.
- Paths under Files/landing/ or Files/tmp/ that are not under a shortcut are safe.
- Match whole folder names: Files/partner does not contain Files/partner_old, and Files/landing_old is not owned.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```
# Module constant.
OWNED_PREFIXES = ...

# True only for owned paths that are not under a shortcut.
def safe_to_delete(
    path: str,
    shortcut_roots: set[str],
) -> bool: ...
```

Starter code:

```
def safe_to_delete(path, shortcut_roots):
    return path.startswith("Files/")
```

Exercise 5. Architecture design (level 4)

Marketing's campaign data lives in an S3 bucket, updated daily. Your lakehouse needs it for Spark and SQL. Choose the set-up.

Options: An S3 shortcut, A nightly copy pipeline into Files.

1. How the data reaches the lakehouse
2. Where the shortcut sits, given the data is Delta
3. Cross-cloud reads
4. Identity used by the connection

Sources

- Shortcut targets, Tables vs Files behaviour, no cascading deletes, deletes inside a shortcut reach the target with permission, caching 1 to 28 days, delegated identity not passed through. (verified: https://

learn.microsoft.com/en-us/fabric/onelake/onelake-shortcuts (updated 2026-07-13))

- notebookutils.fs.ls(path) lists a directory and notebookutils.fs.rm(path, recurse) deletes; MSSparkUtils was renamed NotebookUtils. (verified: https://learn.microsoft.com/en-us/fabric/data-engineering/notebookutils/notebookutils-file-system and .../notebook-utilities)

C3-L2 Mirroring operational databases into OneLake

Concept

Mirroring brings an external database into Fabric as a unit. For sources that store data in their own formats, such as Azure SQL Database, Azure SQL Managed Instance, Azure Cosmos DB, Snowflake or Google BigQuery, Fabric **continuously replicates** the chosen tables into OneLake as Delta tables, in near real time. You get a mirrored database item and a **SQL analytics endpoint** over it, read-only, like a lakehouse's.

Shortcut or mirroring? Microsoft Learn's rule:

- **Shortcut** when you want specific data, already in an open format, in your OneLake view without copying it.
- **Mirroring** when you want an external database or catalog, and the data behind it, in Fabric as a unit. It replicates when the source uses a proprietary format.

Cost. For database mirroring and open mirroring, the background compute that replicates data does not consume capacity. Mirrored storage is free up to **1 TB per capacity unit**: an F64 includes 64 TB of mirroring storage. Queries on the mirrored data (Spark, SQL, Power BI) use capacity as usual.

Limits come per source. For Azure SQL Database, checked September 2026:

- at most 1,000 mirrored tables per database,
- tables whose primary key uses `datetime2(7)`, `datetimeoffset(7)` or `time(7)` cannot be mirrored,
- since April 2025 tables without a primary key can be mirrored, but existing ones are not added automatically, even with "Automatically mirror future tables" turned on.

Status varies too. Most sources are generally available. Some, such as Azure Database for MySQL, Dremio and SharePoint List, were in preview in

September 2026. Check the mirroring overview page before you build on a source.

After you set up a mirror, compare row counts per table between source and mirror (lesson A6-L3). The limits above fail quietly: an ineligible table is simply not there.

Why it matters

A team mirrored an Azure SQL order database and built a dashboard on it. Weeks later, finance noticed that returns were missing. The returns table's primary key included a `datetime2(7)` column, so it had never been mirrored. Nothing errored. A per-table row-count check between source and mirror on the first day would have listed it as absent.

Worked example: weak code

```
1 -- Mirror set up for all tables. Dashboard built on the mirror.
2 SELECT COUNT(*) FROM mirror_orders.dbo.orders;
```

1. **Line 1.** Assumes every table was eligible and replicated.

Worked example: strong code

```
1  -- Run in the source database: tables and row counts
1  SELECT s.name AS schema_name, t.name AS table_name, SUM(p.rows) AS
     source_rows
3  FROM sys.tables AS t
4  JOIN sys.schemas AS s ON s.schema_id = t.schema_id
5  JOIN sys.partitions AS p ON p.object_id = t.object_id AND p.index_id IN
     (0, 1)
6  GROUP BY s.name, t.name;
7
8  -- Run on the mirror's SQL analytics endpoint: which tables arrived
9  SELECT TABLE_SCHEMA, TABLE_NAME
2  FROM INFORMATION_SCHEMA.TABLES
11 WHERE TABLE_TYPE = 'BASE TABLE';
```

1. **Line 2.** The expected list and sizes, from the source.
2. **Line 10.** The actual list in the mirror. Any table missing here was not mirrored.

Mirroring replicates operational databases for free compute and generous storage, within per-source limits. Reconcile the table list after setup.

Recap card

- **Shortcut or mirroring?** Shortcut for open-format data in storage. Mirroring for databases and catalogs, replicated when proprietary.

- **Mirroring cost** Replication compute is free. Storage is free up to 1 TB per CU. Queries use capacity.
- **Azure SQL Database mirroring limits** Up to 1,000 tables. Primary keys on datetime2(7), datetimeoffset(7) or time(7) block a table.
- **After setting up a mirror** Reconcile table lists and row counts with the source.

Practice

Exercise 1. Multiple choice (level 1)

You need an Azure SQL Database's tables in Fabric, kept up to date, for reporting. Shortcut or mirroring?

- a) Shortcut: shortcuts connect to Azure SQL Database.
- b) Mirroring: it replicates the database into OneLake continuously.
- c) Neither: export CSV files nightly.
- d) A shortcut under Tables.

Why?

- r1) Azure SQL stores data in a proprietary format, so Fabric must replicate it, which is what mirroring does.
- r2) Mirroring is always cheaper than shortcuts.

Exercise 2. Predict the output (level 2)

Using the documented rule of 1 TB of free mirroring storage per capacity unit, what does this print?

```
capacity_units = 32
mirrored_tb = 45
free_tb = capacity_units * 1
print(free_tb, max(0, mirrored_tb - free_tb))
```

Exercise 3. Fill the gap (level 2)

List the base tables that arrived in the mirror, on its SQL analytics endpoint.

```
SELECT TABLE_SCHEMA, TABLE_NAME
FROM _____(1).TABLES
WHERE TABLE_TYPE = '_____(2)';
```

Exercise 4. Write the function (level 3)

Write `missing_from_mirror(source: dict[str, int], mirror: dict[str, int]) -> list[str]` for a reconciliation job. Both map `schema.table` to a row count.

Return, sorted, the tables that are absent from the mirror, or whose mirror count differs from the source count.

Your code must pass these checks:

- Tables missing from the mirror must be listed.
- Tables whose counts differ must be listed too.
- Return a sorted list, and ignore tables that exist only in the mirror.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```python
# Source tables that are absent from the mirror or differ in row count.
def missing_from_mirror(
    source: dict[str, int],
    mirror: dict[str, int],
) -> list[str]: ...
```

Starter code:

```python
def missing_from_mirror(source, mirror):
    return [t for t in source if t not in mirror]
```

Exercise 5. Architecture design (level 4)

Plan mirroring for an Azure SQL order database with 1,400 tables. Some primary keys use `datetime2(7)`.

Options: The tables reporting needs, under the 1,000-table limit, All 1,400.

1. Which tables to mirror
2. Tables with datetime2(7) primary keys
3. After setup

Sources

- Shortcuts vs mirroring guidance, and mirroring replicates when the source stores data in a proprietary format. (verified: https://learn.microsoft.com/en-us/fabric/onelake/unify-data (updated 2026-07-15))
- Replication compute is free and does not consume capacity; mirroring storage is free at 1 TB per CU; MySQL, Dremio and SharePoint List sources are in preview; BigQuery is GA. (verified: https://learn.microsoft.com/en-us/fabric/mirroring/overview (updated 2026-08-28))
- Azure SQL Database mirroring: 1,000 tables maximum, datetime2(7)/datetimeoffset(7)/time(7) primary keys unsupported, tables without primary keys supported since April 2025 but not added automatically. (verified: https://learn.microsoft.com/en-us/fabric/mirroring/azure-sql-database-limitations (updated 2026-02-26))

C3-L3 Mirroring Azure Databricks Unity Catalog into Fabric

Concept

Your Track A pipelines write gold tables into Unity Catalog. **Mirroring Azure Databricks Unity Catalog** makes them readable in Fabric without copying them. Unlike database mirroring, **no data moves**. Fabric mirrors only the catalog structure (catalogs, schemas, tables) and reads the underlying Delta files through shortcuts.

You get a mirrored Azure Databricks item with a SQL analytics endpoint, and you can build Power BI reports on it in Direct Lake mode. By default, **Automatically sync future catalog changes for the selected schema** is on, so new tables appear. Propagation takes from seconds to several minutes.

Three rules decide whether it works and whether it is safe (Microsoft Learn, checked September 2026):

1. Some table types do not mirror: tables with row filters or column masks (RLS/CLM policies), Lakehouse federated tables, Delta Sharing tables, streaming tables, views and materialized views. The ADLS Gen2 account behind the Databricks workspace must also be reachable from Fabric.

2. Unity Catalog permissions do not carry over. Grants on catalogs, schemas and tables in Databricks do not apply in Fabric. You rebuild access with Fabric's model: workspace roles, item permissions, SQL permissions, OneLake security.

3. One credential reads everything. The credential used to create the connection is used for all data queries. Unity Catalog policies such as row filters, column masks or ABAC are not enforced at the storage layer, so they would not apply to direct storage access anyway.

Put together: mirror tables that are safe to share at the level you grant in Fabric. For data that needs row-level rules, publish a gold table without the policy that already contains only what the audience may see, or apply the rules again in Fabric.

Why it matters

A team mirrored their gold schema so that regional managers could build Power BI reports. In Databricks, a row filter limited each manager to their region (lesson A4-L3). The filtered table did not appear in Fabric at all, so the

team created an unfiltered copy for the mirror, with a connection that could read every row. Every manager with Viewer access now saw every region. The rules had lived in Unity Catalog, and nothing had been rebuilt in Fabric.

Worked example: weak code

```
1 Mirror: catalog gold_prod, schema sales, all tables, auto-sync on
2 Connection: service principal with SELECT on the whole catalog
3 Fabric workspace: regional managers as Viewers
3 Assumption: Unity Catalog row filters still apply
```

1. **Line 1.** Tables with row filters are not mirrored, and any new table appears automatically.
2. **Line 2.** This credential reads every mirrored table for every user.
3. **Line 4.** Unity Catalog permissions and policies do not carry over to Fabric.

Worked example: strong code

```
1 Mirror: catalog gold_prod, schema sales_shareable (tables approved for all
    managers)
2 Connection: service principal with SELECT on sales_shareable only
3 Regional detail: rebuilt in Fabric with row-level security on the semantic
    model
4 Check: row counts per mirrored table match Databricks after each release
```

1. **Line 1.** A schema of tables that are safe for the whole Fabric audience.
2. **Line 2.** Least privilege on the one credential that reads everything.
3. **Line 3.** Access rules recreated where the users are.

Mirroring Unity Catalog copies no data and no permissions. Decide what is shareable, scope the connection, and rebuild access rules in Fabric.

Recap card

- **What mirroring Unity Catalog moves** Only catalog metadata. Data is read in place through shortcuts.
- **Unity Catalog tables that do not mirror** Tables with row filters or column masks, federated, Delta Sharing and streaming tables, views and materialized views.
- **Unity Catalog permissions in Fabric** Not carried over. Rebuild access with Fabric's permission model.
- **Which identity reads mirrored data?** The connection's credential, for all queries. Scope it tightly.

Practice

Exercise 1. Multiple choice (level 1)

When Fabric mirrors an Azure Databricks Unity Catalog, what moves into OneLake?

- a) A full copy of every table's data
- b) Only the catalog structure. Data is read through shortcuts.
- c) Nightly snapshots of changed partitions
- d) Nothing. Fabric queries Databricks SQL warehouses.

Why?

- r1) Unity Catalog tables are already open Delta files, so Fabric mirrors metadata and reads the files in place.
- r2) Copying is blocked by Databricks licensing.

Exercise 2. Multiple choice (level 2)

A gold table in Unity Catalog has a row filter. You select its schema for mirroring. What happens to that table?

- a) It is mirrored with the row filter enforced in Fabric.
- b) It is mirrored without the filter.
- c) It is not mirrored: tables with RLS or CLM policies are unsupported.
- d) It is mirrored as a view.

Why?

- r1) The limitations page lists tables with RLS/CLM policies as unsupported.
- r2) Row filters make tables read-only.

Exercise 3. Put in order (level 2)

Order the steps to publish Databricks gold tables to Fabric users safely.

- Create a connection identity with SELECT on that schema only
- Compare table lists and row counts with Databricks
- Create the mirrored Azure Databricks item for that schema
- Choose tables safe for the whole Fabric audience, in a dedicated schema
- Grant Fabric access with workspace roles, SQL permissions or model RLS

Exercise 4. Write the function (level 3)

Write `mirror_eligible(tables: list[dict[str, object]]) -> list[str]`. Each dict has `name` (str), `table_type` (str) and `has_row_filter_or_mask` (bool).

Return, sorted, the names that can be mirrored: `table_type` must be `"MANAGED"` or `"EXTERNAL"`, and the table must not have a row filter or column mask. Streaming tables, views, materialized views, federated and Delta Sharing tables arrive with other `table_type` values.

Your code must pass these checks:

- Only MANAGED and EXTERNAL tables qualify, returned sorted.
- Tables with row filters or column masks are not mirrored.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```python
# Module constant.
MIRRORABLE_TYPES = ...

# Unity Catalog tables that Fabric can mirror.
def mirror_eligible(
    tables: list[dict[str, object]],
) -> list[str]: ...
```

Starter code:

```python
def mirror_eligible(tables):
    return [t["name"] for t in tables]
```

Exercise 5. Architecture design (level 4)

Regional managers must each see only their region's sales in Power BI. The data is a Unity Catalog gold table with a row filter. Choose the design.

Options: An unfiltered copy of the table, for all managers, An unfiltered table, with row-level security rebuilt in the Fabric semantic model.

1. What Fabric mirrors
2. Connection identity
3. Manager access to the mirror's SQL endpoint

Sources

- Only the Azure Databricks catalog structure is mirrored and data is accessed through shortcuts, with no data movement; auto-sync of future catalog changes is on by default; propagation takes seconds to several minutes. (verified: https://learn.microsoft.com/en-us/fabric/mirroring/azure-databricks (updated 2025-10-24))
- Unsupported table types: RLS/CLM tables, Lakehouse federated tables, Delta Sharing tables, streaming tables, views and materialized views; the Databricks ADLS account must be accessible to Fabric. (verified: https://

learn.microsoft.com/en-us/fabric/mirroring/azure-databricks-limitations (updated 2025-12-19))

- Unity Catalog permissions are not mirrored; the connection credential is used for all data queries; UC policies are not enforced at the storage layer. (verified: https://learn.microsoft.com/en-us/fabric/mirroring/azure-databricks-security (updated 2026-05-01))
- Unity Catalog information_schema.tables reports table_type values including MANAGED, EXTERNAL, VIEW, MATERIALIZED_VIEW, STREAMING_TABLE and FOREIGN. (verified: https://docs.databricks.com/aws/en/sql/language-manual/information-schema/tables)

C4 Ingestion and orchestration

Roadmap: Months 16 to 18: MLOps and deployment; Current work. Data Factory pipelines, Dataflows Gen2 and copy jobs.

C4-L1 Choosing how data gets into Fabric

Concept

Fabric offers several ways to move data. They overlap, and picking by habit creates pipelines nobody needs. Microsoft Learn's data movement guide (updated April 2026) gives a starting rule by layer:

- **Curated data from an existing system, for reporting: Mirroring.** If the source already does its own ETL and you need the result in Fabric, mirroring is the simplest and cheapest choice (lesson C3-L2).
- **Raw ingestion into bronze: Copy job.** It handles bulk copy, incremental copy and change data capture (CDC), with scheduling and state tracking built in, without building a pipeline.
- **Low-latency events: Eventstream** (lesson C7-L1).

Then add tools only where they earn their place:

- **Pipeline** (Data Factory): orchestration. Run activities in order, branch on success or failure, loop, retry, and call notebooks, stored procedures, dataflows and Copy jobs. A pipeline with a copy activity also moves data, but you manage incremental state yourself.
- **Dataflow Gen2**: Power Query, low-code transformation for analysts, with a data destination per query.
- **Spark notebook**: code-first transformation at scale, the closest match to your Databricks work (lesson C5-L1).

A simple way to decide:

1. Does the source need to be replicated as a whole database? Mirroring.
2. Is it a copy of tables or files into bronze? Copy job.
3. Are there events that must arrive within seconds? Eventstream.
4. Is there transformation? Spark for engineers, Dataflow Gen2 for low-code authors.
5. Do several steps need ordering, branching and alerts? Wrap them in a pipeline.

Two warnings. A pipeline that only copies tables re-creates what a Copy job does, with more to maintain. And Dataflow Gen2 is not a replacement for Spark on large volumes. Check your capacity usage (lesson C10-L3) before you standardise on one tool.

Why it matters

A team built 40 pipelines, each with a copy activity, a lookup of the last loaded timestamp from a control table, and a stored procedure to update it. Two of them had an off-by-one error in the watermark and silently skipped rows every night. A Copy job per source would have tracked incremental state itself, with nothing for the team to get wrong.

Worked example: weak code

```
❶ Source: Azure SQL CRM (200 tables)   -> 200 pipelines, copy activity +
     control table
❷ Source: Clickstream events            -> hourly copy activity from storage
❸ Transform: Dataflow Gen2 on 400 M rows per day
```

1. **Line 1.** Hand-built incremental state for a whole database: mirroring or a Copy job does this for you.
2. **Line 2.** Events that need low latency, handled as hourly batches.
3. **Line 3.** A low-code tool on engineering-scale volume. Spark suits this better.

Worked example: strong code

```
❶ Source: Azure SQL CRM (200 tables)   -> Mirroring (curated reporting copy)
❷ Source: Supplier files in ADLS        -> Copy job, incremental, into
     bronze
  3 Source: Clickstream events           -> Eventstream into an eventhouse
  4 Transform: Spark notebooks, bronze to gold
❸ Orchestration: one pipeline runs the notebooks in order, retries, alerts
     on failure
```

1. **Line 1.** Free replication compute, no incremental logic to maintain.
2. **Line 2.** State tracking built in.
3. **Line 5.** The pipeline orchestrates. It does not re-create copying.

Mirroring for curated databases, Copy job for bronze, Eventstream for events, Spark or Dataflow Gen2 for transformation, and pipelines to orchestrate.

Recap card

- **Microsoft's starting rule by layer** Gold from existing systems: Mirroring. Bronze ingestion: Copy job. Low-latency events: Eventstream.

- **What a pipeline is for** Orchestration: ordering, branching, loops, retries and alerts across activities.
- **Spark or Dataflow Gen2?** Spark for code-first, large-volume engineering. Dataflow Gen2 for low-code Power Query authors.
- **Copy activity vs Copy job for incremental loads** Copy activity needs hand-built state. Copy job tracks the last successful run itself.

Practice

Exercise 1. Multiple choice (level 1)

You need raw CSV deliveries from ADLS copied into a bronze lakehouse every hour, only new files each time. Which tool does Microsoft Learn recommend starting with?

- a) Mirroring
- b) Copy job
- c) Dataflow Gen2
- d) A pipeline with a copy activity and a control table

Why?

- r1) Copy job supports incremental copy and scheduling natively for bronze ingestion.
- r2) Copy job is the only tool that reads ADLS.

Exercise 2. Architecture design (level 2)

Pick the tool for each need.

Options: Mirroring, Copy job, Eventstream, Spark notebook, Pipeline.

1. An ERP database's curated tables, kept current for reporting
2. Sensor events that must trigger alerts within seconds
3. Joining and cleaning 300 M rows a day into silver
4. Run copy, then three notebooks in order, and email on failure

Exercise 3. Put in order (level 2)

Order the questions in the decision sequence from this lesson.

- Transformation needed, and by whom?
- Several steps that need ordering and alerts?
- Copy tables or files into bronze?
- Replicate a whole database or catalog?
- Events that must arrive within seconds?

Exercise 4. Write the function (level 3)

Write `recommend(need: str) -> str` encoding the decision rule. Accepted needs and answers:

- `"replicate_database"` → `"Mirroring"`
- `"bronze_copy"` → `"Copy job"`
- `"low_latency_events"` → `"Eventstream"`
- `"code_transform"` → `"Spark notebook"`
- `"lowcode_transform"` → `"Dataflow Gen2"`
- `"orchestrate"` → `"Pipeline"`

Raise `ValueError` for anything else, with the unknown need in the message.

Your code must pass these checks:

- Each need maps to one tool, as listed.
- Unknown needs must raise ValueError naming the need.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```
# Module constant.
TOOL_FOR_NEED = ...

# Fabric tool for a data movement or processing need.
def recommend(need: str) -> str: ...
```

Starter code:

```
def recommend(need):
    return "Pipeline"
```

Exercise 5. Multiple choice (level 4)

A team copies 60 tables from Azure SQL into bronze with 60 pipelines, each using a copy activity, a control table for the last watermark, and a stored procedure to update it. Two tables lose rows every night. What change removes the whole class of bug?

- a) Add a retry to each copy activity.
- b) Move the tables to Copy jobs with incremental copy, which track state themselves.
- c) Run the pipelines twice a night.
- d) Replace the stored procedure with a Dataflow Gen2.

Why?

- r1) Copy job manages the state of the last successful run, so there is no hand-written watermark logic to get wrong.

- r2) Pipelines cannot copy incrementally.

Sources

- Data movement guide: Mirroring for gold from existing systems, Copy job for bronze ingestion, Eventstream for low-latency events; copy activity requires managing incremental state yourself. (verified: https://learn.microsoft.com/en-us/fabric/data-factory/decision-guide-data-movement (updated 2026-04-13))
- Copy job supports full copy, watermark-based and CDC-based incremental copy, and tracks the state of the last successful run. (verified: https://learn.microsoft.com/en-us/fabric/data-factory/what-is-copy-job (updated 2026-09-17))

C4-L2 Incremental copy with a watermark, or with CDC

Concept

A **Copy job** copies data in one of two modes. **Full copy** moves everything on every run. **Incremental copy** moves everything on the first run, then only new or changed data. The job tracks the state of the last successful run itself (Microsoft Learn, checked September 2026).

Incremental copy works in one of two ways:

- **Watermark.** You pick an incremental column, such as a `ROWVERSION`, a datetime, a date or an increasing integer. Each subsequent load copies rows whose value is **larger than any value copied before**.
- **CDC** (preview in Copy job). If change data capture is enabled on the source database, the job reads its change stream and copies every insert, update and delete since the last run. You do not choose a column.

The watermark method has two blind spots, and both follow from "larger than":

- It **cannot see deletes**. A deleted row simply stops appearing, and the destination keeps it.
- It **misses late rows**. A row committed after the run, with a watermark value at or below the one already copied, is never picked up. A `modified_at` set by the application clock, or several rows written in the same second, causes this.

Prefer CDC when you need deletes, or a `ROWVERSION` column, which the database increments on every change. Microsoft also lets you reset incremental copy to a full copy, for the whole job or per table, to recover.

Choose the **update method** for the destination on purpose. The default is **Append**, which keeps full history, and appends every changed row again as a new row. **Merge** updates existing rows by key (the primary key by default), **Overwrite** replaces the data, and **SCD Type 2** keeps versioned history. For a silver table that must hold one row per key, choose Merge.

Why it matters

An order table was copied incrementally on `modified_at`, set by the ordering app. At 23:59:59 one batch committed after the nightly run had read up to 23:59:59. Those rows carried the same timestamp as the last copied row, so they were never picked up. Finance found 212 missing orders at month end. CDC, or a `ROWVERSION` column, would have caught them.

Worked example: weak code

```
1  def next_batch(rows, last_wm):
(1)    batch = [r for r in rows if r["modified_at"] >= last_wm]
(2)    return batch, max(r["modified_at"] for r in batch)
```

1. **Line 2.**

   = re-copies the last run's boundary rows every time, which Append turns into duplicates.

2. **Line 3.** Fails with ValueError on a run with no new rows.

Worked example: strong code

```
1  Row = dict[str, int]
2
3
4  def next_batch(rows: list[Row], last_wm: int | None) -> tuple[list[Row],
     int | None]:
5      """Rows with a watermark above last_wm, and the new watermark."""
(1)    batch = [r for r in rows if last_wm is None or r["rv"] > last_wm]
7      if not batch:
(2)        return [], last_wm
9      return batch, max(r["rv"] for r in batch)
```

1. **Line 6.** Strictly greater, as Copy job does, on a ROWVERSION-style column.
2. **Line 8.** An empty run keeps the old watermark instead of failing.

Watermarks copy rows above the last value seen: no deletes, no late rows. Use CDC or a ROWVERSION column, and Merge for one row per key.

Recap card

- **Copy job incremental copy** First run copies everything. Later runs copy new or changed data. State is tracked by the job.
- **Watermark blind spots** Cannot see deletes. Misses late rows at or below the stored watermark.
- **When to use CDC** When deletes must replicate, or changes arrive out of order.
- **Copy job update methods** Append (default), Merge, Overwrite, SCD Type 2. Merge for one row per key.

Practice

Exercise 1. Multiple choice (level 1)

A Copy job uses watermark-based incremental copy on `modified_at`. A row is deleted in the source. What happens in the destination?

- a) The row is deleted on the next run.
- b) Nothing. Watermark-based incremental copy cannot detect deletes.
- c) The row is marked with a soft-delete flag.
- d) The job fails until you reset it.

Why?

- r1) A watermark only finds rows with a larger value. A deleted row has no value to find.
- r2) Deletes are never supported by Copy job.

Exercise 2. Predict the output (level 2)

Run 1 copied up to watermark 105. A late row with `rv` 104 commits afterwards. What does run 2 copy?

```
rows = [{"id": 1, "rv": 101}, {"id": 2, "rv": 105}, {"id": 3, "rv": 104},
    {"id": 4, "rv": 107}]
last_wm = 105
batch = [r["id"] for r in rows if r["rv"] > last_wm]
print(batch)
```

Exercise 3. Fill the gap (level 2)

Complete the Copy job settings for a silver table that must hold one current row per `order_id` and reflect source deletes.

```
Incremental method: _____(1)
Update method: _____(2)
```

Exercise 4. Write the function (level 3)

Write `next_batch(rows, last_wm)` for a hand-built incremental load that behaves like Copy job's watermark method. `rows` is a list of dicts with an integer `rv`. `last_wm` is `None` before the first run. Return `(batch, new_wm)`: rows with `rv` strictly greater than `last_wm` (all rows when `last_wm` is `None`), and the largest `rv` in the batch. On an empty batch return `([], last_wm)`.

Your code must pass these checks:

- With last_wm None, copy every row and return the largest rv.
- Copy only rows strictly greater than the watermark. >= re-copies the boundary row.
- An empty batch keeps the previous watermark and must not raise.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```python
# Rows above the watermark, and the new watermark.
def next_batch(
    rows: list[Row],
    last_wm: int | None,
) -> tuple[list[Row], int | None]: ...
```

Starter code:

```python
def next_batch(rows, last_wm):
    batch = [r for r in rows if r["rv"] >= last_wm]
    return batch, max(r["rv"] for r in batch)
```

Exercise 5. Architecture design (level 4)

Configure incremental copy of an orders table into silver. Orders are updated and sometimes deleted. The source has CDC enabled.

Options: CDC, Watermark on modified_at.

1. Incremental method
2. Update method
3. After an outage caused missed changes

Sources

- Copy job modes, watermark column types, strictly larger watermark rule, CDC captures deletes, watermark cannot detect deletes, reset to full copy per job or table. (verified: https://learn.microsoft.com/en-us/fabric/data-factory/incremental-copy-job (updated 2026-09-15) and .../what-is-copy-job (updated 2026-09-17))

- Update methods Append (default), Merge (primary key by default), Overwrite and SCD Type 2. (verified: https://learn.microsoft.com/en-us/fabric/data-factory/what-is-copy-job (updated 2026-09-17))
- CDC replication in Copy job is in preview. (verified: https://learn.microsoft.com/en-us/fabric/data-factory/what-is-copy-job (updated 2026-09-17))

C4-L3 Pipelines that fail fast, retry sensibly and alert

Concept

A Data Factory **pipeline** runs activities: Copy job, notebook, stored procedure, Dataflow, If Condition, ForEach, Until, and more. By default a pipeline holds up to 120 activities. Three settings on each activity decide how a failure behaves (Microsoft Learn, checked September 2026):

- **Timeout.** How long an activity may run before it is stopped. The default is **12 hours**, and the maximum is seven days, written `D.HH:MM:SS`. A notebook that normally takes 10 minutes and hangs will hold its slot, and its capacity, for 12 hours unless you lower this.
- **Retry.** Retries happen only if you enable them. The retry count then defaults to 1, with **30 seconds** between attempts. Retry steps that fail for passing reasons, such as a busy source or a network drop. Do not retry a step that failed because its data is wrong. It will fail again, later.
- **Dependency branches.** Each activity connects to the next through a condition: **Upon Success**, **Upon Failure**, **Upon Completion** (either way) or **Upon Skip**. Put alerts and clean-up on Upon Failure. Upon Completion runs even when the step succeeded.

Parameters and expressions keep one pipeline reusable across sources and environments. Expressions start with `@`: `@pipeline().parameters.wave`. Inside a string, `@{...}` interpolates: `"landing/@{pipeline().parameters.wave}/"`. The result of interpolation is always a **string**, so `"@{pipeline().parameters.max_rows}"` passes text, not a number. When a setting needs a number, use the bare expression `@pipeline().parameters.max_rows`.

Keep business logic out of the pipeline. Transformations belong in notebooks and SQL, where they can be tested (lesson A9-L1). The pipeline decides order, retries and alerts.

Why it matters

A nightly pipeline ran a 10-minute notebook that sometimes hung on a locked source. With the default 12-hour timeout and no failure branch, each hang held its capacity until mid-morning. Downstream reports showed yesterday's data, and nobody was told. A 30-minute timeout, one retry and an Upon Failure email would have raised the alarm at 02:30 instead of 10:00.

Worked example: weak code

```
1  Activity: nb_silver_orders (notebook)
1    Timeout: default
3    Retries: not enabled
2  Activity: nb_gold_sales  depends on nb_silver_orders: Upon Completion
3  Activity: send_failure_email  depends on nb_gold_sales: Upon Completion
```

1. **Line 2.** 12 hours for a 10-minute job: a hang goes unnoticed until morning.
2. **Line 4.** Gold runs even when silver failed, publishing stale or partial data.
3. **Line 5.** The 'failure' email is sent every night, so people learn to ignore it.

Worked example: strong code

```
1  Activity: nb_silver_orders (notebook)
1    Timeout: 0.00:30:00
3    Retries: enabled, 1 retry, 60 seconds apart
2  Activity: nb_gold_sales  depends on nb_silver_orders: Upon Success
3  Activity: send_failure_email  depends on nb_silver_orders: Upon Failure
6  Activity: send_failure_email_gold  depends on nb_gold_sales: Upon Failure
```

1. **Line 2.** Three times the normal run time: a hang fails within half an hour.
2. **Line 4.** Gold runs only on good silver.
3. **Line 5.** Alerts fire only when something failed.

Set timeouts near the real run time, retry only transient failures, chain on Upon Success and alert on Upon Failure.

Recap card

- **Activity timeout default** 12 hours, maximum seven days, format D.HH:MM:SS. Set it near the real run time.
- **Activity retries** Only when enabled. Count defaults to 1, interval 30 seconds. Retry transient failures only.
- **Dependency branches** Upon Success, Upon Failure, Upon Completion (either way), Upon Skip.
- **@{...} vs @...** @{...} inside text always yields a string. A bare @ expression keeps the value's type.

Practice

Exercise 1. Multiple choice (level 1)

You leave an activity's timeout at its default. How long can it run before Fabric stops it?

- a) 30 minutes
- b) 1 hour
- c) 12 hours
- d) 7 days

Why?

- r1) The documented default timeout is 12 hours. Seven days is the maximum you can set.
- r2) Timeouts match the capacity's pause schedule.

Exercise 2. Multiple choice (level 2)

An alert activity is attached to a notebook with **Upon Completion**. When does it run?

- a) Only when the notebook fails
- b) Only when the notebook succeeds
- c) Whether the notebook succeeds or fails
- d) Only when the notebook is skipped

Why?

- r1) Upon Completion runs after success or failure. Upon Failure is the failure-only branch.
- r2) Completion means the capacity finished all jobs.

Exercise 3. Fill the gap (level 2)

Build a folder path from the pipeline parameter `wave`, and pass the numeric parameter `max_rows` as a number.

```
Folder path: landing/_____(1)pipeline().parameters.wave}/
Max rows:    _____(2)pipeline().parameters.max_rows
```

Exercise 4. Write the function (level 3)

Write `parse_timeout(value: str) -> int` for a pipeline linter. It parses the `D.HH:MM:SS` format and returns the total seconds. Raise `ValueError` if the format is wrong, if hours are above 23 or minutes or seconds above 59, or if the total exceeds 7 days.

What you need to know.

Python's `re` module matches text against a pattern:

- `re.compile(r"...")` builds a pattern once, so you reuse it.
- `\d` matches one digit, `\d{2}` exactly two, `\d+` one or more, and `\.` a literal dot.
- Brackets `( )` **capture** part of the match.
- `pattern.fullmatch(text)` matches the **whole** string and returns a match object, or `None` when the text does not fit.
- `match.groups()` returns the captured parts as strings.

```
import re

MONTH = re.compile(r"(\d{4})-(\d{2})")
MONTH.fullmatch("2026-03").groups()   # ('2026', '03')
MONTH.fullmatch("2026-03x")           # None
```

Your code must pass these checks:

- Convert days, hours, minutes and seconds to seconds.
- Reject a missing day part, missing fields, non-numbers and out-of-range hours or minutes.
- The maximum timeout is seven days.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```
# Module constant.
TIMEOUT_PATTERN = ...

# Module constant.
MAX_SECONDS = ...

# Seconds in a pipeline activity timeout written as D.HH:MM:SS.
def parse_timeout(value: str) -> int: ...
```

Starter code:

```
def parse_timeout(value):
    days, rest = value.split(".")
    h, m, s = rest.split(":")
    return int(days) * 86400 + int(h) * 3600 + int(m) * 60 + int(s)
```

Exercise 5. Spot the bug (level 4)

This pipeline publishes gold tables even when silver fails. Tap the line that causes it, then rewrite the dependencies so gold runs only on success and an alert fires on failure.

```
Activity: nb_silver_orders
```

```
  Timeout: 0.00:30:00
Activity: nb_gold_sales  depends on nb_silver_orders: Upon Completion
Activity: email_on_failure  depends on nb_silver_orders: Upon Failure
```

Write down the line number of the bug. Then write the fix.

Sources

- Activity timeout defaults to 12 hours with a seven-day maximum in D.HH:MM:SS; retries when enabled default to 1 with a 30-second interval; up to 120 activities per pipeline; UponSuccess, UponFailure, UponCompletion and UponSkip branches. (verified: https://learn.microsoft.com/en-us/fabric/data-factory/activity-overview (updated 2026-08-10))
- Pipeline expressions use @pipeline().parameters; string interpolation with @{...} always returns a string. (verified: https://learn.microsoft.com/en-us/fabric/data-factory/expression-language (updated 2024-12-18))

C5 Spark notebooks in Fabric

Roadmap: Months 16 to 18: MLOps and deployment; Current work. Differences from Databricks, environments, V-Order and Delta interoperability.

C5-L1 From Databricks notebooks to Fabric notebooks

Concept

Fabric notebooks run the same PySpark you wrote in Track A. The surroundings differ, and those differences break ported code (Microsoft Learn, checked September 2026).

Utilities. There is no `dbutils`. Fabric has **NotebookUtils**, formerly MSSparkUtils. The `mssparkutils` namespace still works but is being retired, so write `notebookutils`:

Databricks	Fabric
`dbutils.fs.ls(path)`	`notebookutils.fs.ls(path)`
`dbutils.secrets.get(scope, key)`	`notebookutils.credentials.getSecret("https://<vault>.vault.azure.net/", "name")`
`dbutils.notebook.run(path, timeout, args)`	`notebookutils.notebook.run(path, timeout_seconds, arguments)`
`dbutils.notebook.exit(value)`	`notebookutils.notebook.exit(value)`

Secrets live in **Azure Key Vault**, not in Databricks secret scopes. You need Get permission on the vault. Output that prints a secret is redacted, but do not print secrets.

Child notebooks time out at 90 seconds by default. `notebookutils.notebook.run(path, timeout_seconds=90, ...)`. A child notebook that runs for five minutes fails unless you pass a longer timeout. For several notebooks, `notebookutils.notebook.runMultiple` runs a DAG of them in parallel.

Default lakehouse. A notebook can attach a default lakehouse. Relative paths such as `Files/landing` and bare table names resolve against it. Name the lakehouse and schema in shared code (lesson C2-L1), so a changed attachment does not change what the code reads.

Inline installs. `%pip install` affects only the current session, and inline commands are **turned off by default in pipeline runs**. Put libraries in an environment instead (lesson C5-L2).

Runtime. Runtime 2.0 (Spark 4.1, Delta Lake 4.2, Python 3.13, Java 21) is GA and recommended for production. New workspaces still default to Runtime 1.3 (Spark 3.5), whose end of support has been announced. Set the runtime deliberately. It changes the Spark version your code meets.

Why it matters

A team ported a Databricks orchestration notebook that called six child notebooks with `mssparkutils.notebook.run(path)`. In development each child took under a minute. On month-end data, the largest took four minutes. Every run failed at 90 seconds with a timeout, and the error pointed at the parent notebook. Passing `timeout_seconds` sized to the real run time fixed it.

Worked example: weak code

```
❶ password = dbutils.secrets.get("prod", "sql-password")
❷ %pip install great_expectations
❸ result = mssparkutils.notebook.run("nb_silver_orders")
❹ df = spark.read.table("orders")
```

1. **Line 1.** dbutils does not exist in Fabric. Secrets come from Azure Key Vault.
2. **Line 2.** Inline installs are turned off by default in pipeline runs, and give unpinned dependencies.
3. **Line 3.** Old namespace, and a 90-second default timeout for the child.
4. **Line 4.** Depends on whichever lakehouse is attached as default, and the dbo schema.

Worked example: strong code

```
1  VAULT_URL = "https://kv-sales-prod.vault.azure.net/"
2
❶ password = notebookutils.credentials.getSecret(VAULT_URL, "sql-password")
4  result = notebookutils.notebook.run(
❷      "nb_silver_orders", timeout_seconds=1800, arguments={"wave":
       "2026-09"}
6  )
❸ df = spark.read.table("lh_sales.silver.orders")
```

1. **Line 3.** Key Vault through NotebookUtils. The vault URL belongs in config.
2. **Line 5.** A timeout sized to the child's real run time, and explicit arguments.
3. **Line 7.** Lakehouse and schema named, independent of the default attachment.

Replace dbutils with notebookutils, secrets with Key Vault, inline installs with environments, and pass real timeouts to child notebooks.

Recap card

- **Fabric replacement for dbutils** notebookutils (formerly mssparkutils).
- **Secrets in Fabric notebooks** notebookutils.credentials.getSecret(vault_url, name), from Azure Key Vault.
- **notebookutils.notebook.run default timeout** 90 seconds. Pass timeout_seconds.
- **%pip in pipeline runs** Turned off by default. Use an environment.

Practice

Exercise 1. Multiple choice (level 1)

Which call reads a secret in a Fabric notebook?

- a) `dbutils.secrets.get("prod", "sql-password")`
- b) `notebookutils.credentials.getSecret("https://kv-sales.vault.azure.net/", "sql-password")`
- c) `os.environ["SQL_PASSWORD"]` set in the workspace settings
- d) `spark.conf.get("secret.sql-password")`

Why?

- r1) Fabric reads secrets from Azure Key Vault through notebookutils.credentials.
- r2) getSecret caches the secret in the notebook.

Exercise 2. Multiple choice (level 2)

`notebookutils.notebook.run("nb_big_load")` calls a child notebook that takes four minutes. What happens?

- a) It completes after four minutes.
- b) It fails after 90 seconds, the default timeout.
- c) It fails after 12 hours.
- d) It runs in the background and returns at once.

Why?

- r1) run() has timeout_seconds with a default of 90.
- r2) Child notebooks are limited to one Spark job.

Exercise 3. Fill the gap (level 2)

Port this Databricks line to Fabric.

```
files = _____(1).fs.ls("Files/landing")
value = _____(1).notebook.run("nb_child", _____(2)=600)
```

Exercise 4. Write the function (level 3)

Write `port_line(line: str) -> str` for a migration helper. It rewrites Databricks utility calls to Fabric:

- `dbutils.fs.` → `notebookutils.fs.`
- `dbutils.notebook.` → `notebookutils.notebook.`
- `mssparkutils.` → `notebookutils.`

If the line contains `dbutils.secrets.`, raise `ValueError` with a message that mentions Key Vault, because it needs a manual change. Leave other text unchanged.

Your code must pass these checks:

- Rewrite dbutils.fs., dbutils.notebook. and mssparkutils. to notebookutils.
- Secrets need a manual change to Key Vault: raise ValueError mentioning Key Vault.
- Do not change text that only contains the word dbutils.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```
# Module constant.
REWRITES = ...

# Rewrite Databricks and legacy Fabric utility calls to notebookutils.
def port_line(line: str) -> str: ...
```

Starter code:

```
def port_line(line):
    return line.replace("dbutils", "notebookutils")
```

Exercise 5. Spot the bug (level 4)

This notebook runs from a nightly pipeline. It worked when run by hand and fails in the pipeline. Tap the line that fails there, and rewrite the notebook without it.

What you need to know.

`import pandera.pandas as pa` loads pandera's pandas API. `spark.read.table(name).toPandas()` reads a table and copies it into a pandas DataFrame on the driver. `SCHEMA.validate(df, lazy=True)` checks that frame against a pandera schema and raises one error listing every failed check, not just the first. Track B, lesson B5-L1 covers pandera in depth.

```
%pip install pandera==0.33.1
import pandera.pandas as pa
df = spark.read.table("lh_sales.silver.orders").toPandas()
SCHEMA.validate(df, lazy=True)
```

Write down the line number of the bug. Then write the fix.

Sources

- MsSparkUtils is renamed NotebookUtils; the mssparkutils namespace will be retired. (verified: https://learn.microsoft.com/en-us/fabric/data-engineering/notebook-utilities)
- notebookutils.notebook.run(path, timeout_seconds=90, arguments=None, workspace='') returns the exit value; runMultiple runs a DAG. (verified: https://learn.microsoft.com/en-us/fabric/data-engineering/notebookutils/notebookutils-notebook-run)
- notebookutils.credentials.getSecret reads from Azure Key Vault and needs Get permission; output redaction is attempted. (verified: https://learn.microsoft.com/en-us/fabric/data-engineering/notebookutils/notebookutils-credentials (updated 2026-09-23))
- Inline installs are turned off by default in pipeline runs. (verified: https://learn.microsoft.com/en-us/fabric/data-engineering/library-management (updated 2026-03-25))
- Runtime 2.0 (Spark 4.1, Delta 4.2, Python 3.13, Java 21) is GA and recommended; new workspaces default to Runtime 1.3, which is at end-of-support-announced stage. (verified: https://learn.microsoft.com/en-us/fabric/data-engineering/runtime (updated 2026-07-24))

C5-L2 Environments, resource profiles and V-Order

Concept

An **environment** is a Fabric item that fixes what a Spark session starts with: the runtime version, compute settings, libraries and small resource files. Attach it to notebooks and Spark job definitions, or make it the **workspace default** so every notebook set to "Workspace default" inherits it. Once an environment is the workspace default, only workspace admins can change its contents.

Libraries publish in one of two modes (Microsoft Learn, checked September 2026):

- **Quick mode** installs libraries when each session starts. It is flexible, and it adds one to three minutes to start-up.
- **Full mode** builds a snapshot of the dependencies once. Microsoft recommends Full mode for pipelines, scheduled runs and shared workloads that need reproducible environments. It is the Fabric equivalent of the lock file in lesson B6-L3.

Resource profiles set a tested group of Spark and Delta settings with one property. New workspaces default to `writeHeavy`, tuned for ingestion. `readHeavyForSpark` and `readHeavyForPBI` suit read-heavy work.

V-Order is a Fabric write-time layout for Parquet files. It keeps files standard, open Parquet, and speeds up reads, especially for Power BI Direct Lake and the SQL engines. The cost is slower writes: Learn cites **around 15%** on average. Since the `writeHeavy` default, **V-Order is off by default in new workspaces** (`spark.sql.parquet.vorder.default=false`).

Turn it on where reads dominate, at the narrowest useful level:

- per table: `TBLPROPERTIES("delta.parquet.vorder.enabled" = "true")`, the best choice for gold tables that Power BI reads,
- per write: the DataFrame option `parquet.vorder.enabled`,
- per session: `spark.sql.parquet.vorder.default`, or the `readHeavyForPBI` profile, which enables it.

In Runtime 1.3 and later the older `spark.sql.parquet.vorder.enable` setting is removed. Code copied from older posts that sets it does nothing.

Why it matters

A team moved its gold layer to a new Fabric workspace and noticed Power BI reports running noticeably slower than before. The old workspace had V-Order on for every write. The new one used the `writeHeavy` default, with V-Order off. Setting `delta.parquet.vorder.enabled` on the dozen gold tables, and optimising them once, restored read performance without slowing bronze and silver ingestion.

Worked example: weak code

```
spark.conf.set("spark.sql.parquet.vorder.enable", "true")
bronze.write.mode("append").saveAsTable("lh.bronze.orders")
gold.write.mode("overwrite").saveAsTable("lh.gold.sales_by_region")
```

1. **Line 1.** Removed setting in Runtime 1.3 and later. It has no effect.

2. **Line 2.** Had the setting worked, bronze would pay the write cost for reads nobody makes.

Worked example: strong code

```
1 bronze.write.mode("append").saveAsTable("lh.bronze.orders")
2
3 spark.sql("""
4     CREATE TABLE IF NOT EXISTS lh.gold.sales_by_region (region STRING,
    sales DECIMAL(19, 4))
5     USING DELTA
2     TBLPROPERTIES ("delta.parquet.vorder.enabled" = "true")
7 """)
8 gold.write.mode("overwrite").saveAsTable("lh.gold.sales_by_region")
```

1. **Line 1.** Write-heavy bronze keeps the fast default.
2. **Line 6.** V-Order only on the gold table that Power BI reads.

Pin libraries in a Full mode environment, keep the writeHeavy default for ingestion, and turn V-Order on for read-heavy gold tables.

Recap card

- **Environment publishing modes** Quick: installs at session start (adds minutes). Full: reproducible snapshot, recommended for pipelines.
- **V-Order in new workspaces** Off by default (writeHeavy profile). Write cost around 15%, faster reads.
- **Turn V-Order on for one table** TBLPROPERTIES("delta.parquet.vorder.enabled" = "true").
- **spark.sql.parquet.vorder.enable** Removed in Runtime 1.3 and later. Use spark.sql.parquet.vorder.default or table and writer options.

Practice

Exercise 1. Multiple choice (level 1)

In a workspace created in 2026 with default settings, is V-Order applied when a notebook writes a Delta table?

- a) Yes, V-Order is always on in Fabric.
- b) No. New workspaces default to the writeHeavy profile, with V-Order off.
- c) Only for tables under Files.
- d) Only when the table is larger than 1 GB.

Why?

- r1) spark.sql.parquet.vorder.default is false in new workspaces to favour write performance.

- r2) V-Order has been retired.

Exercise 2. Multiple choice (level 2)

A nightly pipeline runs notebooks that need pandera and a pinned version of a statistics library. How should the libraries be delivered?

- a) %pip install at the top of each notebook
- b) An environment in Full mode, attached to the notebooks or set as workspace default
- c) An environment in Quick mode
- d) Upload wheel files to Files and install them at runtime

Why?

- r1) Full mode builds a reproducible snapshot, and Microsoft recommends it for pipelines and scheduled runs.
- r2) Full mode is the only mode that supports pandera.

Exercise 3. Fill the gap (level 2)

Turn on V-Order for one gold table only.

```
ALTER TABLE lh.gold.sales_by_region
SET TBLPROPERTIES ("_____(1)" = "_____(2)");
```

Exercise 4. Write the function (level 3)

Write `write_gold(df, table)` that overwrites a gold Delta table with V-Order enabled for this write, using the DataFrame writer option `parquet.vorder.enabled` set to `"true"`. Do not change the session-wide default.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```
# Overwrite a read-heavy gold table with V-Order for this write only.
def write_gold(df: DataFrame, table: str) -> None: ...
```

Starter code:

```
def write_gold(df, table):
    spark.conf.set("spark.sql.parquet.vorder.default", "true")
    df.write.mode("overwrite").saveAsTable(table)
```

Exercise 5. Architecture design (level 4)

Set up Spark for a workspace with bronze and silver ingestion and a gold layer read by Power BI Direct Lake.

Options: One Full mode environment, set as workspace default, %pip in each notebook.

1. Libraries for scheduled notebooks
2. Session-wide resource profile
3. Gold tables

Sources

- Environments set runtime, compute, libraries and resources; a workspace default environment can be changed only by workspace admins. (verified: https://learn.microsoft.com/en-us/fabric/data-engineering/create-and-use-environment (updated 2026-03-25))
- Quick mode installs at session start adding one to three minutes; Full mode is recommended for pipelines and scheduled runs. (verified: https://learn.microsoft.com/en-us/fabric/data-engineering/library-management (updated 2026-03-25))
- V-Order is off by default in new workspaces (writeHeavy); write cost around 15%; table property, writer option and session default; spark.sql.parquet.vorder.enable removed in Runtime 1.3+. (verified: https://learn.microsoft.com/en-us/fabric/data-engineering/delta-optimization-and-v-order (updated 2026-03-01))
- New workspaces default to writeHeavy; readHeavyForPBI enables V-Order. (verified: https://learn.microsoft.com/en-us/fabric/data-engineering/resource-profiles-overview (updated 2026-07-23))

C5-L3 Delta tables shared between Databricks and Fabric

Concept

Delta is an open format, so a table written by Databricks can be read by Fabric through a shortcut or Unity Catalog mirroring (lessons C3-L1 and C3-L3). Open does not mean every reader supports every **table feature**. A table's features decide which Fabric experiences can read it.

From Microsoft Learn's Delta interoperability matrix (updated May 2026):

Feature	Spark Runtime 2.0	SQL analytics endpoint	Direct Lake	Python notebooks
Deletion vectors	yes	yes	yes	no

Feature	Spark Runtime 2.0	SQL analytics endpoint	Direct Lake	Python notebooks
Column mapping, name mode	yes	yes	yes	no
Column mapping, id mode	yes	no	yes	no
`TIMESTAMP_NTZ`	yes	no	no	yes
Liquid clustering	yes	yes	yes	no

Three more limits from the same page:

- **V2 checkpoints**: only Spark notebooks and Spark jobs read and write them. The lakehouse and the SQL analytics endpoint do not list such tables correctly.
- **Databricks-only features**, such as writing identity columns and Lakeflow Spark Declarative Pipelines tables, are not supported.
- Newer **Delta 4.x features**, such as collations and the variant type, work only in Spark notebooks and jobs, not in the SQL endpoint or Direct Lake.

Fabric's own writers differ too. Runtime 2.0 writes new tables at reader version 3 and writer version 7 with **deletion vectors** on by default. Runtime 1.3 writes reader 1, writer 2.

The practical rule: **the table's writer decides which features the table has, and every reader must support all of them.** For gold tables that Power BI or T-SQL users read, use timestamps with a time zone (`TIMESTAMP`) instead of `TIMESTAMP_NTZ`, avoid column mapping in id mode, and check `DESCRIBE DETAIL` for the protocol versions and table features before you publish.

Why it matters

A Databricks team stored event times as `TIMESTAMP_NTZ` in a gold table and shared it with Fabric through a shortcut. Spark in Fabric read it fine. The SQL analytics endpoint and the Direct Lake report did not show the table's time columns, so the Power BI team built a workaround copy with converted columns. One type choice in the writer created a second dataset.

Worked example: weak code

```
1 CREATE TABLE gold.events (
2     event_id STRING,
1     event_time TIMESTAMP_NTZ
4 )
```

```
(2) TBLPROPERTIES ("delta.columnMapping.mode" = "id");
```

1. **Line 3.** Not supported by the SQL analytics endpoint or Direct Lake.
2. **Line 5.** Column mapping in id mode is not supported by the SQL analytics endpoint.

Worked example: strong code

```
1  CREATE TABLE gold.events (
2      event_id STRING,
(1)    event_time TIMESTAMP
4  );
5
6  -- Before sharing: check protocol versions and table features
(2) DESCRIBE DETAIL gold.events;
```

1. **Line 3.** A time-zone-aware timestamp that every Fabric reader supports.
2. **Line 7.** Shows minReaderVersion, minWriterVersion and table features to compare with the readers' support.

Open format, but not every feature for every reader. Design shared gold tables for the weakest reader that needs them.

Recap card

- **Delta interoperability rule** Every reader must support every feature the writer put in the table.
- **TIMESTAMP_NTZ in Fabric** Spark yes. SQL analytics endpoint and Direct Lake no.
- **Runtime 2.0 default table protocol** Reader 3, writer 7, deletion vectors on. Python notebooks cannot read deletion vectors.
- **Check a table's features** DESCRIBE DETAIL: minReaderVersion, minWriterVersion, table features.

Practice

Exercise 1. Multiple choice (level 1)

A Databricks gold table uses `TIMESTAMP_NTZ` columns. Which Fabric reader handles them?

- a) The SQL analytics endpoint
- b) Direct Lake semantic models
- c) Fabric Spark Runtime 2.0
- d) All of them

Why?

- r1) The interoperability matrix lists TIMESTAMP_NTZ as supported in Spark but not in the SQL endpoint or Direct Lake.
- r2) Only Spark can read Delta tables.

Exercise 2. Multiple choice (level 2)

You create a new Delta table with a Spark notebook on Fabric Runtime 2.0 and default settings. Which reader might fail to read it?

- a) The SQL analytics endpoint
- b) A Python notebook (non-Spark) using the default Delta reader
- c) Direct Lake
- d) Fabric Spark Runtime 2.0

Why?

- r1) Runtime 2.0 writes deletion vectors by default, and Python notebooks do not support deletion vectors.
- r2) Python notebooks cannot read OneLake.

Exercise 3. Fill the gap (level 2)

Inspect a table's protocol versions and features before sharing it.

```
1 _____(1) DETAIL gold.events;
```

Exercise 4. Write the function (level 3)

Write `unsupported(reader: str, features: set[str]) -> list[str]` using this support table from the lesson. Return the table's features the reader does not support, sorted. Raise `KeyError` for an unknown reader.

- `spark_2_0`: supports all of `deletion_vectors`, `column_mapping_name`, `column_mapping_id`, `timestamp_ntz`, `liquid_clustering`.
- `sql_endpoint`: all except `column_mapping_id` and `timestamp_ntz`.
- `direct_lake`: all except `timestamp_ntz`.
- `python_notebook`: only `timestamp_ntz`.

Your code must pass these checks:

- The SQL endpoint lacks column mapping id mode and TIMESTAMP_NTZ.
- Check each reader against its supported set.
- Unknown readers must raise KeyError.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```python
# Module constant.
ALL_FEATURES = ...

# Module constant.
SUPPORTED = ...

# Table features a Fabric reader does not support.
def unsupported(
    reader: str,
    features: set[str],
) -> list[str]: ...
```

Starter code:

```python
def unsupported(reader, features):
    return []
```

Exercise 5. Architecture design (level 4)

A Databricks team publishes a gold table that Fabric users read through the SQL endpoint and Direct Lake. Choose the table design.

Options: TIMESTAMP, TIMESTAMP_NTZ.

1. Event time column
2. Column mapping
3. Before each release

Sources

- Feature support matrix for deletion vectors, column mapping, TIMESTAMP_NTZ and liquid clustering across Spark runtimes, SQL analytics endpoint, Direct Lake and Python notebooks; Runtime 2.0 writes reader 3, writer 7 with deletion vectors; V2 checkpoints, identity columns writing, Lakeflow Spark Declarative Pipelines and Delta 4.x features outside Spark are limitations. (verified: https://learn.microsoft.com/en-us/fabric/fundamentals/delta-lake-interoperability (updated 2026-05-08))

C6 Semantic models and Direct Lake

Roadmap: Months 16 to 18: MLOps and deployment; Current work. Star schemas, DAX best practices and Power BI reports.

C6-L1 Star schemas for semantic models

Concept

A Power BI semantic model works best on a **star schema**: fact tables surrounded by dimension tables. Microsoft's Power BI guidance (checked September 2026) defines the parts:

- **Fact tables** store observations or events: survey responses, orders, balances. They hold dimension key columns and numeric measure columns. The dimension keys decide the fact table's **grain**, for example one row per respondent per wave per brand.
- **Dimension tables** describe business entities: respondent, brand, region, date. Each has a unique key column and descriptive columns used to filter and group. A **date dimension** belongs in almost every model.

Relationships run **one-to-many** from each dimension to the fact table, and filters flow from the dimension to the fact. Microsoft recommends that you **minimise bi-directional relationships**, which can slow queries and confuse report users with unexpected filtering. Many-to-many relationships between facts and dimensions usually mean the grain is unclear.

Build the star in the gold layer, not in the model:

- Compute derived columns such as age bands or weighted flags in Spark or SQL, in gold tables. Direct Lake on SQL analytics endpoints does not support calculated columns, and on OneLake they are in preview with restrictions. Materialised calculated columns in Import models slow refresh.
- Give every fact table a clear grain in its docstring and table comment (lesson A2-L3).
- Use surrogate or natural keys consistently, and check they are unique in each dimension (lesson C2-L2 showed warehouse keys are not enforced).

A good model is boring: one fact table per grain, conformed dimensions shared between facts, single-direction one-to-many relationships, and measures that do the arithmetic (lesson C6-L2).

Why it matters

A brand tracker model joined a respondent table and a brand-awareness table with a bi-directional many-to-many relationship, because both held respondent_id and brand. Selecting a brand in a slicer filtered respondents, which filtered back to every brand those respondents had mentioned. Awareness for small brands showed as much higher than the survey measured. A single fact table at respondent-brand grain with one-way dimensions returned the correct figures.

Worked example: weak code

```
1 respondents (respondent_id, age, region, weight, brand_list)
2 brand_awareness (respondent_id, brand, aware)
2 Relationship: respondents[respondent_id] <->
    brand_awareness[respondent_id], many-to-many, both directions
3 Calculated column: respondents[age_band] = SWITCH(TRUE(), ...)
```

1. **Line 1.** A list column mixes grains: one respondent, many brands.
2. **Line 3.** Bi-directional many-to-many: filters flow both ways and inflate results.
3. **Line 4.** Derived column in the model. Not supported in Direct Lake on SQL endpoints.

Worked example: strong code

```
1 fact_awareness (respondent_key, brand_key, date_key, region_key, aware,
    weight)   grain: respondent x brand x wave
2 dim_respondent (respondent_key, age_band, gender)
3 dim_brand (brand_key, brand, category)
4 dim_date (date_key, date, wave, month)
5 dim_region (region_key, region, nation)
3 Relationships: each dim[key] 1 -> * fact_awareness[key], single direction
```

1. **Line 1.** One fact table with a stated grain.
2. **Line 2.** age_band computed in gold, not in the model.
3. **Line 6.** Filters flow one way, from dimensions to facts.

Facts with a stated grain, dimensions with unique keys, one-to-many single-direction relationships, and derived columns computed upstream.

Recap card

- **Star schema parts** Fact tables (keys and measures, with a grain) and dimension tables (unique key, descriptive columns).
- **Relationship guidance** One-to-many, single direction. Minimise bi-directional relationships.

- **Calculated columns and Direct Lake** Not supported on SQL endpoints, preview on OneLake. Compute derived columns in gold.
- **Grain** Set by the fact table's dimension keys. State it in the table's docstring and comment.

Practice

Exercise 1. Multiple choice (level 1)

In a star schema, what decides the grain of a fact table?

- a) The number of measure columns
- b) The dimension keys it holds, and their values
- c) The number of rows
- d) The storage mode

Why?

- r1) Dimension key columns set the dimensionality, and their values set the granularity.
- r2) Grain is chosen in Power BI settings.

Exercise 2. Multiple choice (level 2)

What does Microsoft's Power BI guidance recommend about bi-directional relationships?

- a) Use them by default so slicers work everywhere.
- b) Minimise them. They can hurt performance and confuse report users.
- c) Use them only on date tables.
- d) They are not supported in Direct Lake.

Why?

- r1) Bi-directional filters can slow queries and filter in ways users do not expect.
- r2) They are deprecated.

Exercise 3. Put in order (level 2)

Order the steps to build a model for a brand tracker.

- Build the fact table with dimension keys and measure columns
- Write DAX measures
- Build dimension tables with unique keys in gold
- Create one-to-many, single-direction relationships

- State the fact table's grain

Exercise 4. Write the function (level 3)

Write `model_warnings(relationships: list[dict[str, str]]) -> list[str]` for a model review. Each relationship has `from_table`, `to_table`, `cardinality` (`"one_to_many"` or `"many_to_many"`) and `direction` (`"single"` or `"both"`). Return, in input order, a warning for each problem:

- `"<from>-<to>: many-to-many"` for a many-to-many relationship,
- `"<from>-<to>: bi-directional"` for direction both.

A relationship with both problems produces both warnings, many-to-many first.

Your code must pass these checks:

- A one-to-many single-direction relationship has no warnings.
- Flag many-to-many and bi-directional relationships, in order.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```python
# Warnings for relationships that break star-schema guidance.
def model_warnings(
    relationships: list[dict[str, str]],
) -> list[str]: ...
```

Starter code:

```python
def model_warnings(relationships):
    return []
```

Exercise 5. Architecture design (level 4)

A Direct Lake model on the SQL analytics endpoint needs an age band on respondents. Choose where it is built.

Options: In the gold dim_respondent table, in Spark or SQL, A calculated column in the semantic model.

1. Where age_band is computed
2. Awareness fact table grain
3. Relationship from dim_respondent to the fact

Sources

- Fact and dimension definitions; dimension keys set dimensionality and their values granularity. (verified: https://learn.microsoft.com/en-us/power-bi/guidance/star-schema)

- Microsoft recommends minimising bi-directional relationships because of performance and confusing experiences. (verified: https://learn.microsoft.com/en-us/power-bi/guidance/relationships-bidirectional-filtering)
- Calculated columns: Direct Lake on SQL analytics endpoints no; Direct Lake on OneLake user-context only, preview. (verified: https://learn.microsoft.com/en-us/fabric/fundamentals/direct-lake-overview (updated 2026-09-02))

C6-L2 DAX measures that state their filter context

Concept

In a semantic model, arithmetic belongs in **measures**. A measure is evaluated at query time, in the **filter context** of the visual: the slicers, rows and columns that apply to that cell. A calculated column is evaluated row by row when the model refreshes and stored, which costs memory and refresh time and cannot react to slicers.

Three habits make measures correct and readable (Microsoft's DAX guidance, checked September 2026):

- **Use `DIVIDE`, not `/`.** `DIVIDE(numerator, denominator)` returns BLANK when the denominator is zero or BLANK, instead of an error or infinity. You can pass an alternate result as a third argument. Microsoft notes it is also faster than testing for zero yourself.
- **Use variables.** `VAR ... RETURN` evaluates an expression once and names it. That improves performance, readability and debugging.
- **Change filter context only with `CALCULATE`**, and say why. `CALCULATE([Weighted base], ALL(dim_brand))` removes the brand filter, which is what a share-of-total denominator needs.

A weighted share, the survey version of lesson B8-L1:

```
Weighted aware % =
VAR aware_w = SUMX ( fact_awareness, fact_awareness[aware] *
    fact_awareness[weight] )
VAR base_w  = SUM ( fact_awareness[weight] )
RETURN DIVIDE ( aware_w, base_w )
```

Put the unweighted base next to it as a separate measure, `Base (n) = DISTINCTCOUNT(fact_awareness[respondent_key])`, so report users see how many respondents stand behind each percentage.

Name measures for what they return, store them in one place (a measures table or the fact table), and set their format strings. A measure that nobody can read is a measure nobody can check.

Why it matters

A report computed awareness as `SUM(aware_weight) / SUM(weight)` in a measure. In a new market with no fieldwork yet, the base was zero, and the visual showed "Infinity" and "NaN" in cells that the board pack exported to PDF. `DIVIDE` would have returned BLANK, and the empty cells would have been left empty.

Worked example: weak code

```
❶ Aware % = SUM(fact_awareness[aware_weight]) / SUM(fact_awareness[weight])
❷ Brand share = [Aware %] / CALCULATE([Aware %], ALL(fact_awareness))
```

1. **Line 1.** The / operator breaks on a zero base.
2. **Line 2.** ALL on the whole fact table removes every filter, including date and region. The share compares against the wrong total.

Worked example: strong code

```
 1 Weighted aware % =
 2 VAR aware_w = SUMX ( fact_awareness, fact_awareness[aware] *
     fact_awareness[weight] )
 3 VAR base_w = SUM ( fact_awareness[weight] )
❶ RETURN DIVIDE ( aware_w, base_w )
 5
 6 Share of aware across brands =
 7 VAR this_brand = [Weighted aware %]
❷ VAR all_brands = CALCULATE ( [Weighted aware %], ALL ( dim_brand ) )
 9 RETURN DIVIDE ( this_brand, all_brands )
```

1. **Line 4.** BLANK on a zero base, not an error.
2. **Line 8.** Removes only the brand filter. Date, region and wave filters still apply.

Measures, not calculated columns. DIVIDE for every ratio, variables for clarity, and CALCULATE with the narrowest filter change.

Recap card

- **Measure vs calculated column** Measures compute at query time in the visual's filter context. Calculated columns are stored at refresh.
- **DIVIDE in DAX** Returns BLANK (or an alternate result) when the denominator is zero or BLANK.
- **VAR ... RETURN** Evaluates once, names parts, improves performance, readability and debugging.

- **Share of total across brands** CALCULATE([measure], ALL(dim_brand)): remove only the brand filter.

Practice

Exercise 1. Multiple choice (level 1)

What does `DIVIDE(10, 0)` return in DAX?

- a) An error
- b) BLANK
- c) 0
- d) Infinity

Why?

- r1) DIVIDE returns BLANK when the denominator is zero or BLANK, unless an alternate result is given.
- r2) DAX has no concept of division by zero.

Exercise 2. Multiple choice (level 2)

A share-of-total measure uses `CALCULATE([Weighted aware %], ALL(fact_awareness))` as its denominator. A report filters to wave 12 and region North. What does the denominator ignore?

- a) Only the brand filter
- b) Every filter on fact_awareness, including wave and region
- c) Nothing: ALL only affects slicers
- d) Only filters from other pages

Why?

- r1) ALL on the fact table removes all its filters, including those that reach it through dimensions.
- r2) ALL always removes every filter in the model.

Exercise 3. Fill the gap (level 2)

Complete the weighted share measure.

```
Weighted aware % =
_____(1) aware_w = SUMX ( fact_awareness, fact_awareness[aware] *
    fact_awareness[weight] )
VAR base_w = SUM ( fact_awareness[weight] )
RETURN _____(2) ( aware_w, base_w )
```

Exercise 4. Write the function (level 3)

Write a measure `Share of aware across brands` that divides `[Weighted aware %]` by the same measure with the brand filter removed through `ALL ( dim_brand )`. Use variables named `this_brand` and `all_brands`, and `DIVIDE`.

Shape of the answer. The reference answer follows this outline. Replace each ... with your own code.

```
Share of aware across brands =
VAR ...
VAR ...
RETURN ...
```

Starter code:

```
Share of aware across brands = [Weighted aware %] / CALCULATE ( [Weighted
    aware %], ALL ( fact_awareness ) )
```

Exercise 5. Architecture design (level 4)

Decide where each calculation lives in a Direct Lake model on the SQL analytics endpoint.

Options: Gold table column, DAX measure, Calculated column.

1. Respondent age band
2. Weighted awareness percentage
3. Unweighted base (number of respondents)

Sources

- DIVIDE returns BLANK for a zero or BLANK denominator unless an alternate result is passed; it outperforms explicit zero checks. (verified: https://learn.microsoft.com/en-us/dax/best-practices/dax-divide-function-operator)
- Variables improve performance, reliability, readability and debugging. (verified: https://learn.microsoft.com/en-us/dax/best-practices/dax-variables)
- Materialised calculated columns can hurt refresh performance because values are derived at refresh time. (verified: https://learn.microsoft.com/en-us/power-bi/transform-model/desktop-calculated-columns (updated 2026-08-23))

C6-L3 Direct Lake, framing and the fallback you did not plan

Concept

Direct Lake lets a semantic model read Delta tables in OneLake directly, loading only the columns a query needs into memory. It combines Import-like query speed with no copy of the data. A refresh does not reload data. It performs **framing**: it records which Delta table versions the model uses. Framing takes seconds, and afterwards queries see the data as of that frame. Automatic updates, a schedule or an API call can trigger framing.

There are two kinds (Microsoft Learn, updated September 2026):

- **Direct Lake on OneLake** reads Delta tables from one or more Fabric items. It never falls back to DirectQuery. If it cannot load data, the query fails.
- **Direct Lake on SQL analytics endpoints** reads one Fabric item through its endpoint. When it cannot load a table directly, it **falls back to DirectQuery**: for example when the table is a SQL view, or when the warehouse uses SQL-based granular access control. Fallback queries are slower.

The model property **Direct Lake behavior** controls fallback for the SQL kind:

- `Automatic` (default): silently fall back to DirectQuery. Reports keep working, more slowly.
- `DirectLakeOnly`: fail with an error instead. Use it in development to find fallbacks.
- `DirectQueryOnly`: always use DirectQuery, to measure the difference.

Guardrails depend on the SKU. For each table, Fabric limits Parquet files, row groups and rows. Over a guardrail, framing can fail (or, on the SQL kind, queries fall back). From Learn's table:

SKU	Parquet files per table	Rows per table	Max memory
F2 to F32	1,000	300 million	3 to 10 GB
F64	5,000	1.5 billion	25 GB
F512 and above	10,000	12 to 24 billion	200 to 400 GB

Small files hit the file guardrail first. Compact gold tables with `OPTIMIZE` (lesson A3-L3), and give Direct Lake tables V-Order (lesson C5-L2).

Why it matters

A sales model on an F32 capacity was fast in testing. After six months of hourly appends, the fact table held 2,400 small Parquet files, above the F32 guardrail of 1,000. The model used Direct Lake on the SQL endpoint with the Automatic behaviour, so queries quietly fell back to DirectQuery. Reports took 40 seconds instead of two. Nobody saw an error. A nightly `OPTIMIZE` kept the table at a few dozen files and restored Direct Lake.

Worked example: weak code

```
Model: sales (Direct Lake on SQL analytics endpoint), behaviour: Automatic
Table fact_sales: 2,400 Parquet files, appended hourly, never optimised
Table v_sales_clean: SQL view over fact_sales
Capacity: F32
```

1. **Line 1.** Automatic hides both problems: queries silently fall back.
2. **Line 2.** Above the F32 guardrail of 1,000 files per table.
3. **Line 3.** A SQL view always forces DirectQuery fallback on the SQL kind.

Worked example: strong code

```
Model: sales (Direct Lake on SQL analytics endpoint), behaviour:
  DirectLakeOnly in dev, Automatic in prod
Table fact_sales: nightly OPTIMIZE, V-Order on, about 40 Parquet files
Table gold.sales_clean: Delta table written by Spark, replacing the SQL
  view
Capacity: F32, fact rows 120 million (limit 300 million)
Check: framing failures reviewed in refresh history each morning
```

1. **Line 1.** Development fails loudly on fallback. Production keeps working if one slips through.
2. **Line 3.** Materialise the view's logic as a Delta table so Direct Lake reads it directly.
3. **Line 4.** Row count checked against the SKU's guardrail.

Direct Lake stays fast when tables are Delta (not views), compact, within the SKU's guardrails, and fallback is made visible in development.

Recap card

- **Direct Lake refresh** Framing: records Delta table versions in seconds. No data copy.
- **Direct Lake on OneLake vs on SQL** OneLake: never falls back. SQL: falls back to DirectQuery for views and SQL-based security.
- **Direct Lake behavior values** Automatic (default, silent fallback), DirectLakeOnly (error), DirectQueryOnly (always DirectQuery).

- **F64 per-table guardrails** 5,000 Parquet files, 1.5 billion rows, 25 GB max memory.

Practice

Exercise 1. Multiple choice (level 1)

What does a refresh of a Direct Lake semantic model do?

- a) Reloads all data into the model, like Import.
- b) Frames the model: records which Delta table versions queries will use.
- c) Rewrites the Delta tables.
- d) Switches the model to DirectQuery.

Why?

- r1) Framing copies metadata only, takes seconds, and sets the point in time queries see.
- r2) Direct Lake models never refresh.

Exercise 2. Multiple choice (level 2)

A Direct Lake on SQL analytics endpoint model includes a SQL view. The Direct Lake behavior property is Automatic. What happens when a report queries the view?

- a) The query fails.
- b) The query falls back to DirectQuery and runs more slowly.
- c) The view is materialised automatically.
- d) The view is read directly from OneLake.

Why?

- r1) Views cannot be loaded directly from Delta, and Automatic falls back silently.
- r2) Views are always faster than tables.

Exercise 3. Fill the gap (level 2)

Set the Direct Lake behaviour so that development fails loudly on fallback, and production keeps working.

```
Development model: Direct Lake behavior = _____(1)
Production model:  Direct Lake behavior = _____(2)
```

Exercise 4. Write the function (level 3)

Write `guardrail_problems(sku: str, parquet_files: int, rows_millions: float) -> list[str]` using these limits from the lesson: F2 to F32 allow 1,000 files and 300 million rows per table, F64 allows 5,000 and 1,500, F128 5,000 and 3,000, F256 5,000 and 6,000. Return `"files"` and/or `"rows"` for each limit exceeded, in that order. Raise `KeyError` for other SKUs.

Your code must pass these checks:

- F2 to F32: 1,000 files and 300 million rows. Equal to the limit is allowed.
- F64: 5,000 files, 1,500 million rows. F256: 5,000 files, 6,000 million rows.
- Unknown SKUs must raise KeyError.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```
# Module constant.
SMALL = ...

# Module constant.
GUARDRAILS = ...

# Direct Lake per-table guardrails a table exceeds on a SKU.
def guardrail_problems(
    sku: str,
    parquet_files: int,
    rows_millions: float,
) -> list[str]: ...
```

Starter code:

```
def guardrail_problems(sku, parquet_files, rows_millions):
    return []
```

Exercise 5. Spot the bug (level 4)

A gold table read by Direct Lake on an F32 grows by one small file per hour and has passed the file guardrail. Tap the line that causes the build-up, and rewrite the job so the table is compacted after the append.

What you need to know.

`df.repartition(n)` redistributes the rows into `n` partitions. Spark writes one file per partition, so `repartition(1)` before a write produces one file per write.

`df.sparkSession` is the session the DataFrame belongs to, so a function can run SQL with `df.sparkSession.sql(...)` without a separate `spark` parameter.

```
def publish_hourly(df, table):
    """Append the latest hour to a Direct Lake gold table."""
```

```
3     df.repartition(1).write.mode("append").saveAsTable(table)
4     return None
```

Write down the line number of the bug. Then write the fix.

Sources

- Direct Lake on OneLake never falls back; Direct Lake on SQL falls back to DirectQuery for SQL views and SQL-based granular access control; framing copies metadata in seconds; guardrails per SKU table. (verified: https://learn.microsoft.com/en-us/fabric/fundamentals/direct-lake-overview (updated 2026-09-02))
- Direct Lake behavior values Automatic (default), DirectLakeOnly and DirectQueryOnly; framing can fail when a table exceeds guardrails. (verified: https://learn.microsoft.com/en-us/fabric/fundamentals/direct-lake-how-it-works (updated 2025-09-22))

C7 Real-Time Intelligence

Roadmap: Months 16 to 18: MLOps and deployment; Current work. Eventstream, Eventhouse, KQL and Activator.

C7-L1 Eventstream, events in, routed out

Concept

An **eventstream** brings real-time events into Fabric, optionally transforms them, and routes them to destinations, without code. It is Microsoft's recommended path for low-latency, event-driven ingestion (lesson C4-L1).

Sources include Azure Event Hubs, IoT Hub, custom apps, and change data capture (CDC) connectors for Azure SQL Database, PostgreSQL, MySQL and SQL Server on a VM. A CDC source first captures a snapshot of the current data, then streams the changes.

Transformations are no-code operators on the stream: filter, manage fields, aggregate over a time window, group by, union, join, expand. You can split one stream into **derived streams** for different consumers.

Destinations (Microsoft Learn, checked September 2026):

- **Eventhouse**, for KQL analysis (lesson C7-L2). Two ingestion modes: **direct ingestion**, or **event processing before ingestion** when you apply transformations in the eventstream first.
- **Lakehouse**. Events are converted to Delta Lake format and written to lakehouse tables.
- **Activator**, to trigger actions on conditions (lesson C7-L3).
- A **custom endpoint**, to route events to your own application.

Writing to an eventhouse, lakehouse or Activator destination needs **Contributor or higher** on the destination's workspace.

Design choices that matter:

- **Filter early.** Drop events nobody needs in the eventstream, before they cost storage and query time.
- **Keep the raw stream.** Route an unfiltered copy to an eventhouse or lakehouse table as your bronze layer, and derive cleaned streams from it. You cannot replay events you dropped.

- **Decide the event time.** Events carry their own timestamp. Use it for windows and analysis, not the time of arrival, which drifts when senders buffer or reconnect.

Why it matters

A fleet team filtered its eventstream to keep only engine-fault events and wrote those to an eventhouse. Three months later, engineers needed the normal readings before each fault to build a failure model. They had never been stored. Keeping the raw stream in a cheap table, and deriving the fault stream from it, would have preserved the history the model needed.

Worked example: weak code

```
1 Source: IoT Hub (all vehicle telemetry)
1 Transform: Filter event_type == "fault"
3 Destination: Eventhouse table faults
```

1. **Line 2.** The only copy of the data is filtered. Normal readings are lost for good.

Worked example: strong code

```
1 Source: IoT Hub (all vehicle telemetry)
1 Destination 1: Eventhouse table telemetry_raw        (direct ingestion,
    every event)
2 Derived stream: faults = Filter event_type == "fault"
3 Destination 2: Activator rule on faults               (alert within
    seconds)
5 Destination 3: Lakehouse table bronze.vehicle_faults  (Delta, for Spark
    models)
```

1. **Line 2.** The raw stream is kept as bronze. Anything else can be rebuilt from it.
2. **Line 3.** Consumers get a filtered stream without losing the source.
3. **Line 4.** Alerts come from the stream, not from a scheduled query.

Land the raw stream once, derive filtered streams for consumers, and route each to the destination that suits it.

Recap card

- **What an eventstream does** Ingests real-time events, transforms them without code, and routes them to destinations.
- **Eventstream destinations** Eventhouse (direct or processed ingestion), Lakehouse (Delta tables), Activator, custom endpoint.
- **Permission for eventhouse, lakehouse or Activator destinations** Contributor or higher on the destination workspace.
- **Protect replay** Land the raw stream first. Derive filtered streams from it.

Practice

Exercise 1. Multiple choice (level 1)

An eventstream sends events to a lakehouse destination. In what format are they stored?

- a) JSON files under Files
- b) Delta Lake tables
- c) CSV files
- d) KQL tables

Why?

- r1) The lakehouse destination converts real-time events to Delta Lake format in lakehouse tables.
- r2) Eventstream only writes Delta.

Exercise 2. Multiple choice (level 2)

Which permission does the person setting up an eventstream need on the workspace of an eventhouse destination?

- a) Viewer
- b) Contributor or higher
- c) Admin only
- d) None: eventstreams write as the service

Why?

- r1) Microsoft Learn requires Contributor or higher on the destination workspace for eventhouse, lakehouse and Activator destinations.
- r2) Contributors can create any item in any workspace.

Exercise 3. Put in order (level 2)

Order the design steps for a new telemetry eventstream.

- Connect the source
- Connect each derived stream to its destination
- Create derived streams with filters for each consumer
- Route every event to a raw table
- Compare event counts at source and destinations

Exercise 4. Write the function (level 3)

Write `route(event: dict[str, object]) -> list[str]` modelling the strong design. Every event goes to `"telemetry_raw"`. Events with `event_type == "fault"` also go to `"activator_faults"` and `"bronze.vehicle_faults"`. Events without an `event_type` key go to `"telemetry_raw"` and `"quarantine"`. Return destinations in that order.

Your code must pass these checks:

- Every event is kept in telemetry_raw.
- Faults go to the raw table, Activator and the bronze faults table, in that order.
- Events without event_type are kept raw and quarantined, not dropped or crashing.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```
# Destinations for one telemetry event. Every event is kept raw.
def route(event: dict[str, object]) -> list[str]: ...
```

Starter code:

```
def route(event):
    if event["event_type"] == "fault":
        return ["activator_faults"]
    return []
```

Exercise 5. Architecture design (level 4)

Stream order changes from an Azure SQL database into Fabric for a live dashboard and for nightly Spark models.

Options: Azure SQL Database CDC source, A pipeline polling the table every minute.

1. Source connector
2. Destination for the live dashboard
3. Destination for nightly Spark models

Sources

- Eventstreams ingest, transform without code and route events; CDC source connectors for Azure SQL, PostgreSQL, MySQL and SQL Server on VM capture a snapshot then changes. (verified: https://learn.microsoft.com/en-us/fabric/real-time-intelligence/event-streams/overview (updated 2026-04-29))

- Destinations include custom endpoint, eventhouse (direct ingestion or processing before ingestion), lakehouse (Delta) and Activator; Contributor or higher is required for eventhouse, lakehouse and Activator destinations. (verified: https://learn.microsoft.com/en-us/fabric/real-time-intelligence/event-streams/add-manage-eventstream-destinations (updated 2026-03-19))

C7-L2 Eventhouse and KQL for time-series questions

Concept

An **eventhouse** holds one or more **KQL databases**, built for high-volume, time-stamped data such as telemetry, logs and clickstreams. You query them with the **Kusto Query Language (KQL)**: a pipeline of operators joined by `|`, read top to bottom.

```
telemetry_raw
| where event_time > ago(1h)
| where event_type == "reading"
| summarize avg_rpm = avg(rpm), n = count() by vehicle_id, bin(event_time, 5m)
```

- Put `where` filters first, especially on time. They cut the data every later operator touches.
- `ago(1h)` is one hour before now. `bin(event_time, 5m)` rounds times **down** to five-minute buckets.
- `summarize ... by` aggregates per group.

Three KQL behaviours catch people out (Microsoft Learn, checked September 2026):

- `take` **returns arbitrary rows.** There is no guarantee which records come back, and repeated runs can differ, unless the data is sorted. For "the latest 10", use `top 10 by event_time desc`.
- `summarize` **leaves gaps.** A five-minute bin with no events produces no row, so a chart draws a straight line across an outage. `make-series ... default = 0 ... step 5m` fills missing bins.
- **Empty input.** `summarize count()` without `by` returns one row. With `by`, empty input returns no rows at all.

Data policies are set per database or table. The **retention policy** removes old data automatically. The default is 3,650 days, the maximum 36,500, the minimum one day, and you can set it to unlimited. The **caching policy** decides how much recent data stays in the hot cache on local SSD: faster

queries, higher cost. Set retention from the use: raw telemetry kept 90 days, daily aggregates kept for years.

Turn on **OneLake availability** for a database or table to expose its data as Delta in OneLake, so Spark and the SQL analytics endpoint can read it without a copy.

Why it matters

An on-call dashboard showed the "latest readings" from a KQL query ending in `| take 10`. During an incident, the panel showed readings from two hours earlier, mixed with current ones, and the engineer concluded the sensor had recovered. It had not. `take` returned arbitrary rows. `top 10 by event_time desc` would have shown the true latest readings.

Worked example: weak code

```
telemetry_raw
| summarize avg_rpm = avg(rpm) by vehicle_id, bin(event_time, 5m)
| where event_time > ago(1h)
| take 10
```

1. **Line 2.** Aggregates the whole table before filtering to the last hour.
2. **Line 4.** Arbitrary rows, different on each run.

Worked example: strong code

```
telemetry_raw
| where event_time > ago(1h)
| where event_type == "reading"
| make-series avg_rpm = avg(rpm) default = 0 on event_time step 5m by
    vehicle_id

telemetry_raw
| where event_time > ago(15m)
| top 10 by event_time desc
```

1. **Line 2.** Time filter first: less data for every later step.
2. **Line 4.** Every five-minute bin exists, so outages show as zeros, not as smooth lines.
3. **Line 8.** The true latest ten, every time.

Filter on time first, use top for "latest", make-series for continuous charts, and set retention and caching per table on purpose.

Recap card

- **take vs top in KQL** take: arbitrary rows, not repeatable. top N by col desc: ordered, repeatable.

- **Gaps in time series** summarize omits empty bins. make-series ... default = 0 fills them.
- **KQL retention policy defaults** Default 3,650 days, maximum 36,500, minimum 1 day, or unlimited.
- **Caching policy** How much recent data stays in the hot SSD cache: faster queries, higher cost.

Practice

Exercise 1. Multiple choice (level 1)

A query ends with `| take 10`. What do you get?

- a) The 10 most recent rows
- b) The first 10 rows ingested
- c) 10 arbitrary rows, possibly different on each run
- d) An error unless the table is sorted

Why?

- r1) take guarantees nothing about which records it returns unless the data is sorted.
- r2) take samples randomly with a fixed seed.

Exercise 2. Multiple choice (level 2)

A sensor sent nothing between 10:05 and 10:20. What does `summarize n = count() by bin(event_time, 5m)` return for those bins?

- a) Rows with n = 0
- b) No rows for those bins
- c) Rows with n = null
- d) An error

Why?

- r1) summarize groups existing rows. Bins with no rows do not appear. make-series fills them.
- r2) KQL drops zero counts to save space.

Exercise 3. Fill the gap (level 2)

Return the 10 most recent readings from the last hour.

```
1 telemetry_raw
2 | where event_time > _____(1)(1h)
3 | _____(2) 10 by event_time desc
```

Exercise 4. Write the function (level 3)

Write a KQL query on `telemetry_raw` that returns, for the last 24 hours, the average `rpm` per `vehicle_id` in one-hour steps, with **every** hour present and missing hours as 0. Filter on time first. Name the series `avg_rpm`.

Shape of the answer. The reference answer follows this outline. Replace each ... with your own code.

```
telemetry_raw
| where ...
| make-series ...
```

Starter code:

```
telemetry_raw
| summarize avg_rpm = avg(rpm) by vehicle_id, bin(event_time, 1h)
```

Exercise 5. Architecture design (level 4)

Set data policies for a telemetry eventhouse.

Options: 90 days, Leave the default of 3,650 days, 7 days.

1. Retention for telemetry_raw (queried for 30 days, needed for audits for 90)
2. Hot cache for telemetry_raw
3. Retention for daily aggregates used in yearly trends

Sources

- take guarantees nothing about which records it returns unless the data is sorted; bin rounds down; summarize on empty input with a by clause returns no rows; make-series assigns a default to absent values. (verified: https://learn.microsoft.com/en-us/kusto/query/ (take-operator, bin-function, summarize-operator, make-series-operator; view=microsoft-fabric))
- KQL database retention defaults to 3,650 days, maximum 36,500, minimum one day, unlimited option; caching policy controls data kept in the hot cache. (verified: https://learn.microsoft.com/en-us/fabric/real-time-intelligence/data-policies (updated 2026-03-22))
- OneLake availability can be enabled per database or table. (verified: https://learn.microsoft.com/en-us/fabric/real-time-intelligence/eventhouse (updated 2026-06-15))

C7-L3 Activator, rules that act on events

Concept

Activator is Fabric's no-code event detection engine. It watches data and takes an action when a condition is met. Its sources include eventstreams, Fabric and Azure events, Power BI reports and Real-Time dashboards.

Activator groups **events** into **objects** by a shared ID, such as `vehicle_id`. It evaluates each rule **per object** and keeps state for each one. "Vehicle 17 overheated" is a separate fact from "vehicle 42 overheated".

Two kinds of condition behave differently (Microsoft Learn, checked September 2026):

- **Change conditions** fire on a transition. `Increases above 100` fires when a value crosses 100 from below. `Becomes true` fires when a boolean changes from false to true.
- **State conditions** fire on every event that matches. `Is greater than 100` fires for each reading above 100.

For alerts, you want a change condition, or a state condition with an **occurrence** setting: "When it has been true for 10 minutes" fires only if the condition holds for the whole period. **No presence of data** (heartbeat) fires when no new events arrive within a set time. Use it to detect a silent sensor.

Actions include an email, a Teams message, a Power Automate flow, or running a Fabric item such as a pipeline or notebook.

Limits to design around:

- Up to 10,000 incoming events per second per rule. Above that, Activator stops the rule.
- Emails and Teams messages: 30 per rule per recipient per hour.
- Late events: by default, Activator drops events that arrive after their time window closes. The **Wait time for late-arriving events** setting (default two minutes) lets them in, at the cost of added latency.

Why it matters

A plant set a rule `Temperature Is greater than 80` on a stream that reports every 10 seconds. One oven ran hot for 20 minutes. The rule tried to send 120 emails to the shift lead. After 30, the rate limit throttled the rest, and the lead muted the alerts. An `Increases above 80` rule, or `Is greater than 80` with an occurrence of five minutes, would have sent one useful alert.

Worked example: weak code

```
1 Object: all events (no ID)
2 Condition: temperature Is greater than 80
3 Occurrence: Every time the condition is met
4 Action: Email shift lead
```

1. **Line 1.** One oven's reading clears the state of another. Group by oven.
2. **Line 2.** A state condition fires on every hot reading.

Worked example: strong code

```
1 Object: oven, keyed by oven_id
2 Condition: temperature Is greater than 80
3 Occurrence: When it has been true for 5 minutes
4 Action: Teams message to the line channel
5 Rule 2: No presence of data for 2 minutes per oven_id
```

1. **Line 1.** State is tracked per oven.
2. **Line 3.** One alert for a sustained problem, not one per reading.
3. **Line 5.** A silent sensor is a problem too. Heartbeat detects it.

Key each rule by the object it describes. Fire on change or sustained state, not on every event. Add a heartbeat for silence.

Recap card

- **Events and objects in Activator** Events are grouped into objects by a shared ID. Rules run and keep state per object.
- **Increases above vs Is greater than** Increases above fires on the crossing. Is greater than fires on every matching event.
- **Detect a silent sensor** No presence of data (heartbeat) fires when no events arrive within a set time.
- **Activator limits to design around** 10,000 events per second per rule. 30 emails or Teams messages per rule per recipient per hour.

Practice

Exercise 1. Multiple choice (level 1)

A stream reports `temperature` every 10 seconds. It rises above 100 and stays there for 3 minutes. How many times does `Increases above 100` fire?

- a) Once
- b) 18 times, once per reading
- c) Never, because the value never decreases
- d) Once every minute

Why?

- r1) Increases above is a change condition. It fires when the value crosses the threshold from below.
- r2) Activator sends one alert per rule per day.

Exercise 2. Multiple choice (level 2)

A sensor stops sending data. Which Activator condition detects it?

- a) Is less than 1
- b) No presence of data
- c) Becomes false
- d) Changes

Why?

- r1) No presence of data fires when no new events arrive within a set time. Other conditions need an event to evaluate.
- r2) A silent sensor sends zeros.

Exercise 3. Fill the gap (level 2)

Alert once when a freezer stays above -15 for 10 minutes. Complete the rule.

```
Object: freezer, keyed by _____(1)
Condition: temperature Is greater than -15
Occurrence: When it has been _____(2) for 10 minutes
```

Exercise 4. Write the function (level 3)

Model an Activator `Increases above` rule. Write `activations(events: list[tuple[str, float]], threshold: float) -> list[int]`. Each event is `(object_id, value)` in arrival order. Return the indexes of events where that object's value goes above `threshold` and its previous value was at or below it. An object's first event counts as crossing if it is above the threshold. Track state per object.

Your code must pass these checks:

- Fire on each crossing from at or below to above, not on every hot reading.
- Track state per object. Oven b's cool reading does not reset oven a.
- A value equal to the threshold is not above it.
- No events, no activations.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```
# Indexes of events where an object's value crosses above a threshold.
```

```
def activations(
    events: list[tuple[str, float]],
    threshold: float,
) -> list[int]: ...
```

Starter code:

```
def activations(events, threshold):
    return [i for i, (_, value) in enumerate(events) if value > threshold]
```

Exercise 5. Architecture design (level 4)

Design alerts for 400 delivery vans that send GPS and engine data every 5 seconds. The fleet manager wants one Teams message when a van's engine runs hot for a sustained period, and to know when a van goes silent.

Options: van_id, One object for the whole fleet.

1. Object key
2. Hot-engine condition
3. Silent van

Sources

- Activator is a no-code event detection engine; events are grouped into objects by a shared identifier; rules are evaluated per object with state tracking; subsecond latency for stateless rules on streaming data. (verified: https://learn.microsoft.com/en-us/fabric/real-time-intelligence/data-activator/activator-introduction (updated 2026-04-17))
- Conditions include Increases above, Decreases below, Is greater than, Becomes true/false, Changes and No presence of data; occurrence options are every time, n times, and true for a duration. (verified: https://learn.microsoft.com/en-us/fabric/real-time-intelligence/data-activator/activator-detection-conditions (updated 2026-05-04))
- Actions include email, Teams message (individual, group chat, channel), Power Automate and running Fabric items such as pipelines and notebooks. (verified: https://learn.microsoft.com/en-us/fabric/real-time-intelligence/data-activator/rule-actions (updated 2026-05-11))
- Up to 10,000 incoming events per second per rule, after which the rule stops; 30 email or Teams messages per rule per recipient per hour. (verified: https://learn.microsoft.com/en-us/fabric/real-time-intelligence/data-activator/activator-limitations (updated 2026-04-17))
- Late events are dropped by default; the wait time for late-arriving events defaults to two minutes and adds latency. (verified: https://

learn.microsoft.com/en-us/fabric/real-time-intelligence/data-activator/activator-latency (updated 2026-04-30))

C8 Copilot and AI in Fabric

Roadmap: Months 16 to 18: MLOps and deployment; Current work. Copilot across workloads, data agents, limits and verification.

C8-L1 Copilot in Fabric, where it runs and what it costs

Concept

Copilot in Fabric is an AI assistant built into each workload. It writes notebook code, pipelines, SQL, KQL and DAX, and summarises reports. Before you rely on it, check three things.

1. Is it switched on for you? Copilot needs a paid Fabric capacity of **F2 or higher**, or Power BI Premium **P1 or higher**, in a supported region. Fabric administrators control it through tenant settings, scoped to security groups. If your capacity's region lacks Azure OpenAI, an admin must also allow data to be processed outside that region. Copilot stops working when a Fabric trial expires.

2. What is its status? Release status differs by workload (Microsoft Learn, checked September 2026):

Experience	Status
Copilot in notebooks (Data Engineering, Data Science)	Preview
Data agents	GA
Copilot in Data Factory (Dataflow Gen2, pipelines)	GA
Copilot for SQL queries in a Warehouse	Preview
Copilot for semantic models and reports in Power BI	GA

3. What does it cost? Copilot requests consume capacity units, metered per 1,000 tokens. About 1,000 tokens equal 750 words. Input costs 100 CU seconds per 1,000 tokens, cached input 10 and output 400. A request with 2,000 input tokens and 500 output tokens costs (2,000 × 100 + 500 × 400) / 1,000 = 400 CU seconds. Copilot runs as a **background** operation, so Fabric smooths its cost over 24 hours.

On data handling, Microsoft states your data is not used to train models. It also states that Copilot output is sometimes inaccurate, and that people able to judge its accuracy should review it. Lesson C8-L3 shows you how.

Why it matters

An analytics team put Copilot-generated SQL into a scheduled report during the Warehouse Copilot preview. Nobody recorded which features were preview. When a reviewer asked whether the process relied on anything outside GA, the team spent a week tracing it. Check status first, record it, and keep production logic reviewed and owned by a person.

Worked example: weak code

```
Plan: use Copilot to write all transformation SQL
Capacity: trial
Review: none, Copilot output goes straight to the scheduled job
Cost estimate: Copilot is included
```

1. **Line 2.** Copilot stops working when the trial expires.
2. **Line 3.** Microsoft states output is sometimes inaccurate. Someone qualified must review it.
3. **Line 4.** Every request consumes CUs from your capacity.

Worked example: strong code

```
Plan: Copilot drafts SQL, an engineer reviews, tests and owns it
Capacity: F8, Copilot tenant setting scoped to the data-engineering group
Status check: Warehouse Copilot = preview (Learn, September 2026), noted
  in the design record
Cost estimate: about 400 CU seconds per request, smoothed over 24 hours
```

1. **Line 1.** Copilot drafts. A person decides.
2. **Line 3.** Preview features change. Record the status and the date.
3. **Line 4.** Estimated from the published token rates.

Confirm Copilot is enabled on a paid capacity, record the release status of each experience, and budget its CU use.

Recap card

- **Copilot in Fabric capacity requirement** Paid F2+ or P1+ in a supported region, with tenant settings enabled. Not after a trial expires.
- **Copilot release status** Per experience. Notebooks and Warehouse SQL preview. Data agents, Data Factory and Power BI GA (September 2026).
- **Copilot token rates per 1,000 tokens** Input 100, cached input 10, output 400 CU seconds. Background job, smoothed over 24 hours.
- **Copilot output** Sometimes inaccurate. A person able to judge it reviews it before use.

Practice

Exercise 1. Multiple choice (level 1)

Which capacity runs Copilot in Fabric?

- a) Any capacity, including an expired trial
- b) A paid F2 or higher, or P1 or higher, in a supported region
- c) Only F64 or higher
- d) No capacity: Copilot runs on the user's licence

Why?

- r1) Microsoft Learn lists a paid F2+ or P1+ capacity in a supported region, with tenant settings enabled.
- r2) Copilot is part of every Power BI Pro licence.

Exercise 2. Multiple choice (level 2)

Your design uses Copilot in notebooks and Copilot in Data Factory pipelines. What do you record about their status?

- a) Both GA
- b) Notebooks preview, Data Factory GA, with the date you checked
- c) Both preview
- d) Nothing: Copilot has one status across Fabric

Why?

- r1) Microsoft Learn's release-status page lists each Copilot experience separately.
- r2) Everything in Data Factory is GA.

Exercise 3. Fill the gap (level 2)

Complete the Copilot cost for 1,000 input tokens and 1,000 output tokens, with no cache.

```
input:  1,000 tokens x 100 / 1,000 = 100 CU seconds
output: 1,000 tokens x _____(1) / 1,000 = _____(2) CU seconds
total: 500 CU seconds
```

Exercise 4. Write the function (level 3)

Write `copilot_cu_seconds(input_tokens: int, output_tokens: int, cached_input_tokens: int = 0) -> float`. `cached_input_tokens` is the part of `input_tokens` served from the prompt cache. Rates per 1,000 tokens: input

100, cached input 10, output 400 CU seconds. Raise `ValueError` if any count is negative or if cached tokens exceed input tokens.

Your code must pass these checks:

- 2,000 input and 500 output tokens cost 400 CU seconds. Output is 400 per 1,000.
- Cached input costs 10 per 1,000, and those tokens are not charged at the full input rate as well.
- Cached tokens are part of input tokens. More cached than input is invalid.
- Negative token counts are invalid.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```python
# Module constant.
INPUT_RATE = ...

# Module constant.
CACHED_INPUT_RATE = ...

# Module constant.
OUTPUT_RATE = ...

# Module constant.
TOKENS_PER_UNIT = ...

# CU seconds for one Copilot request at the published token rates.
def copilot_cu_seconds(
    input_tokens: int,
    output_tokens: int,
    cached_input_tokens: int = 0,
) -> float: ...
```

Starter code:

```python
def copilot_cu_seconds(input_tokens, output_tokens,
   cached_input_tokens=0):
     return (input_tokens + output_tokens) * 100 / 1000
```

Exercise 5. Architecture design (level 4)

Your organisation wants Copilot for its data-engineering team only, on an F16 capacity, with an audit trail of which Copilot features the pipelines depend on.

Options: Enabled for the data-engineering security group, Enabled for the whole organisation.

1. Tenant setting scope
2. Design record

3. Copilot output in production

Sources

- Copilot requires a paid F2+ or P1+ capacity in a supported region; tenant settings scoped to security groups; cross-geo processing settings; stops after trial expiry. (verified: https://learn.microsoft.com/en-us/fabric/fundamentals/copilot-enable-fabric (updated 2026-08-10); troubleshoot-copilot-unavailable-after-trial (updated 2026-07-10))
- Release status: notebooks preview, Data Factory GA, Warehouse SQL Copilot preview, Power BI semantic models and reports GA. Data agent GA per its concept page; the release-status page lists it in a Data Science row marked preview alongside AI functions. (verified: https://learn.microsoft.com/en-us/fabric/fundamentals/copilot-ai-feature-state (updated 2026-06-19); data-science/concept-data-agent (updated 2026-05-11))
- Copilot consumption: 100 CU seconds per 1,000 input tokens, 10 cached input, 400 output; about 1,000 tokens = 750 words; background operation smoothed over 24 hours; worked example 400 CU seconds. (verified: https://learn.microsoft.com/en-us/fabric/fundamentals/copilot-fabric-consumption (updated 2026-05-22))
- Data not used to train models; output is sometimes inaccurate and should be reviewed by people able to evaluate it. (verified: https://learn.microsoft.com/en-us/fabric/fundamentals/copilot-privacy-security (updated 2025-06-25))

C8-L2 Data agents, questions in plain language

Concept

A **Fabric data agent** (GA, Microsoft Learn, checked September 2026) answers questions in plain language over your data. It picks a data source, writes a query and runs it:

- **SQL** for lakehouses and warehouses.
- **DAX** for Power BI semantic models.
- **KQL** for KQL databases in an eventhouse.

One agent takes up to **five data sources**. Users do not write the query, but the query is still what decides the answer.

Three rules keep it safe:

- **Read-only.** The agent generates only read queries. It never creates, updates or deletes data, and it does not run notebooks or other write workflows.
- **The user's permissions.** The agent runs with the credentials of the person asking, so each user only reaches data they already have access to.
- **No unstructured files.** PDFs, Word documents and text files are not supported sources.

The agent is only as good as the context you give it. Microsoft's guidance:

- **Select only the tables and columns** the agent needs. Every extra table is another chance to pick the wrong one.
- **Use clear names** such as `order_submission_date`, not `date1`.
- **Write instructions** that explain business terms and which source answers which kind of question.
- **Add example queries**: question and query pairs. Each example is validated against the schema, and invalid ones are not used. Lakehouses, warehouses and KQL databases support examples. Semantic models do not.

Then **evaluate** the agent. Build a ground-truth set of questions with known answers, run them through the agent, and measure how many it gets right. The `fabric-data-agent-sdk` package provides an evaluation API for this. Rerun the set after every change to instructions, examples or schema.

Every query costs tokens. Instructions, example queries and conversation history all add to the input and so to CU use.

Why it matters

A sales team published a data agent over a lakehouse with 60 tables, including three with a column called `revenue`: gross, net and forecast. "What was revenue last quarter?" returned the forecast. Nobody noticed for a month. Selecting only the net-revenue table, plus an instruction defining revenue as net of returns, fixed it. A 20-question ground-truth test would have caught it on day one.

Worked example: weak code

```
1 Sources: sales_lakehouse (all 60 tables)
2 Instructions: (none)
3 Example queries: (none)
```

```
❷ Testing: tried three questions in the chat, looked fine
```

1. **Line 1.** Three revenue columns. The agent picks one.
2. **Line 4.** Looking fine is not a measured accuracy.

Worked example: strong code

```
❶ Sources: sales_lakehouse (fact_sales, dim_customer, dim_date only)
❷ Instructions: "Revenue" means net_revenue_gbp, after returns. Quarters are
    calendar quarters.
  3 Example queries: 12 question and SQL pairs, all passing schema validation
❸ Testing: 40 ground-truth questions, rerun after every change, accuracy
    recorded
```

1. **Line 1.** Fewer tables, fewer wrong choices.
2. **Line 2.** Business terms defined once, for every question.
3. **Line 4.** A number you track, not a feeling.

Narrow the schema, define your terms, add validated examples, and measure accuracy against known answers.

Recap card

- **Data agent query languages** SQL for lakehouse and warehouse, DAX for semantic models, KQL for KQL databases. Up to five sources.
- **Data agent security** Read-only queries only. Runs with the asking user's permissions.
- **Example queries** Question and query pairs, validated against the schema. Not supported for semantic models.
- **Measure a data agent** Ground-truth questions with known answers, rerun after every change, accuracy recorded.

Practice

Exercise 1. Multiple choice (level 1)

A user asks a data agent to "delete the test orders". What happens?

- a) The agent writes and runs a DELETE statement
- b) The agent generates only read queries, so it does not delete data
- c) The agent deletes them if the user is a workspace Admin
- d) The agent asks the data owner for approval

Why?

- r1) Microsoft Learn states the data agent generates only read queries and keeps read-only connections.

- r2) The agent has no permissions of its own.

Exercise 2. Multiple choice (level 2)

Ana has access to the UK sales table but not the EU one. Both are sources of the same data agent. Ana asks for total EU sales. What does the agent do?

- a) Returns EU sales, because the agent's owner has access
- b) Answers only from data Ana has permission to read
- c) Returns EU sales with the numbers masked
- d) Returns a random estimate

Why?

- r1) The agent uses the requesting user's credentials and permissions.
- r2) The agent filters rows by country automatically.

Exercise 3. Put in order (level 2)

Order the steps to build a data agent over a sales lakehouse.

- Select only the tables and columns the questions need
- Add example question and SQL pairs
- Publish and share the agent
- Run the ground-truth question set and record accuracy
- Write instructions that define business terms

Exercise 4. Write the function (level 3)

Write `evaluate(expected: dict[str, float], actual: dict[str, float | None], rel_tol: float = 0.001) -> tuple[float, list[str]]`. `expected` maps question ids to ground-truth answers. `actual` maps them to the agent's answers, where `None` means the agent gave no number. Return the share of questions answered correctly (within `rel_tol`, and exactly for an expected 0.0) and a sorted list of the failed ids. A question missing from `actual` fails. Raise `ValueError` if `expected` is empty.

What you need to know.

`math.isclose(a, b, rel_tol=0.001)` is `True` when `a` and `b` differ by at most 0.1% of the larger of the two. Use it for floats, where `==` fails on rounding noise. With its default absolute tolerance of 0, a comparison with 0.0 is true only for exactly 0.0.

```
1 import math
2
```

```
math.isclose(100.0, 100.05, rel_tol=0.001)   # True
math.isclose(0.0, 1e-12)                     # False
```

Your code must pass these checks:

- q2 is 10% off, so it fails. One of two correct gives 0.5.
- An answer of 0.0 is a real answer. Test for None with `is None`, not truthiness.
- None and missing answers fail, and accuracy divides by the number of expected questions.
- An empty ground-truth set has no accuracy. Raise ValueError.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```
# Accuracy of a data agent against a ground-truth set.
def evaluate(
    expected: dict[str, float],
    actual: dict[str, float | None],
    rel_tol: float = 0.001,
) -> tuple[float, list[str]]: ...
```

Starter code:

```
def evaluate(expected, actual, rel_tol=0.001):
    failed = [q for q in expected if not actual.get(q)]
    return 1 - len(failed) / len(actual), failed
```

Exercise 5. Architecture design (level 4)

Configure a data agent for finance analysts. Sources: a finance warehouse and a Power BI semantic model of the same numbers. Contracts in PDF live in a lakehouse Files folder.

Options: Leave them out: unstructured files are not supported, Add the Files folder as a source.

1. The PDF contracts
2. Where to add example queries
3. Instruction to add

Sources

- Data agent is GA per its concept page; supports up to five sources (lakehouse, warehouse, semantic model, KQL database, and others); generates SQL, DAX, KQL; read-only; uses the requesting user's credentials; no unstructured data such as PDF, docx, txt. (verified: https://learn.microsoft.com/en-us/fabric/data-science/concept-data-agent (updated 2026-05-11); copilot-ai-feature-state (updated 2026-06-19))

- Select only needed tables and columns; use descriptive names; instructions and descriptions supply business meaning. (verified: https://learn.microsoft.com/en-us/fabric/data-science/data-agent-configuration-best-practices (updated 2026-08-27))
- Example queries are validated against the schema and supported for lakehouse, warehouse and KQL databases, not semantic models or ontology. (verified: https://learn.microsoft.com/en-us/fabric/data-science/data-agent-example-queries (updated 2025-09-11))
- fabric-data-agent-sdk provides evaluation against ground-truth question sets. (verified: https://learn.microsoft.com/en-us/fabric/data-science/evaluate-data-agent (updated 2026-09-01))
- Instructions, example queries and conversation history add to token usage and CU consumption. (verified: https://learn.microsoft.com/en-us/fabric/fundamentals/data-agent-consumption (updated 2025-03-20))

C8-L3 Verify what Copilot writes

Concept

Microsoft's own guidance for Copilot in notebooks (preview, Microsoft Learn, checked September 2026) says: always review AI-generated code before you apply it. Output might be inaccurate, incomplete, or based on outdated library syntax. The chat pane shows changes as an approval diff. Approving the diff is your review, so make it a real one.

Code that runs without error proves only that the names exist and the syntax parses. Use four checks, in this order.

1. **Read every line.** Check each table, column and function against the schema and the docs. Check the grain: one row per what? Check joins: inner joins drop unmatched rows.
2. **Run it on known data.** Build five to ten rows where you have worked out the answer by hand. Include the awkward cases: a null, a zero denominator, a duplicate key, an empty group.
3. **Compare with a second method.** Compute the same number another way: a plain `count()` against the generated aggregate, SQL against PySpark, the new query against last month's report.
4. **Keep the checks.** Turn steps 2 and 3 into tests that run with the pipeline, so the next edit, by you or by Copilot, is checked too.

Errors to look for first, because they run cleanly and give plausible numbers:

- **Average of averages.** `AVG(avg_basket)` over stores weights a small store the same as a large one. Divide total sales by total baskets instead.
- **Wrong join type.** An inner join to a dimension drops facts with no match.
- `COUNT(*)` **for distinct things.** Customers with two orders count twice.
- **Unguarded division.** A zero denominator fails in some engines and returns null or infinity in others.
- **Invented names.** A column or function that does not exist. This one fails loudly.

Why it matters

An analyst asked Copilot for average basket value by region. The query ran and the numbers looked sensible. It averaged each store's average, so a pop-up store with four baskets counted as much as a flagship with 40,000. The regional figure was 11% too high. A five-row test with two stores of different sizes shows the difference at once.

Worked example: weak code

```
1 # Copilot suggestion, applied after it ran without error
2 region_basket = (
3     store_summary.join(dim_region, "store_id")
4     .groupBy("region")
2     .agg(F.avg("avg_basket").alias("avg_basket"))
6 )
```

1. **Line 1.** Running is not a check.
2. **Line 5.** Average of averages: every store weighs the same.

Worked example: strong code

```
1  from pyspark.sql import functions as F
2
3  region_basket = (
4      store_summary.join(dim_region, "store_id", "left")
5      .groupBy("region")
1      .agg(F.try_divide(F.sum("total_sales"),
       F.sum("basket_count")).alias("avg_basket"))
7  )
8
9  # Known data: store A 4 baskets, 40.00 total. Store B 400 baskets,
     2,000.00 total.
2  # Hand answer: 2,040.00 / 404 = 5.0495...  Average of averages would give
     7.50.
```

```
11 assert abs(result_for("North") - 2040.0 / 404) < 1e-9
```

1. **Line 6.** Total over total: each basket counts once. try_divide returns null for a zero denominator, where plain division fails under ANSI mode.
2. **Line 10.** Known data picked so the wrong method gives a visibly different answer.

Read it, run it on data with a hand-worked answer, compare it with a second method, and keep those checks as tests.

Recap card

- **A Copilot query ran without error. What does that prove?** Syntax parses and names exist. Nothing about correctness.
- **Four checks for Copilot output** Read every line. Run on known data. Compare with a second method. Keep the checks as tests.
- **Average of averages** Weights small groups like large ones. Divide total by total instead.
- **Known data for verification** Five to ten hand-built rows with a null, a zero denominator, a duplicate key and an empty group.

Practice

Exercise 1. Multiple choice (level 1)

A Copilot-generated query runs without an error. What has that shown?

- a) The results are correct
- b) The syntax parses and the referenced names exist
- c) The query is efficient
- d) The grain matches the requirement

Why?

- r1) An engine checks syntax and names before it runs a query. It does not know what answer you wanted.
- r2) Copilot tests its own queries before it shows them.

Exercise 2. Spot the bug (level 2)

Copilot wrote this T-SQL for average basket value by region. `store_summary` has one row per store with `total_sales`, `basket_count` and `avg_basket`. Tap the line that gives the wrong number, then fix it. Guard the division.

```
1 SELECT r.region,
2        AVG(s.avg_basket) AS avg_basket
3 FROM dbo.store_summary AS s
```

```
4 LEFT JOIN dbo.dim_region AS r ON s.store_id = r.store_id
5 GROUP BY r.region;
```

Write down the line number of the bug. Then write the fix.

Exercise 3. Put in order (level 2)

Order the checks for a Copilot-generated transformation.

- Run it on a small dataset with a hand-worked answer
- Compare the result with a second method on real data
- Keep the known-data and comparison checks as tests
- Read every line: names, grain, join types

Exercise 4. Write the function (level 3)

Write `mismatches(candidate: Callable[[list[float]], float | None], reference: Callable[[list[float]], float | None], cases: list[list[float]]) -> list[int]`. Run both functions on each case. Return the indexes of cases where the candidate raises an exception, or where the results differ. Two `None` results match. Floats match within `math.isclose` defaults. Import `Callable` from `collections.abc`.

What you need to know.

`Callable` from `collections.abc` types a parameter that holds a function. `Callable[[list[float]], float | None]` takes a list of floats and returns a float or `None`. Call the parameter like any function.

`try: ... except Exception:` catches any error the call raises, so one failing case does not stop the loop:

```python
from collections.abc import Callable

def safe(fn: Callable[[int], float], x: int) -> float | None:
    try:
        return fn(x)
    except Exception:
        return None
```

Your code must pass these checks:

- Report the index where the results differ.
- A candidate that raises on a case is a mismatch, not a crash of the harness.
- 0.1 + 0.2 and 0.3 differ in the last bit. Compare floats with math.isclose.
- None and 0.0 are different results.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```python
# Indexes of cases where a candidate function disagrees with a reference.
def mismatches(
    candidate: Callable[[list[float]], float | None],
    reference: Callable[[list[float]], float | None],
    cases: list[list[float]],
) -> list[int]: ...
```

Starter code:

```python
def mismatches(candidate, reference, cases):
    return [i for i, case in enumerate(cases) if candidate(case) !=
    reference(case)]
```

Exercise 5. Architecture design (level 4)

Copilot wrote a PySpark step that deduplicates survey respondents and computes weighted reach per wave. Plan its verification before it goes into the silver pipeline.

Options: 8 rows: a duplicate respondent, a null weight, a wave with zero total weight, A 1% random sample of production.

1. Known dataset
2. Second method
3. After it passes

Sources

- Copilot for Data Engineering and Data Science is preview; always review AI-generated code; output might be inaccurate, incomplete or based on outdated library syntax; chat pane uses an approval diff. (verified: https://learn.microsoft.com/en-us/fabric/data-engineering/copilot-notebooks-overview (updated 2026-04-01))
- With ANSI mode on, Spark division by zero raises DIVIDE_BY_ZERO; F.try_divide returns null instead. (verified: Local run on PySpark 4.x (practice venv), September 2026; error message names try_divide.)

C9 Governance and security

Roadmap: Months 16 to 18: MLOps and deployment; Current work. Domains, OneLake security, sensitivity labels, Purview and lineage.

C9-L1 Domains and endorsement, finding the right data

Concept

A tenant with 400 workspaces needs two things before anyone trusts its data: a way to find data by business area, and a way to tell the authoritative copy from the rest. Fabric gives you **domains** and **endorsement**. Microsoft Learn shows neither with a preview label (checked September 2026).

Domains group workspaces by business area, such as Finance or Marketing. You can split a domain into **subdomains**.

- You associate **workspaces** with a domain. Every item in the workspace inherits the domain.
- Users filter the **OneLake catalog** by domain to find relevant data.
- Some tenant settings can be **delegated** to domain level, so each area sets its own rules.
- **Domain assignment does not grant or restrict access.** Workspace roles and item permissions decide who sees what.

Three roles manage domains. **Fabric admins** create domains and name domain admins. **Domain admins**, ideally the business owners, edit their domain and associate workspaces. **Domain contributors** are workspace admins allowed to assign their own workspaces to the domain.

Endorsement labels an item with one of three badges:

Badge	Meaning	Who applies it
Promoted	The creators think it is ready to share	Anyone with write access to the item
Certified	An authorised reviewer confirms it meets the organisation's standards	Only users a Fabric admin names
Master data	The authoritative source for core data such as customer lists	Only users a Fabric admin names

Any item except a Power BI dashboard can be promoted or certified. Master data applies only to items that hold data, such as lakehouses and semantic

models. Certification and master data work only after a Fabric admin enables them.

Endorsed items get a badge and appear first in some lists. That helps people pick the right copy. It does not stop anyone using an uncertified one.

Why it matters

A company had five semantic models called "Sales" across four workspaces. Two finance directors presented different quarterly revenue to the board. Both had used a model called "Sales". Certifying one model, retiring the others, and filing the finance workspaces under a Finance domain made the right model the first result anyone found.

Worked example: weak code

```
1  Workspaces: 400, no domains
1  Semantic models named "Sales": 5, none endorsed
2  Access plan: put finance workspaces in the Finance domain to restrict them
```

1. **Line 2.** Nothing tells users which copy is authoritative.
2. **Line 3.** Domains do not control access. Workspace roles do.

Worked example: strong code

```
1  Domains: Finance, Marketing, Operations (subdomain Finance > Tax)
2  Domain admins: business owners. Domain contributors: finance workspace
     admins
1  Certified: finance_sales_model (by a Fabric-admin-approved certifier)
4  Master data: crm_lakehouse, the single home of dim_customer
2  Access: workspace roles and OneLake security (lesson C9-L2)
```

1. **Line 3.** One authoritative model, labelled by an authorised reviewer.
2. **Line 5.** Discovery and access are separate controls.

Domains help people find data. Endorsement tells them which copy to trust. Neither controls access.

Recap card

- **What a domain does** Groups workspaces by business area for discovery and delegated settings. Does not control access.
- **Domain roles** Fabric admin creates domains. Domain admin edits one. Domain contributor, a workspace admin, assigns their workspaces.
- **Endorsement badges** Promoted (creators), Certified (named reviewers), Master data (named users, data items only). Dashboards get none.

- **Does endorsement block use of other copies?** No. It adds a badge and ranks the item first in some lists.

Practice

Exercise 1. Multiple choice (level 1)

You associate a workspace with the Finance domain. What changes for users outside finance?

- a) They lose access to the workspace
- b) Nothing about access. The workspace's items show under Finance in the OneLake catalog
- c) They gain read access to the items
- d) They must request access from the domain admin

Why?

- r1) Domain assignment does not affect item visibility or access. Workspace roles and item permissions do.
- r2) Domains only apply to Power BI items.

Exercise 2. Multiple choice (level 2)

A data engineer with Contributor access to a lakehouse wants to mark it as **Certified**. What happens?

- a) It works: Contributors can certify their items
- b) They can request certification. Only users a Fabric admin specifies can certify
- c) Certification is automatic once a data quality test passes
- d) Lakehouses cannot be certified

Why?

- r1) Any user can request certification, but only users specified by a Fabric administrator can certify.
- r2) Only workspace Admins can certify.

Exercise 3. Fill the gap (level 2)

Complete the domain roles.

```
1 Creates domains and names domain admins: Fabric _____(1)
2 Assigns their own workspace to a domain: domain _____(2), who must be a
    workspace admin
```

Exercise 4. Write the function (level 3)

Write `allowed_badges(item_type: str) -> set[str]` for the endorsement rules. Use the badge names `"promoted"`, `"certified"` and `"master_data"`. A `"dashboard"` gets no badges. Data items (`"lakehouse"`, `"warehouse"`, `"semantic_model"`, `"kql_database"`) get all three. Every other item type gets promoted and certified. Keep the data item types in a module constant.

Your code must pass these checks:

- Power BI dashboards cannot be promoted or certified.
- Items that hold data take all three badges.
- Master data applies only to items that hold data.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```python
# Module constant.
DATA_ITEM_TYPES = ...

# Module constant.
NO_BADGE_TYPES = ...

# Endorsement badges an item type can receive.
def allowed_badges(item_type: str) -> set[str]: ...
```

Starter code:

```python
def allowed_badges(item_type):
    return {"promoted", "certified", "master_data"}
```

Exercise 5. Architecture design (level 4)

Organise a tenant where finance has 30 workspaces, three copies of the customer table, and complaints that nobody finds the right sales model.

Options: A Finance domain with finance workspaces associated, finance leads as domain admins, A naming convention only: prefix finance workspaces with fin_.

1. Grouping
2. The customer table
3. Keeping non-finance users out of finance data

Sources

- Workspaces associate with domains and items inherit; OneLake catalog filters by domain; tenant settings can be delegated; domain assignment doesn't affect visibility or access; roles Fabric admin, domain admin, domain contributor (must be workspace admin); subdomains. (verified:

https://learn.microsoft.com/en-us/fabric/governance/domains (updated 2025-05-01))

- Badges Promoted, Certified, Master data; any item except dashboards can be promoted (by users with write permission) or certified; only admin-specified users certify or label master data; master data only for data items; must be enabled by a Fabric admin. (verified: https://learn.microsoft.com/en-us/fabric/governance/endorsement-overview (updated 2024-07-11))

C9-L2 OneLake security, roles down to rows and columns

Concept

OneLake security controls who reads which data in OneLake: tables, folders, schemas, rows and columns. Every supported engine enforces it (Microsoft Learn, checked September 2026).

Roles. Each role has a permission (Read or ReadWrite), the data it covers, and its members: Entra users or groups. The model is **deny by default** and supports only **grant** roles. There are no deny roles. A lakehouse starts with a **DefaultReader** role that gives users with the ReadAll permission access to its data. Remove or narrow it before your own roles mean anything.

Who it restricts. OneLake security governs **Viewers** and users with Read permission on the item. Workspace Admins, Members and Contributors already have data access through their workspace role. Make analysts Viewers.

Row and column security. A role can filter rows (RLS) and hide columns (CLS) on a table. Within one role, access is the **intersection** of its table, row and column rules. Across roles, access is the **union**: row filters combine with OR, so a user in roles for `city = 'Leeds'` and `city = 'York'` sees both cities. Two exceptions:

- In the **SQL analytics endpoint**, CLS uses deny semantics: if any of the user's roles hides a column, the endpoint blocks it.
- `SELECT *` on a table with hidden columns returns the allowed columns in Spark, but **fails** in the SQL analytics endpoint.

Keep row and column rules that belong together in **the same role**. OneLake rejects a combination where two roles allow different columns of a table and either role also filters its rows. Roles with ReadWrite cannot carry RLS or CLS.

Engines. RLS and CLS are GA in Spark notebooks, lakehouses, Direct Lake on OneLake semantic models, and the SQL analytics endpoint in **user's identity** access mode. Eventhouse supports RLS only, in preview.

Why it matters

A team added a OneLake security role limiting analysts to UK rows of `fact_sales`, then found the analysts still saw every country. Two causes. The analysts were Contributors on the workspace, so roles did not restrict them. And the DefaultReader role was still in place, granting full reads to every Viewer with ReadAll. Making analysts Viewers and removing DefaultReader fixed both.

Worked example: weak code

```
❶ Analysts' workspace role: Contributor
❷ Roles: DefaultReader (unchanged), uk_sales (fact_sales where country =
    'UK')
❸ Salary hiding: role hr_cols hides salary, role hr_rows filters team =
    'Ops'
```

1. **Line 1.** Contributors already have data access. OneLake roles do not restrict them.
2. **Line 2.** DefaultReader still grants full reads. The union of roles gives everything.
3. **Line 3.** Row and column rules split across roles. Keep them in one role.

Worked example: strong code

```
❶ Analysts' workspace role: Viewer
  2 Roles: DefaultReader removed. uk_sales (fact_sales where country = 'UK')
❷ Salary hiding: role hr_ops (dim_employee where team = 'Ops', columns
    without salary)
❸ SQL analytics endpoint: user's identity access mode
```

1. **Line 1.** Viewers are the users OneLake security governs.
2. **Line 3.** Rows and columns in one role: the intersection applies.
3. **Line 4.** The endpoint enforces OneLake security only in this mode.

Readers are Viewers, defaults are removed, related row and column rules share a role, and every engine runs in a mode that enforces them.

Recap card

- **OneLake security combines roles how?** Within a role: intersection of table, row and column rules. Across roles: union. Row filters OR together.
- **Whom does OneLake security restrict?** Viewers and Read-permission users. Admins, Members and Contributors already have data access.

- **SELECT * with a hidden column** Spark returns allowed columns. SQL analytics endpoint returns an error.
- **Engines enforcing OneLake RLS and CLS (GA)** Spark, lakehouse, Direct Lake on OneLake, SQL analytics endpoint in user's identity mode. Eventhouse: RLS only, preview.

Practice

Exercise 1. Multiple choice (level 1)

A user belongs to two OneLake security roles on `fact_sales`: one filters `region = 'North'`, the other `region = 'South'`. Which rows do they see?

- a) North and South rows
- b) No rows: the filters contradict each other
- c) North rows only: the first role wins
- d) All rows

Why?

- r1) Roles combine as a union. Row filters from different roles combine with OR.
- r2) OneLake picks the most permissive single role.

Exercise 2. Multiple choice (level 2)

An analyst is a **Contributor** on the workspace. A OneLake security role limits them to UK rows. What do they see in Spark?

- a) UK rows only
- b) All rows: Contributors already have data access through their workspace role
- c) No rows
- d) An error

Why?

- r1) OneLake security governs Viewers and users with Read permission. Admins, Members and Contributors already have data access.
- r2) Spark ignores OneLake security.

Exercise 3. Multiple choice (level 2)

A table hides the `salary` column from a user. They run `SELECT * FROM dim_employee` in Spark, then in the SQL analytics endpoint. What happens?

- a) Spark returns the allowed columns. The endpoint returns an error

- b) Both return the allowed columns
- c) Both return an error
- d) Both return salary as null

Why?

- r1) Microsoft Learn documents that CLS behaves differently: Spark shows only allowed columns, the endpoint blocks the query.
- r2) Spark does not support column security.

Exercise 4. Write the function (level 3)

Write `visible_columns(roles: list[dict[str, set[str]]], table: str, engine: Literal["spark", "sql_endpoint"]) -> set[str]`. Each role maps table names to the columns it allows. Consider only roles that include `table`. If none do, return an empty set (deny by default). For `"spark"`, return the union of the allowed columns. For `"sql_endpoint"`, return the intersection, because a column hidden by any role is blocked. Raise `ValueError` for any other engine.

What you need to know.

`Literal` from `typing` limits a parameter to fixed values: `mode: Literal["fast", "safe"]` tells readers and mypy that only those two strings are valid. Python does not check it at run time, so a caller can still pass any string. Raise an error yourself when a wrong value must not get through.

`set().union(*sets)` combines sets: every item in any of them. `set.intersection(*sets)` keeps only items in all of them.

Your code must pass these checks:

- Spark combines column access across roles as a union.
- The SQL analytics endpoint blocks a column hidden by any role: intersect.
- Ignore roles that do not include the table. No role for the table means no access.
- Unknown engines raise ValueError.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```python
# Columns a user sees in one table across their OneLake security roles.
def visible_columns(
    roles: list[dict[str, set[str]]],
    table: str,
    engine: Literal['spark', 'sql_endpoint'],
) -> set[str]: ...
```

Starter code:

```python
def visible_columns(roles, table, engine):
    columns = set()
    for role in roles:
        columns |= role[table]
    return columns
```

Exercise 5. Architecture design (level 4)

HR analysts must see employee rows for their own region, without the `salary` column, in Spark and through a Direct Lake report.

Options: Viewer, Contributor.

1. Analysts' workspace role
2. Role design
3. Semantic model mode
4. DefaultReader role

Sources

- Role components (Read/ReadWrite, data, members); deny by default; grant roles only; DefaultReader role; default roles and ReadWrite matter only for Viewers since Admin/Member/Contributor have write access; ReadWrite roles cannot contain RLS/CLS. (verified: https://learn.microsoft.com/en-us/fabric/onelake/security/data-access-control-model (updated 2026-08-28); get-started-security (updated 2026-09-01))
- Within a role OLS∩RLS∩CLS; across roles union; RLS combines with OR; SQL analytics endpoint intersects CLS; roles with different columns plus RLS on the same table unsupported. (verified: https://learn.microsoft.com/en-us/fabric/onelake/security/data-access-control-model (updated 2026-08-28))
- SELECT * with hidden columns: Spark shows allowed columns, SQL analytics endpoint returns an error. (verified: https://learn.microsoft.com/en-us/fabric/onelake/security/table-column-row-security (updated 2026-08-19))
- RLS/CLS GA in lakehouse, Direct Lake on OneLake, Spark notebooks, SQL analytics endpoint in user's identity mode; Eventhouse RLS only in public preview; third-party engines preview. (verified: https://learn.microsoft.com/en-us/fabric/onelake/security/read-secured-data (updated 2026-08-28))

C9-L3 Sensitivity labels, Purview and lineage

Concept

Access control decides who opens an item. **Sensitivity labels** say how sensitive its content is, and travel with it. Fabric uses labels from **Microsoft Purview Information Protection**. You need a Purview licence, and labelling Power BI items also needs a Pro or Premium Per User licence (Microsoft Learn, checked September 2026).

How labels spread:

- **Downstream inheritance.** A label on a semantic model flows to the reports and dashboards built on it. It never overwrites a label someone applied manually, and never replaces a label with a less restrictive one.
- **Default and mandatory labelling.** Admins can apply a default label to new items, or require users to pick one before they save.
- **Export.** Labels, and the protection a Purview policy attaches, stay with data exported to Excel, PDF and PowerPoint.

Protection policies link a label to access. When an item carries the label, users and groups named in the policy keep the permissions they already have. Everyone else is blocked. The user who last applied the label is not blocked.

Purview also gives you:

- **Unified Catalog**: metadata about Fabric items, visible in Purview.
- **Data loss prevention (DLP)**: detects sensitive data uploaded to OneLake and raises policy tips and alerts. It also restricts access to sensitive assets in KQL databases, SQL databases and warehouses.
- **Audit**: compliance admins track label activity.

Lineage shows how items connect. Every workspace has a lineage view that any user with a workspace role can open. It covers all items in the workspace, plus external sources one step upstream. **Impact analysis** starts from one item and lists the downstream items and workspaces a change might affect, and lets you notify their contacts.

Run impact analysis before a schema change. A label names the risk. A protection policy acts on it.

Why it matters

An engineer renamed a column in a lakehouse table on a Friday. Monday morning, 14 reports in three other workspaces showed errors, and a finance close was delayed. Impact analysis on the table would have listed all 14 reports and their workspaces, and notified their owners before the change.

Worked example: weak code

```
  1 Change: rename cust_id to customer_id in fact_sales
(1) Checks: none, tested in the dev lakehouse notebook
(2) Label on hr_salaries model: none, "the workspace is private"
```

1. **Line 2.** Your notebook works. The 14 downstream reports break.
2. **Line 3.** Exports to Excel leave the workspace. A label and policy would travel with them.

Worked example: strong code

```
  1 Change: rename cust_id to customer_id in fact_sales
(1) Checks: impact analysis lists 14 reports in 3 workspaces. Contacts
      notified. Views keep cust_id for one release
(2) Label on hr_salaries model: Highly Confidential, with a protection policy
      for the HR group
(3) Downstream: reports inherit Highly Confidential. Excel exports keep the
      label
```

1. **Line 2.** Know who breaks, tell them, and give them time.
2. **Line 3.** The policy blocks anyone outside the HR group, even if someone shares the item by mistake.
3. **Line 4.** Protection follows the data out of Fabric.

Run impact analysis before changes. Label sensitive items, attach protection policies, and let inheritance carry the label downstream.

Recap card

- **Downstream label inheritance rules** Flows to downstream items. Never overwrites a manual label. Never downgrades.
- **Protection policy** Tied to a label. Named users and groups keep existing permissions. Everyone else is blocked.
- **Lineage view** Every workspace has one. All items in the workspace plus external sources one step upstream.
- **Impact analysis** Downstream items and workspaces a change might affect, with a way to notify their contacts.

Practice

Exercise 1. Multiple choice (level 1)

A report built on a **Confidential** semantic model has a **Highly Confidential** label that someone applied by hand. The model's label changes to **General**. What happens to the report's label?

- a) It becomes General
- b) It stays Highly Confidential
- c) It becomes Confidential
- d) It is removed

Why?

- r1) Downstream inheritance never overwrites a manually applied label, and never downgrades a label.
- r2) Reports never inherit labels.

Exercise 2. Multiple choice (level 2)

You plan to change a column type in a lakehouse table. Which tool lists every downstream item and workspace that the change might break?

- a) Impact analysis
- b) The OneLake catalog domain filter
- c) Sensitivity labels
- d) The Capacity Metrics app

Why?

- r1) Impact analysis shows affected downstream items and workspaces and can notify their contacts.
- r2) Impact analysis blocks the change until owners approve.

Exercise 3. Fill the gap (level 2)

Complete the protection policy behaviour.

```
Item labelled Highly Confidential, policy allows the HR group.
HR group member with Viewer access: _____(1) their Viewer access.
Finance user with Viewer access: _____(2).
```

Exercise 4. Write the function (level 3)

Write `inherit_label(current: str | None, manual: bool, upstream: str, order: list[str]) -> str | None`. It returns the downstream item's label after its upstream item's label becomes `upstream`. `order` lists labels from least to

most restrictive. Rules: a manual label is kept. An unlabelled item takes `upstream`. Otherwise the item takes `upstream` only if it is more restrictive than `current`. Raise `ValueError` if a label is not in `order`.

Your code must pass these checks:

- Inheritance fills in and raises labels.
- Inheritance never replaces a label with a less restrictive one.
- Inheritance never overwrites a manually applied label.
- Unknown labels raise ValueError.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```python
# Downstream label after the upstream label changes.
def inherit_label(
    current: str | None,
    manual: bool,
    upstream: str,
    order: list[str],
) -> str | None: ...
```

Starter code:

```python
def inherit_label(current, manual, upstream, order):
    return upstream
```

Exercise 5. Architecture design (level 4)

Protect a payroll semantic model and its reports. Analysts outside HR sometimes receive shared links by mistake, and HR exports summaries to Excel.

Options: Highly Confidential, with reports inheriting it downstream, No label, the workspace is restricted.

1. Label on the payroll model
2. Access control for mis-shared links
3. Before renaming a payroll column

Sources

- Labels come from Purview Information Protection; Purview licence plus Pro or PPU needed to label Power BI items; labels persist on export to Excel, PDF, PowerPoint; default and mandatory labelling exist. (verified: https://learn.microsoft.com/en-us/fabric/governance/information-protection (updated 2026-07-13))

- Downstream inheritance never overwrites manual labels or downgrades to a less restrictive label. (verified: https://learn.microsoft.com/en-us/fabric/governance/service-security-sensitivity-label-downstream-inheritance (updated 2026-07-13))
- Protection policies: users in the policy retain existing permissions, others blocked; label issuer exempt. (verified: https://learn.microsoft.com/en-us/fabric/governance/protection-policies-overview (updated 2025-12-03))
- Purview Unified Catalog, DLP on OneLake uploads and access restriction for KQL, SQL database, warehouse; audit of label activity. (verified: https://learn.microsoft.com/en-us/fabric/governance/microsoft-purview-fabric (updated 2026-02-09))
- Every workspace has a lineage view accessible to any workspace role; shows workspace items and external sources one step upstream; impact analysis lists downstream items and workspaces and notifies contacts. (verified: https://learn.microsoft.com/en-us/fabric/governance/lineage (updated 2023-11-06); impact-analysis (updated 2023-05-23))

C9-L4 Audit evidence, Purview scans, label coverage and the audit log

Concept

An auditor asks three questions about talent data. What do you hold? How is it protected? Who touched it? Answer each with evidence from a system, dated, not with a statement of intent. Fabric and Purview give you one source for each.

What you hold: a Purview Data Map scan. Register the Fabric tenant in the Microsoft Purview Data Map and scan it, in full, incrementally, or scoped to chosen workspaces. The scan brings in item metadata and lineage. For lakehouses it also reads tables and files. Two limits matter. The scan does not classify or label Fabric data. And once metadata reaches Purview, Purview permissions, not Fabric roles, decide who sees it, so restrict the collection that holds HR items.

How it is protected: label coverage. The **Govern** tab of the OneLake catalog reports the share of items with no sensitivity label, by workspace and item type. It refreshes once a day, and it does not count tables inside an item.

Who touched it: the audit log. Fabric records activities in the Microsoft Purview **unified audit log**. You need the Audit Logs role to search it. Export results to CSV, or script the search with `Search-UnifiedAuditLog`. **Audit (Standard)** keeps records for 180 days. **Audit (Premium)** keeps them up to a year by policy, longer with an add-on licence.

If your audit period is longer than your retention, export on a schedule into storage you control.

An evidence pack holds the scan date and scope, label coverage per workspace with the gaps listed, and the access records for sensitive items, each with the query that produced it. Status: the scan and the audit log are generally available. The Govern tab has no preview label but is still rolling out by region (Microsoft Learn, November 2026).

Why it matters

An internal audit asked who had opened the salary semantic model in the last financial year. The team ran an audit search in month 11. Audit (Standard) returned only the last six months, because the first five had expired. The finding read "access cannot be evidenced for 5 of 12 months". A monthly export would have closed it.

Worked example: weak code

```
# Evidence pack, HR workspace
inventory: we know what is in there
labels: all sensitive items are labelled
access: run an audit search when the auditor asks
```

1. **Line 2.** A belief, not evidence. Nothing shows when it was checked or what it covered.
2. **Line 3.** No coverage figure and no list of gaps.
3. **Line 4.** Records past the 180-day retention will be gone by then.

Worked example: strong code

```
# Evidence pack, HR workspace, period April 2026 to March 2027
inventory:
  source: Purview Data Map scan, scoped to HR workspaces
  run: 2027-04-02, incremental, 214 items
labels:
  source: OneLake catalog, Govern, label coverage
  coverage: 97% labelled, 6 unlabelled items listed with owners
access:
  source: unified audit log, monthly CSV export to the evidence store
  query: activities on hr_salaries_model, all users, each month
```

```
❹ collection_access: Purview collection restricted to HR data stewards
```

1. **Line 4.** Dated, scoped and counted. An auditor can repeat it.
2. **Line 7.** A figure plus the gaps, each with someone to fix it.
3. **Line 9.** Monthly exports keep a year of records that Audit (Standard) would drop after 180 days.
4. **Line 11.** Scanned metadata follows Purview permissions, so restrict who sees it.

Answer what you hold, how it is protected and who touched it with dated system evidence. Export audit records before they expire, and restrict who sees scanned HR metadata in Purview.

Recap card

- **Three audit questions and their sources** What you hold: a Purview Data Map scan. How it is protected: label coverage in the OneLake catalog. Who touched it: the unified audit log.
- **Audit (Standard) retention** 180 days. Audit (Premium) keeps up to a year by policy, longer with an add-on. Export on a schedule for longer periods.
- **Who sees scanned Fabric metadata in Purview?** Purview permissions decide, not Fabric roles. Restrict the collection that holds HR items.
- **What a Fabric scan does not do** It does not classify or label Fabric data. It brings in metadata and lineage.

Practice

Exercise 1. Multiple choice (level 1)

Which source answers "who opened the salary report last month?"

- a) The unified audit log in Microsoft Purview
- b) A Purview Data Map scan
- c) Label coverage in the OneLake catalog
- d) The lineage view

Exercise 2. Multiple choice (level 2)

Your audit period is 12 months and you have Audit (Standard). What should you do?

- a) Export audit records on a schedule into storage you control
- b) Search the log once, at the end of the period
- c) Nothing, because Audit (Standard) keeps records for a year
- d) Rely on the Data Map scan instead

Why?

- r1) Audit (Standard) keeps records for 180 days, so the first months would expire before the end.
- r2) Exports are needed because the portal cannot search more than seven days.

Exercise 3. Fill the gap (level 2)

Complete the three questions and their sources.

```
1 What do you hold?          a Purview Data Map _____(1) of the Fabric
    tenant
2 How is it protected?       label _____(2) in the OneLake catalog Govern
    tab
3 Who touched it?            the unified _____(3) log
```

Exercise 4. Write the function (level 3)

Write `label_coverage(items: list[dict]) -> dict[str, float]`. Each item is a dict with `"workspace"` and `"label"`, where `"label"` is a label name or `None`.

Return, for each workspace that has items, the share of its items with a label, from 0.0 to 1.0. Return an empty dict for an empty list. Raise `ValueError` if an item has no `"workspace"` key.

What you need to know.

`dict.setdefault(key, default)` returns the value for `key`, adding `default` first if the key is missing:

```
1 counts = {}
2 counts.setdefault("red", [0, 0])[0] += 1
3 counts   # {"red": [1, 0]}
```

`"key" in some_dict` checks whether a key exists: `"a" in {"a": 1}` is `True`.

Your code must pass these checks:

- Return the share of labelled items for each workspace.
- An empty list has no workspaces, so return an empty dict without dividing by zero.
- Raise ValueError when an item has no workspace.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```
1 # Share of items with a sensitivity label, per workspace.
2 def label_coverage(items: list[dict]) -> dict[str, float]: ...
```

Starter code:

```
1 def label_coverage(items):
```

```
    labelled = sum(1 for item in items if item["label"])
    return {"all": labelled / len(items)}
```

Exercise 5. Architecture design (level 4)

Design the evidence process for the HR workspaces before the first annual audit.

Options: A scheduled Purview scan scoped to HR workspaces, with its date and item count kept, A spreadsheet the team updates when it remembers.

1. Inventory
2. Access records
3. Who sees scanned HR metadata in Purview

Sources

- A Purview Data Map scan of a same-tenant Fabric tenant supports full, incremental and scoped scans and brings in metadata and lineage. For items other than Power BI, only item-level metadata and lineage, with sub-item metadata for lakehouse tables and files. Once metadata is in Purview, Purview permissions, not Fabric permissions, decide who sees it. (verified: https://learn.microsoft.com/en-us/purview/register-scan-fabric-tenant (ms.date 2026-04-28, updated 2026-07-31))
- The Data Map source table lists Fabric with no automatic classification and no sensitivity labels applied to Data Map assets, with lineage and live view supported. (verified: https://learn.microsoft.com/en-us/purview/data-map-data-sources (2026-07-01))
- The OneLake catalog Govern section reports sensitivity label coverage and unlabelled items, refreshes from admin monitoring storage once a day, and does not include subitems such as tables. It is rolling out by region, with no preview label on the page. (verified: https://learn.microsoft.com/en-us/fabric/governance/onelake-catalog-govern (ms.date 2026-08-26))
- Fabric activities are in the Microsoft Purview audit log; searching it needs the Audit Logs role. Audit (Standard) retains 180 days; Audit (Premium) up to one year by policy and ten years with an add-on licence; Search-UnifiedAuditLog and CSV export are available. (verified: https://learn.microsoft.com/en-us/fabric/admin/track-user-activities (2026-02-12) and https://learn.microsoft.com/en-us/purview/audit-solutions-overview (2026-05-18))

C10 DevOps and cost

Roadmap: Months 16 to 18: MLOps and deployment; Current work. Git integration, deployment pipelines and capacity monitoring.

C10-L1 Git integration for Fabric workspaces

Concept

Git integration connects a Fabric workspace to a branch and folder in a Git repository. Supported providers are Azure DevOps, GitHub and GitHub Enterprise, all cloud-based only. Some item types are still in preview, so check the supported-items list for each one you use (Microsoft Learn, checked September 2026).

What Git holds. Item definitions, not data. Each item gets its own folder of definition files. For a lakehouse, Git tracks metadata only: its name, shortcuts and OneLake security roles. Tables and files are not tracked, and Git operations never overwrite them. Rebuilding a workspace from Git gives you empty lakehouses. Your pipelines must reload the data.

Two directions:

- **Commit** exports workspace changes to the branch.
- **Update** imports branch changes into the workspace and overwrites the workspace version.

The source control panel shows each item's state. If an item changed in both places, you have a **conflict** to resolve before you continue.

Who does what. Only a workspace **Admin** connects or disconnects the workspace. Members and Contributors commit and update, limited by their permissions in the Git repository. Viewers see no Git information.

One workspace, one branch. A workspace is a shared runtime. Anything one person changes there affects everyone using it. Microsoft's recommended pattern is a **separate workspace per developer**, each connected to its own feature branch. You develop in your workspace, commit to your branch, and open a pull request into the main branch. The shared workspace connected to main then updates from Git.

Alternatively, develop in client tools such as VS Code for notebooks or Power BI Desktop for reports, and push to your branch.

Keep notebooks small and parameterised, and keep configuration in files your pipeline reads. Reviewers read diffs. A 900-line notebook with hard-coded paths produces diffs nobody can review.

Why it matters

Three engineers shared one development workspace connected to main. One of them updated from Git while another had an hour of uncommitted notebook edits. The update overwrote the workspace version, and the edits were lost. With a workspace and branch per developer, each person's work stays isolated until it merges.

Worked example: weak code

```
❶ Workspace: sales_dev, connected to main, used by 3 engineers
❷ Process: edit in sales_dev, commit to main when it works
❸ Recovery plan: restore the lakehouse from Git
```

1. **Line 1.** A shared runtime: one person's update overwrites another's work.
2. **Line 2.** No review before main.
3. **Line 3.** Git holds lakehouse metadata only. Tables and files are not in Git.

Worked example: strong code

```
❶ Workspaces: sales_dev_ana (branch feature/ana-dedupe), sales_dev_raj
    (branch feature/raj-gold)
❷ Process: commit to your feature branch, pull request into main, review,
    merge
  3 Shared: sales_integration connected to main, updated from Git after each
    merge
❸ Recovery plan: redeploy items from Git, then rerun ingestion pipelines to
    reload data
```

1. **Line 1.** One workspace per developer, one branch per workspace.
2. **Line 2.** Changes reach main through review.
3. **Line 4.** Items come from Git. Data comes from pipelines.

One workspace per developer on its own branch, pull requests into main, and a data reload plan, because Git holds definitions, not data.

Recap card

- **What Git integration stores for a lakehouse** Metadata only: name, shortcuts, OneLake security roles. Not tables or files.
- **Commit vs Update** Commit exports workspace changes to Git. Update imports Git changes and overwrites the workspace version.

- **Who connects a workspace to Git?** Admin only. Members and Contributors commit and update. Viewers see no Git information.
- **Recommended development process** One workspace per developer on its own branch. Pull requests into main.

Practice

Exercise 1. Multiple choice (level 1)

A lakehouse is connected to Git. What does the repository hold for it?

- a) The Delta tables and files
- b) Metadata only, such as its name, shortcuts and OneLake security roles
- c) Nothing: lakehouses are not supported
- d) A full backup, restorable to any commit

Why?

- r1) Microsoft Learn: only lakehouse metadata is tracked. Git operations never overwrite table or file data.
- r2) Git has a size limit that excludes tables.

Exercise 2. Multiple choice (level 2)

You click **Update** in a workspace. A notebook has an uncommitted edit in the workspace and a different change in Git. What does Fabric report?

- a) A conflict you must resolve
- b) Nothing: Git wins silently
- c) Nothing: the workspace wins silently
- d) The two versions merge automatically

Why?

- r1) An item changed in both the workspace and the branch is a conflict, shown in the source control panel.
- r2) Fabric merges notebooks cell by cell.

Exercise 3. Multiple choice (level 2)

Which workspace role connects a workspace to a Git repository?

- a) Admin
- b) Member
- c) Contributor
- d) Viewer

Why?

- r1) Connect and disconnect are Admin operations. Members and Contributors commit and update.
- r2) Only Admins have Git accounts.

Exercise 4. Write the function (level 3)

Your team's rule for the source control panel: resolve conflicts first, then commit your changes, then update from Git. Write `next_step(statuses: dict[str, str]) -> str`. Values are `"synced"`, `"uncommitted"`, `"update_required"` or `"conflict"`. Return `"resolve"` if any item is in conflict, else `"commit"` if any is uncommitted, else `"update"` if any needs an update, else `"none"`. Raise `ValueError` for an unknown status. Use a constant for the allowed statuses.

Your code must pass these checks:

- Conflicts come before everything else.
- Commit your changes before you update, or the update overwrites them.
- Update when only Git changed. Nothing to do when everything is synced.
- Unknown statuses raise ValueError.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```
# Module constant.
ALLOWED_STATUSES = ...

# Next source control action for a workspace.
def next_step(statuses: dict[str, str]) -> str: ...
```

Starter code:

```
def next_step(statuses):
    if "uncommitted" in statuses.values():
        return "commit"
    return "update"
```

Exercise 5. Architecture design (level 4)

Set up source control for a team of four engineers building notebooks and pipelines in Fabric.

Options: One workspace per engineer, each on its own feature branch, One shared dev workspace connected to main.

1. Development workspaces
2. Path to main

3. After a workspace is rebuilt from Git

Sources

- Providers Azure DevOps, GitHub, GitHub Enterprise (cloud only); some item types preview; workspace structure and subfolders preserved. (verified: https://learn.microsoft.com/en-us/fabric/cicd/git-integration/intro-to-git-integration (updated 2026-07-21))
- Commit exports and overwrites Git content; update overwrites workspace content; Admin connects; Member/Contributor commit and update per Git role; Viewer sees no Git info. (verified: https://learn.microsoft.com/en-us/fabric/cicd/git-integration/git-integration-process (updated 2026-06-15))
- A workspace connects to a single branch; recommended separate workspace per developer (branched workspace) or client tools. (verified: https://learn.microsoft.com/en-us/fabric/cicd/git-integration/manage-branches (updated 2026-06-15))
- Lakehouse Git tracks metadata only (name, shortcuts, data access roles); tables and files not tracked and never overwritten. (verified: https://learn.microsoft.com/en-us/fabric/data-engineering/lakehouse-git-deployment-pipelines (updated 2026-05-08))
- A conflict occurs when the same item changes in both the workspace and the repository; Commit is disabled until it is resolved. (verified: https://learn.microsoft.com/en-us/fabric/cicd/git-integration/conflict-resolution (updated 2026-06-15))

C10-L2 Deployment pipelines, dev to test to production

Concept

Git integration (lesson C10-L1) versions your items. **Deployment pipelines** move them between environments. A pipeline has **2 to 10 stages**, with Development, Test and Production as the default three. Each stage is a workspace (Microsoft Learn, checked September 2026).

What a deployment copies. When you deploy from one stage to the next, Fabric copies item content and metadata to the target stage. It does not copy **data**. After deploying a semantic model, you refresh it in the target stage. After deploying a lakehouse, you load its tables there.

Overwrite, not replace. If the item already exists in the target stage, Fabric overwrites its content. The item's **ID, URL and permissions stay the same**, so reports and links in production keep working.

Deployment rules give each stage its own settings:

- **Data source rules**: point a semantic model at the production database, not the test one.
- **Parameter rules**: set a parameter's value per stage.
- **Default lakehouse rules**: attach a notebook to the stage's own lakehouse.

You define a rule in the **target** stage, on the item. It applies from the next deployment into that stage.

Autobinding. If a deployed item depends on another item, such as a report on a semantic model, and the linked item exists in the target stage, Fabric binds to the target copy automatically.

Compare shows which items differ between two stages before you deploy. Use it every time. The deployment pipelines REST APIs automate the same steps from a build system.

Many teams combine the tools. Developers work in Git-connected workspaces and merge through pull requests. The merged branch updates the Development stage, and deployment pipelines promote tested content to Test and Production. Microsoft's CI/CD guidance also describes Git-only release options, so pick one and document it.

Why it matters

A team deployed a notebook from Test to Production. It still pointed at the test lakehouse, so for a week the production job wrote its gold tables into Test. The dashboard in Production read stale tables and nobody noticed. A default lakehouse rule on the notebook in the Production stage would have attached it to the production lakehouse on every deployment.

Worked example: weak code

```
1 Stages: Development, Production
1 Notebook nb_gold: default lakehouse lh_test (hard-coded)
3 Semantic model sales_model: data source test SQL database
2 Before deploying: nothing
```

1. **Line 2.** Production runs against test data.
2. **Line 4.** No compare step: you learn what changed after it breaks.

Worked example: strong code

```
1 Stages: Development, Test, Production
1 Rule in Production on nb_gold: default lakehouse = lh_prod
3 Rule in Production on sales_model: data source = production SQL database
```

```
❷ Before deploying: compare Test with Production, review the differing items
❸ After deploying: refresh sales_model, run the load pipeline
```

1. **Line 2.** Rules live in the target stage and apply on every deployment.
2. **Line 4.** Know what changes before it changes.
3. **Line 5.** Deployment copies items, not data.

Stage-specific settings belong in deployment rules. Compare before you deploy. Reload data after, because deployments copy items, not data.

Recap card

- **Deployment pipeline stages** 2 to 10 stages, default Development, Test, Production. One workspace per stage.
- **What a deployment copies** Item content and metadata. Not data: refresh models and reload tables after.
- **Deployment rule types** Data source, parameter, default lakehouse. Defined in the target stage on the item.
- **Deploying over an existing item** Content is overwritten. ID, URL and permissions stay the same.

Practice

Exercise 1. Multiple choice (level 1)

You deploy a semantic model from Test to Production for the first time. What must you do next?

- a) Nothing: the data is copied with it
- b) Refresh the model in Production, because data is not copied between stages
- c) Recreate the report permissions
- d) Reconnect the workspace to Git

Why?

- r1) Microsoft Learn: a semantic model refresh is required because data isn't copied from one stage to another.
- r2) Deployment pipelines always clear data in Production.

Exercise 2. Multiple choice (level 2)

Where do you define a rule that points a notebook at `lh_prod` in the Production stage?

- a) In the Production stage, on the notebook, as a default lakehouse rule

- b) In the Development stage, on the notebook
- c) In the notebook's code, with an if statement on the workspace name
- d) In the Git repository

Why?

- r1) Rules are defined in the target stage on the item, and apply from the next deployment into that stage.
- r2) Rules are global to the pipeline.

Exercise 3. Put in order (level 2)

Order a release from Test to Production.

- Check row counts and a known report figure in Production
- Check the deployment rules in the Production stage
- Compare Test with Production and review differences
- Deploy Test to Production
- Refresh semantic models and run load pipelines in Production

Exercise 4. Write the function (level 3)

Model deployment rules. Write `deployed_settings(source: dict[str, str], rules: dict[str, dict[str, str]], stage: str) -> dict[str, str]`. `source` holds an item's settings in the source stage. `rules` maps a stage name to the settings its rules override. Return the settings in `stage`: the source values, with that stage's rules applied. Do not modify `source`. Raise `ValueError` if a rule names a setting the item does not have.

Your code must pass these checks:

- Apply the target stage's rules over the source values.
- A stage without rules keeps the source values.
- Do not modify the source settings: return a new dict.
- A rule for a setting the item lacks is a configuration error.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```python
# Item settings in a target stage after its deployment rules apply.
def deployed_settings(
    source: dict[str, str],
    rules: dict[str, dict[str, str]],
    stage: str,
) -> dict[str, str]: ...
```

Starter code:

```
def deployed_settings(source, rules, stage):
    source.update(rules[stage])
    return source
```

Exercise 5. Architecture design (level 4)

Design the release process for a team that uses Git integration and wants a Test stage signed off before Production.

Options: Workspace connected to main, updated from Git after each merged pull request, Engineers edit directly in the Development stage workspace.

1. Development stage
2. Promotion to Test and Production
3. After each Production deployment

Sources

- Deployment pipelines have 2 to 10 stages, default three. (verified: https://learn.microsoft.com/en-us/fabric/cicd/deployment-pipelines/intro-to-deployment-pipelines (updated 2026-07-17))
- Data isn't copied between stages, semantic model refresh required; existing items overwritten with ID, URL and permissions unchanged; autobinding to linked items in the target stage; REST APIs available. (verified: https://learn.microsoft.com/en-us/fabric/cicd/deployment-pipelines/understand-the-deployment-process (updated 2025-12-15))
- Rule types: data source, parameter, default lakehouse; defined in the target stage; apply from the next deployment. (verified: https://learn.microsoft.com/en-us/fabric/cicd/deployment-pipelines/create-rules (updated 2025-12-15))
- CI/CD options include Git-based release options and deployment pipelines; many organisations use a hybrid. (verified: https://learn.microsoft.com/en-us/fabric/cicd/manage-deployment (updated 2026-03-18))

C10-L3 Capacity, smoothing and throttling

Concept

Every Fabric operation draws capacity units (CUs) from its workspace's capacity (lesson C1-L1). Fabric lets operations **burst** above the capacity's size, then **smooths** the cost over future time (Microsoft Learn, checked September 2026).

Smoothing. Fabric measures usage in 30-second **timepoints**, 2,880 per day.

- **Interactive** operations, such as report queries, are smoothed over at least 5 minutes and up to 64 minutes.
- **Background** operations, such as scheduled Spark jobs, pipeline runs and model refreshes, are smoothed over **24 hours**.

Example: an F2 has 2 CUs, or 48 CU hours per day. A background job that uses 1 CU hour adds 1 / 48, about 2.1%, to every timepoint for the next 24 hours.

Throttling starts when smoothed usage borrows too much future capacity:

Future capacity already used	What happens
Up to 10 minutes	Overage protection: no throttling
Over 10, up to 60 minutes	Interactive jobs are delayed by 20 seconds
Over 60 minutes, up to 24 hours	Interactive jobs are rejected
Over 24 hours	All requests are rejected, background too

Background jobs keep running well after users feel the delay. A capacity deep in debt takes hours to recover, because only idle capacity pays it back.

Surge protection lets capacity admins reject background jobs earlier, before the capacity reaches deep throttling. At workspace level, it caps a workspace's share of CU use over a rolling 24 hours, unless you tag the workspace **Mission Critical**.

The Capacity Metrics app shows what happened. A capacity admin installs it and shares it. The **Health** page flags throttling and rejections across your capacities. The **Compute** page gives 14 days of utilisation by item and operation.

Practical rules: schedule heavy background jobs away from business hours, keep production on its own capacity, and check the Compute page before you buy a larger SKU.

Why it matters

A team scheduled a full rebuild of 40 gold tables at 08:00 on an F16. By 09:30 the capacity had borrowed more than an hour of future use, and Power BI reports started failing for every user. The rebuild itself kept running. Moving it to 02:00, and making it incremental, kept the morning reports responsive without a larger SKU.

Worked example: weak code

```
❶ Capacity: F16, shared by production reports and data science experiments
❷ Schedule: full rebuild of gold tables at 08:00 on weekdays
❸ Monitoring: users report slow dashboards
```

1. **Line 1.** Experiments draw from the same CUs as production.
2. **Line 2.** Heavy background work just before peak interactive use.
3. **Line 3.** You find out from users, after throttling has started.

Worked example: strong code

```
  1 Capacity: F16 for production only. Experiments on a separate F4
❶ Schedule: incremental gold build at 02:00
❷ Surge protection: background rejection threshold set on the F16
❸ Monitoring: Capacity Metrics app Health page checked daily, Compute page
     reviewed weekly
```

1. **Line 2.** Less work, smoothed over hours when few users are active.
2. **Line 3.** Background jobs are stopped before users are rejected.
3. **Line 4.** You see throttling in the data before users feel it.

Know which of your jobs are background, run them off-peak, protect production with surge protection, and read the Metrics app before you resize.

Recap card

- **Smoothing windows** Interactive: 5 to 64 minutes. Background: 24 hours. Timepoints are 30 seconds, 2,880 a day.
- **Throttling stages** Up to 10 min: none. 10 to 60: 20 s interactive delay. 60 min to 24 h: interactive rejection. Over 24 h: all rejected.
- **Surge protection** Capacity level: earlier background rejection. Workspace level: cap a workspace's CU share over 24 hours, except Mission Critical.
- **Capacity Metrics app** Installed by a capacity admin. Health page for throttling. Compute page for 14 days of usage.

Practice

Exercise 1. Multiple choice (level 1)

Over what period does Fabric smooth the CU cost of a background Spark job?

- a) 24 hours
- b) 5 minutes
- c) The job's own duration
- d) It is not smoothed

Why?

- r1) Microsoft Learn: background operations are smoothed over a 24-hour period.
- r2) Spark jobs run for exactly 24 hours.

Exercise 2. Multiple choice (level 2)

A capacity has used 90 minutes of future capacity. What happens to a user opening a Power BI report, and to a scheduled pipeline?

- a) The report is rejected. The pipeline still runs
- b) Both run with a 20-second delay
- c) Both are rejected
- d) Both run normally

Why?

- r1) Between 60 minutes and 24 hours, interactive jobs are rejected. Background rejection starts only above 24 hours.
- r2) Pipelines always have priority.

Exercise 3. Fill the gap (level 2)

A background job uses 2 CU hours on an F4. Complete the smoothing calculation.

```
F4 capacity per day: 4 CUs x 24 hours = _____(1) CU hours
Share of every timepoint for 24 hours: 2 / _____(1) = about _____(2)%
```

Exercise 4. Write the function (level 3)

Write `throttle_stage(future_minutes: float) -> str` returning the throttling stage for minutes of future capacity already used: `"none"` up to 10, `"interactive_delay"` above 10 up to 60, `"interactive_rejection"` above 60 up to 1,440 (24 hours), and `"background_rejection"` above that. Raise `ValueError` for negative input. Keep the limits in constants.

Your code must pass these checks:

- Four bands: 10 minutes, 60 minutes, 24 hours.
- Each limit is inclusive: exactly 10 minutes is still overage protection.
- Negative usage is invalid.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```python
# Module constant.
OVERAGE_PROTECTION_MINUTES = ...
```

```
3
4  # Module constant.
5  INTERACTIVE_DELAY_MINUTES = ...
6
7  # Module constant.
8  INTERACTIVE_REJECTION_MINUTES = ...
9
10 # Fabric throttling stage for the future capacity already consumed.
11 def throttle_stage(future_minutes: float) -> str: ...
```

Starter code:

```
1  def throttle_stage(future_minutes):
2      if future_minutes < 10:
3          return "none"
4      if future_minutes < 60:
5          return "interactive_delay"
6      return "interactive_rejection"
```

Exercise 5. Architecture design (level 4)

Your F32 throttles every weekday between 08:30 and 10:00. The Compute page shows a 06:00 full refresh of a large semantic model and data science notebooks running all day on the same capacity.

Options: Make it incremental and move it to 01:00, Keep it: refreshes are background and smoothed.

1. The 06:00 full refresh
2. Data science notebooks
3. Protection for report users

Sources

- Timepoints 30 s, 2,880 per day; interactive smoothed 5 to 64 minutes; background 24 hours; F2 1 CU hour example ~2.1%; throttling stages at 10 min, 60 min, 24 h with 20 s interactive delay. (verified: https://learn.microsoft.com/en-us/fabric/enterprise/throttling (updated 2026-08-14))
- Surge protection: capacity-level background rejection threshold; workspace-level CU cap over rolling 24 h with Mission Critical exclusion. (verified: https://learn.microsoft.com/en-us/fabric/enterprise/surge-protection (updated 2026-05-27))
- Capacity Metrics app installed by capacity admin; Health page; Compute page 14-day view. (verified: https://learn.microsoft.com/en-us/fabric/enterprise/metrics-app (updated 2026-01-14))

Track C best-practice checklist

☐ Check the status (GA or preview) of every Fabric feature before you depend on it.

☐ One copy of each dataset. Use shortcuts or mirroring before you copy.

☐ Lakehouse for Spark and files. Warehouse for T-SQL writes and multi-table transactions.

☐ The SQL analytics endpoint is read-only for table data. Write through Spark or a Warehouse.

☐ Semantic models use star schemas. DAX measures state their filter context.

☐ Direct Lake models are checked for fallback to DirectQuery.

☐ Every Copilot output is read, run on known data and compared against a second method.

☐ Workspaces map to domains. Access uses roles and OneLake security, not item sharing.

☐ Items live in Git. Changes move through deployment pipelines.

☐ Capacity use is monitored, and heavy jobs are scheduled off peak.

Appendix: solutions and answer keys

Attempt each exercise before you read its solution. Track D answer keys are here too.

C0-L1 Data platform vocabulary, files, tables, lakes and warehouses

C0-L1 exercise 1

Hints. (1) The data files are Parquet either way. (2) What sits in `_delta_log`? (3) A record of every write.

Answer. a) A transaction log that records every write as a numbered version Because: r1) The log tells readers which files form each version, so writes are all-or-nothing and old versions stay readable.

Delta is Parquet plus a transaction log. The log gives versions, all-or-nothing writes and reads of older versions.

Common wrong answers.

- `b`: Delta data files are ordinary Parquet files. The log beside them makes the table.
- `c`: Delta stores data as Parquet, which keeps types. CSV keeps none.
- `d`: Engines read Delta tables, but the table exists without any engine running.
- `a|r2`: Right part, wrong reason. Parquet does the compressing. The log records versions.

C0-L1 exercise 2

Hints. (1) Which format stores the column type? (2) What does a reader guess for `01234` in a text file? (3) CSV guesses a number. Parquet remembers text.

Answer.

```
1 1234 01234
```

CSV keeps only text, so the reader infers a number and the zero is gone. Parquet records that the column is text and returns it unchanged.

Common wrong answers.

- `01234 01234`: CSV stores no types, so read_csv guesses a number and drops the leading zero.

- `1234 1234`: Parquet stores the column's type, text, so the leading zero survives.

C0-L1 exercise 3

Hints. (1) A columnar, typed file format. (2) Lake plus warehouse. (3) Extract, load, then transform.

Answer.

```
A Delta table is a folder of Parquet files plus a transaction log.
A lakehouse keeps open-format tables in a lake, so Spark and SQL read the
    same data.
With ELT, you load raw data first and transform it inside the platform.
```

Delta is Parquet plus a log, a lakehouse puts open tables in a lake, and ELT loads before it transforms.

Common wrong answers.

- g1 = `CSV`; g2 = `lakehouse`; g3 = `ELT`: Delta data files are Parquet, which keeps types and compresses by column.
- g1 = `Parquet`; g2 = `warehouse`; g3 = `ELT`: A warehouse holds tables for SQL. Open tables in a lake, shared by Spark and SQL, make a lakehouse.
- g1 = `Parquet`; g2 = `lakehouse`; g3 = `ETL`: ETL transforms before loading. Loading first, then transforming, is ELT.

C0-L1 exercise 4

Hints. (1) Raw first, clean next, summary last. (2) Bronze, silver, gold. (3) Reports read the summary.

Answer.

1. The delivery file lands in the lake, unchanged.
2. Load it into a bronze table, as received, with audit columns.
3. Clean, rename and type it into a silver table.
4. Aggregate it into gold tables for reporting.
5. A semantic model and its reports read the gold tables.

Land, load raw to bronze, clean to silver, aggregate to gold, then report. Each stage keeps the one before it, so you can rebuild after a fix.

Common wrong answers.

- Bronze keeps the delivery as received. Cleaning comes after, in silver, so you can always rebuild from the original.
- Aggregate clean data. Gold builds on silver.

C0-L1 exercise 5

Hints. (1) Structure known or unknown? (2) One copy, several engines. (3) Seconds, not nights.

Answer.

- Unknown structure, kept cheaply: files in a lake.
- One typed table for several engines: Delta in a lakehouse.
- Seconds matter: streaming, not nightly batches.
- A CSV export copies the data and loses its types, so it fits none of these.

Raw, unknown files go to the lake, shared typed tables go to the lakehouse, and seconds-fresh data needs streaming.

Common wrong answers.

- shared = `csv_export`: A CSV export makes a second copy without types. Both engines can read the Delta table directly.
- telemetry = `lakehouse`: A lakehouse table can hold the readings, but getting them there within seconds is a streaming job.

C0-L2 Power BI basics, reports, semantic models and measures

C0-L2 exercise 1

Hints. (1) Which layer do all reports read from? (2) Calculations that adapt to filters are measures. (3) Measures live in the model.

Answer. a) The semantic model, by adding a measure Because: r1) Reports query the model, so a measure defined there is available to all of them.

Reports query the semantic model, so a measure there serves every report and adapts to each visual's filters.

Common wrong answers.

- `b`: A visual-level calculation lives in that one report. Other reports cannot reuse it.
- `c`: A measure adapts to any filter at query time. A column per filter does not scale.
- `d`: A page filter narrows what one page shows. It adds no calculation.
- `a|r2`: Right layer, wrong reason. With Direct Lake or DirectQuery the model stores no data copy. It is shared because reports query it.

C0-L2 exercise 2

Hints. (1) Row ratios: 1 / 2 and 9 / 10. (2) Average them: (0.5 + 0.9) / 2. (3) The measure: (1 + 9) / (2 + 10).

Answer.

```
0.7 0.83
```

The average of row ratios is 0.7, because the small row counts as much as the large one. The measure divides the totals: 10 / 12 = 0.83.

Common wrong answers.

- `0.83 0.83`: The column holds 0.5 and 0.9 per row. Their plain average is 0.7.
- `0.7 0.7`: The measure divides the totals: 10 / 12 is 0.83.

C0-L2 exercise 3

Hints. (1) The DAX function that divides safely. (2) Total the reach the same way you total the base. (3) `DIVIDE` and `SUM`.

Answer.

```
Weighted reach % =
DIVIDE ( SUM ( fact[reach] ), SUM ( fact[base] ) )
```

`DIVIDE` returns blank for a zero denominator, and summing both parts makes the ratio correct at any level of the report.

Common wrong answers.

- g1 = `DIVIDE`; g2 = `AVERAGE`: Divide total by total. An average of reach over a sum of base mixes two different levels.

C0-L2 exercise 4

Hints. (1) Which table holds numbers you add up? (2) Which tables have one row per key? (3) Describing tables are dimensions.

Answer.

- fact_response holds the events and numbers, one row per respondent per wave.
- dim_region and dim_wave describe keys, one row per key, and filter the facts through one-to-many relationships.

The fact table records events with numbers. Dimensions describe the keys and filter the facts.

Common wrong answers.

- region = `fact`: dim_region describes a key, one row per region. It filters the facts, so it is a dimension.
- responses = `dimension`: fact_response holds the numbers you add up, one row per event: a fact table.

C0-L2 exercise 5

Hints. (1) Data, then model, then report. (2) The slicer sets the filter context. (3) Filter first, then calculate.

Answer.

1. Gold Delta tables hold the responses in OneLake.
2. The semantic model uses those tables and relates dim_region to the fact table.
3. A slicer on the report page selects North.
4. The filter flows from dim_region to the fact rows for North.
5. The measure divides the summed reach by the summed base for those rows.

Data sits in tables, the model relates them, the slicer sets the filter, the filter narrows the facts, and the measure computes over the remaining rows.

Common wrong answers.

- The filter narrows the rows first. The measure then evaluates over what is left.
- The model describes tables that already exist at the source.

C0-L3 Two engines, one table, Spark notebooks and the SQL analytics endpoint

C0-L3 exercise 1

Hints. (1) Which engine writes lakehouse tables? (2) What does the SQL analytics endpoint do with INSERT? (3) Spark writes, the endpoint reads.

Answer. a) In a Spark notebook, with `df.write.mode("append").saveAsTable("respondents")` Because: r1) Spark writes lakehouse tables. The SQL analytics endpoint is read-only for table data.

Spark writes lakehouse tables. The SQL analytics endpoint queries them and is read-only for their data.

Common wrong answers.

- `b`: The SQL analytics endpoint does not write lakehouse table data. INSERT fails there.
- `c`: Reports read data. They do not load it.
- `d`: The SQL analytics endpoint is read-only for lakehouse table data.
- `a|r2`: Right place, wrong reason. It is about which engine can write, not speed.

C0-L3 exercise 2

Hints. (1) Which rows survive w > 0? (2) Then count rows per region. (3) collect returns Row objects.

Answer.

```
[Row(region='N', count=1), Row(region='S', count=1)]
```

The filter drops the zero weight, leaving one row per region. `count()` after `groupBy` adds a `count` column, and `collect()` returns `Row` objects.

Common wrong answers.

- `[Row(region='N', count=1), Row(region='S', count=2)]`: The filter runs first and drops the S row with w 0.0.
- `[('N', 1), ('S', 1)]`: collect() returns Row objects, which print with their field names.

C0-L3 exercise 3

Hints. (1) The reader method that takes a table name. (2) Add rows, do not replace them. (3) `table` and `append`.

Answer.

```
spark.read.table("staging_w14").write.mode("append").saveAsTable("respondents")
```

`spark.read.table(...)` reads a lakehouse table, and `mode("append")` adds rows instead of replacing them.

Common wrong answers.

- g1 = `table`; g2 = `overwrite`: overwrite replaces the rows already in the table. The task keeps them: append.

C0-L3 exercise 4

Hints. (1) Start with `SELECT region, SUM(final_weight) AS total_weight FROM dbo.respondents`. (2) Filter rows before grouping: `WHERE final_weight > 0`. (3) Then `GROUP BY region ORDER BY region`.

Answer.

```
1 SELECT region, SUM(final_weight) AS total_weight
2 FROM dbo.respondents
3 WHERE final_weight > 0
4 GROUP BY region
5 ORDER BY region;
```

`WHERE` filters rows, `GROUP BY` makes one row per region, and `ORDER BY` sorts the result. The query only reads, which is all the endpoint allows on table data.

C0-L3 exercise 5

Hints. (1) Which statements only read? (2) Can the endpoint change lakehouse table data? (3) It is read-only for table data.

Answer. The bug is on line 2.

The SQL analytics endpoint is read-only for lakehouse table data, so the INSERT fails. Add the row from a Spark notebook, or keep data that needs T-SQL writes in a Warehouse.

Common wrong answers.

- Line 1: Reading a lakehouse table on the endpoint is exactly what it is for.
- Line 3: This SELECT would run. The line before it cannot.

C1-L1 Tenant, capacity, workspace and item

C1-L1 exercise 1

Hints. (1) What does a capacity provide? (2) Compute, measured in a unit. (3) Capacity units.

Answer. b) The capacity has 32 capacity units (CUs) of compute. Because: r1) For F SKUs the number equals the capacity units: F2 is 2 CUs, F64 is 64.

F SKUs are named by their capacity units. F32 provides 32 CUs.

Common wrong answers.

- `a`: Capacities are not sized by user count. The number is compute in CUs.
- `c`: OneLake storage is billed separately from capacity compute.
- `d`: The SKU does not cap workspaces in this way. It measures compute.
- `b|r2`: Right answer, wrong reason. The number is exactly the CU count.

C1-L1 exercise 2

Hints. (1) Which level holds organisation-wide settings? (2) Which level provides compute to workspaces? (3) Items live in workspaces.

Answer.

1. Tenant
2. Capacity
3. Workspace
4. Item, such as a lakehouse

Tenant settings apply to all capacities. Each workspace draws compute from its capacity. Items live in workspaces.

Common wrong answers.

- A workspace is assigned to a capacity, which provides its compute. Capacity sits above workspace.
- Capacities belong to a tenant, where tenant settings apply to everything.

C1-L1 exercise 3

Hints. (1) Which SKU lets free-licence users view Power BI content? (2) What happens when two workloads share one pool of CUs? (3) Size and isolation.

Answer.

- F64 or larger, because smaller F SKUs require each viewer to hold a Pro, PPU or trial licence.
- Production only, so experiments cannot use up the compute the report needs.

Licensing and isolation both point to a dedicated F64 or larger for this report.

Common wrong answers.

- sku = `f16`: Below F64, viewers need Pro, Premium Per User or trial licences. 200 free-licence users could not open the report.
- share = `sandbox`: A heavy notebook in the sandbox consumes the same CUs and can slow or throttle the report.

C1-L1 exercise 4

Hints. (1) Build a mapping from each valid SKU name to its CUs. (2) Check membership in the mapping, then the hours. (3) Multiply and return a float.

Answer.

```
F_SKUS: dict[str, int] = {f"F{n}": n for n in (2, 4, 8, 16, 32, 64, 128,
    256, 512, 1024, 2048)}
```

```python


def cu_hours(sku: str, hours: float) -> float:
    """Capacity-unit hours for running an F SKU capacity.

    Args:
        sku: F SKU name such as "F64".
        hours: Hours the capacity runs.

    Returns:
        CUs multiplied by hours.

    Raises:
        ValueError: For an unknown SKU or negative hours.
    """
    if sku not in F_SKUS:
        raise ValueError(f"unknown F SKU: {sku!r}")
    if hours < 0:
        raise ValueError("hours must not be negative")
    return float(F_SKUS[sku] * hours)
```

An explicit list of SKUs rejects typos such as F3 or f64 instead of pricing a capacity that does not exist.

C1-L1 exercise 5

Hints. (1) Was the afternoon's compute already used? (2) What happens to usage Fabric smoothed over the coming hours? (3) It is settled at pause.

Answer. b) Smoothed and overage usage still owed is added to the Azure bill at once, and items on the capacity stop working until it resumes. Because: r1) Fabric smooths usage over time. Pausing settles what is still owed and makes the capacity's items unavailable.

Pausing saves money from then on, but the usage already smoothed forward is billed at once, and every item on the capacity stops until it resumes.

Common wrong answers.

- `a`: The work already happened. Pausing settles the remaining smoothed and overage usage on the bill.
- `c`: A paused capacity is not operational. Its items stop until you resume it.
- `d`: Pausing affects compute. It does not delete OneLake data.
- `b|r2`: Right answer, wrong reason. There is no double rate. Pausing settles usage already incurred.

C1-L2 OneLake, one lake per tenant

C1-L2 exercise 1

Hints. (1) Can you create a second OneLake? (2) What is a workspace in OneLake terms? (3) A folder.

Answer. b) One for the whole tenant Because: r1) Every tenant has a single OneLake. Workspaces organise it; they do not create new lakes.

One tenant, one OneLake. Workspaces and items are folders inside it.

Common wrong answers.

- `a`: You cannot create more OneLakes. Business units get workspaces and domains inside the one lake.
- `c`: Capacities provide compute. Storage lives in the tenant's single OneLake.
- `d`: Each workspace is a folder in OneLake, not a separate lake.
- `b|r2`: Right answer, wrong reason. It is a platform design, not a licensing unit.

C1-L2 exercise 2

Hints. (1) The workspace goes before the @. (2) The item name carries its type in lower case. (3) Tables for managed Delta tables.

Answer.

```
1 abfss://sales_prod@onelake.dfs.fabric.microsoft.com/lh_sales.lakehouse/
    Tables/orders
```

Workspace before `@`, then `item.itemtype`, then `Tables/<table>`.

Common wrong answers.

- g3 = `Files`: Files holds unmanaged files. Managed Delta tables live under Tables.
- g1 = `lh_sales`: The part before @ is the workspace, not the item.

C1-L2 exercise 3

Hints. (1) What character does the workspace name contain? (2) The abfs URI does not allow it. (3) Use the GUIDs, which never change.

Answer. The bug is on line 1.

Fixed version:

```
1 WORKSPACE_ID = CONFIG["workspace_id"]
2 LAKEHOUSE_ID = CONFIG["lakehouse_id"]
```

```
3 path = f"abfss://{WORKSPACE_ID}@onelake.dfs.fabric.microsoft.com/
    {LAKEHOUSE_ID}/Tables/orders"
```

The space in the workspace name breaks the abfs URI. Renaming the workspace would fix this job and break everything else that uses the old name. GUIDs fix both.

Common wrong answers.

- Line 3: The format of line 3 is right for names. The value it receives on line 1 is the problem.

C1-L2 exercise 4

Hints. (1) Check the item type against a fixed set. (2) Reject spaces in workspace and item names. (3) relative.lstrip('/') avoids a double slash.

Answer.

```
1  ONELAKE_HOST = "onelake.dfs.fabric.microsoft.com"
2  ITEM_TYPES = frozenset({"lakehouse", "warehouse"})
3
4
5  def onelake_path(workspace: str, item: str, item_type: str, relative:
     str) -> str:
6      """abfss path to data in a OneLake item, addressed by name.
7
8      Args:
9          workspace: Workspace name, without spaces.
10         item: Item name, without spaces.
11         item_type: "lakehouse" or "warehouse".
12         relative: Path inside the item, such as Tables/orders.
13
14     Returns:
15         The abfss URI.
16
17     Raises:
18         ValueError: For an unknown item type or names with spaces.
19     """
20     if item_type not in ITEM_TYPES:
21         raise ValueError(f"unsupported item type: {item_type!r}")
22     if " " in workspace or " " in item:
23         raise ValueError("abfs URIs do not allow spaces; use GUIDs
       instead")
24     return f"abfss://{workspace}@{ONELAKE_HOST}/{item}.{item_type}/
       {relative.lstrip('/')}"
```

Validating inputs turns a confusing URI error at read time into a clear message at build time.

C1-L2 exercise 5

Hints. (1) What changes and what does not? (2) Which endpoint guarantees the region? (3) Does Databricks need a copy to read Delta?

Answer.

- GUIDs survive renames and avoid URI problems with special characters.
- The regional endpoint keeps requests in the workspace's region, which the residency rule needs.
- OneLake speaks ADLS APIs and stores open Delta tables, so the job reads them without a second copy.

Stable identifiers, a regional endpoint, and no copies.

Common wrong answers.

- address = `names`: Names change, and names with spaces break abfs URIs.
- endpoint = `global`: Microsoft Learn warns that the global endpoint can route data out of your region.
- copy = `copy_nightly`: A nightly copy is a second dataset to govern and keep in sync. Read the Delta tables where they are.

C1-L3 Workspace roles, granted to groups

C1-L3 exercise 1

Hints. (1) Which rights sit only at the top of the role table? (2) Workspace-level settings. (3) Admin.

Answer. a) Admin only Because: r1) Connecting Git, like deleting the workspace or creating a workspace identity, is reserved for Admins.

Only Admins connect a workspace to Git, create a workspace identity, and update or delete the workspace.

Common wrong answers.

- `b`: Members can add people and reshare, but connecting Git is an Admin right.
- `c`: Contributors create and change items, but cannot connect the workspace to Git.
- `d`: Viewers cannot change anything in the workspace.
- `a|r2`: Right answer, wrong reason. It is a permission boundary, not a licence.

C1-L3 exercise 2

Hints. (1) Which two read rights does the role table list? (2) Which one does Viewer have? (3) ReadData, the T-SQL one.

Answer. b) It fails: Viewers read lakehouse data with T-SQL, not through Spark or OneLake APIs. Because: r1) Viewer includes ReadData (T-SQL) but not ReadAll (Spark and OneLake APIs).

Point Viewers to the SQL analytics endpoint, or grant ReadAll on the item if Spark access is really needed.

Common wrong answers.

- `a`: Viewer grants T-SQL reads only. Spark reads need ReadAll, which starts at Contributor.
- `c`: There is no row limit tied to the role. The read is refused.
- `d`: Viewers can open and read notebooks. They cannot run them or read data through Spark.
- `b|r2`: Right answer, wrong reason. Spark is available. The Viewer role lacks ReadAll.

C1-L3 exercise 3

Hints. (1) Who manages, who writes, who reads? (2) Least privilege. (3) Groups, not individuals.

Answer.

- Platform team: Admin, as a group, so management does not depend on one person.
- Deployment identity: Contributor, the only writer in production.
- Analysts: Viewer. They query with T-SQL and use reports.

One managing group, one automated writer, and readers as Viewers.

Common wrong answers.

- analysts = `contributor`: Contributors can change and delete every item in production. Analysts need to read.
- deploy = `admin`: Deployment needs to write items, not manage people or delete the workspace. Contributor is enough.
- platform = `viewer`: Someone must be able to manage the workspace. Give that right to a group.

C1-L3 exercise 4

Hints. (1) A dictionary from action to role. (2) Spark and OneLake reads (ReadAll) start at Contributor. (3) Indexing a dict raises KeyError for unknown keys.

Answer.

```python
MIN_ROLE: dict[str, str] = {
    "connect_git": "Admin",
    "delete_workspace": "Admin",
    "add_members": "Member",
    "run_notebook": "Contributor",
    "spark_read": "Contributor",
    "tsql_read": "Viewer",
}


def min_role(action: str) -> str:
    """Lowest Fabric workspace role that allows an action.

    Raises:
        KeyError: For an action not in the table.
    """
    return MIN_ROLE[action]
```

Encoding the role table as data lets access requests be checked automatically, and an unknown action fails loudly instead of defaulting to a guess.

C1-L3 exercise 5

Hints. (1) Which line grants write access? (2) Do analysts need to change production items? (3) Also replace the individual with a group.

Answer. The bug is on line 4.

Fixed version:

```yaml
workspace: sales_prod
roles:
  Admin: [grp-fabric-platform-admins]
  Contributor: [sp-sales-deployment]
  Viewer: [grp-sales-analysts, grp-sales-readers]
```

Analysts as Contributors can change or delete production items. Make them Viewers, keep automation as the only writer, and replace the individual with a group.

Common wrong answers.

- Line 5: An individual assignment breaks the groups rule, but it only grants read access. Line 4 gives write access to production.

- Line 3: An admin group is the right pattern.

C2-L1 Lakehouse Tables and Files, and schemas

C2-L1 exercise 1

Hints. (1) Which format does the endpoint read? (2) Which folder holds managed tables? (3) Delta, in Tables.

Answer. b) Nothing. Only Delta tables in Tables appear in the endpoint. Because: r1) The endpoint exposes Delta tables only. Parquet and CSV cannot be queried through it.

Only Delta tables reach the SQL engines. Files is for raw and non-Delta data.

Common wrong answers.

- `a`: Files is not scanned for tables. Write a Delta table to Tables with saveAsTable.
- `c`: Refreshing metadata picks up Delta tables. It never turns a Parquet file into a table.
- `d`: Fabric does not convert it. You write the Delta table yourself.
- `b|r2`: Right answer, wrong reason. Files is visible. It is just not tables.

C2-L1 exercise 2

Hints. (1) What fills in the missing parts of the name? (2) The default lakehouse and the default schema. (3) dbo.

Answer. a) lh_survey.dbo.respondents Because: r1) A bare table name resolves to the dbo schema of the default lakehouse.

`respondents` resolves to `lh_survey.dbo.respondents`. If the table lives in `sales`, name the schema.

Common wrong answers.

- `b`: Spark does not search schemas. Missing parts come from defaults: default lakehouse, dbo schema.
- `c`: Omitted parts are allowed and filled from defaults.
- `d`: Nothing is unioned. One table is resolved.
- `a|r2`: Right answer, wrong reason. Spark reads any schema you name. dbo is only the default.

C2-L1 exercise 3

Hints. (1) The four-part name is workspace.lakehouse.schema.table. (2) The third part is a schema. (3) The employees table lives in schema hrm in Microsoft's example, or dbo by default.

Answer.

```
SELECT e.name, d.dept_name
FROM ops_ws.hr_lakehouse.hrm.employees AS e
JOIN global_ws.corporate_lakehouse.company.departments AS d
  ON e.deptno = d.deptno
```

`workspace.lakehouse.schema.table` reaches any table you have permission to read, across lakehouses and workspaces.

Common wrong answers.

- g1 = `Tables`: Tables is a OneLake folder, not a schema. The third part is the schema name, such as dbo or hrm.

C2-L1 exercise 4

Hints. (1) Read with .schema(RESPONDENT_SCHEMA) and the header option. (2) Write with .write.mode("append").saveAsTable(table). (3) Return the DataFrame.

Answer.

```
from pyspark.sql import DataFrame, SparkSession
from pyspark.sql import types as T

RESPONDENT_SCHEMA = T.StructType([
    T.StructField("respondent_id", T.StringType(), False),
    T.StructField("wave", T.IntegerType(), False),
    T.StructField("weight", T.DoubleType(), True),
])


def load_delivery(spark: SparkSession, path: str, table: str) -> DataFrame:
    """Read a CSV delivery and append it to a managed Delta table.

    Args:
        spark: Active session.
        path: CSV path, usually under Files/landing.
        table: Target table, ideally lakehouse.schema.table.

    Returns:
        The DataFrame that was read.
    """
    df = spark.read.option("header", True).schema(RESPONDENT_SCHEMA).csv(path)
```

```
    df.write.mode("append").saveAsTable(table)
    return df
```

A managed Delta table in Tables serves Spark, the SQL analytics endpoint and Direct Lake from one copy.

C2-L1 exercise 5

Hints. (1) Which area holds non-Delta files? (2) Which area do SQL and Power BI read? (3) Schemas organise tables by domain.

Answer.

- Raw deliveries in Files, as received.
- Cleaned and reporting data as Delta tables in Tables, under schemas such as survey_silver and survey_gold.

Files for raw, Tables for Delta, and schemas to keep the tables organised.

Common wrong answers.

- raw = `tables_dbo`: CSV files under Tables are not Delta tables. They end up in the Unidentified area.
- gold = `files`: Power BI's Direct Lake and the SQL endpoint read Delta tables in Tables, not files.
- clean = `tables_dbo`: It works, but everything lands in one flat schema. Named schemas keep domains apart as the lakehouse grows.

C2-L2 Warehouse T-SQL, and where it differs from SQL Server

C2-L2 exercise 1

Hints. (1) What does NOT ENFORCED mean? (2) Does the engine check the key on insert? (3) No.

Answer. b) Both rows are stored. The key is not enforced. Because: r1) NOT ENFORCED keys are optimiser hints. The engine does not check them.

Both rows land. Check uniqueness after each load with a GROUP BY ... HAVING COUNT(*) > 1 query.

Common wrong answers.

- `a`: That is SQL Server behaviour for enforced keys. Fabric Warehouse keys are NOT ENFORCED.
- `c`: INSERT never replaces rows. You would need MERGE for upsert behaviour.

- `d`: Nothing locks. Both rows are simply stored.
- `b|r2`: Right answer, wrong reason. No background job removes duplicates. You must check.

C2-L2 exercise 2

Hints. (1) Not clustered. (2) Not checked by the engine. (3) `NONCLUSTERED` and `NOT ENFORCED`.

Answer.

```
1 ALTER TABLE dbo.dim_customer
2     ADD CONSTRAINT pk_dim_customer PRIMARY KEY NONCLUSTERED (customer_id)
    NOT ENFORCED;
```

`PRIMARY KEY NONCLUSTERED (...) NOT ENFORCED` is the only accepted form.

Common wrong answers.

- g1 = `CLUSTERED`: Fabric Warehouse supports primary keys only as NONCLUSTERED.

C2-L2 exercise 3

Hints. (1) Two SQL Server types have no Fabric Warehouse equivalent. (2) One for money, one for Unicode text. (3) decimal and varchar replace them.

Answer. The bug is on line 4, 5.

Fixed version:

```
1 CREATE TABLE dbo.fact_payment (
2     payment_id BIGINT NOT NULL,
3     customer_id INT NOT NULL,
4     amount DECIMAL(19, 4) NOT NULL,
5     note VARCHAR(500),
6     paid_at DATETIME2(6) NOT NULL
7 );
```

`money` and `nvarchar` are not supported. Use `DECIMAL(19, 4)` and `VARCHAR(500)`. Varchar uses a UTF-8 collation, so it stores Unicode text.

Common wrong answers.

- Line 6: datetime2 with up to 6 fractional digits is supported.
- Line 2: BIGINT is supported.

C2-L2 exercise 4

Hints. (1) Count rows per customer_id. (2) Filter groups, not rows. (3) HAVING COUNT(*) > 1.

Answer.

```sql
SELECT customer_id, COUNT(*) AS n
FROM dbo.dim_customer
GROUP BY customer_id
HAVING COUNT(*) > 1;
```

With keys unenforced, this query is the guard. Any row it returns means a duplicate got in.

C2-L2 exercise 5

Hints. (1) What no longer enforces uniqueness? (2) Which feature is not supported at all? (3) Use the documented alternatives.

Answer.

- Keys become hints, so add a duplicate check that fails the load.
- Triggers are not supported. Do the audit write explicitly in the loading procedure.
- Map each type to its documented alternative, keeping precision and scale.

A port succeeds when every lost guarantee is replaced by an explicit step.

Common wrong answers.

- pk = `trust`: Sources resend files and pipelines rerun. Without a check, duplicates load silently.
- trigger = `keep`: Fabric Warehouse does not support triggers. The CREATE TRIGGER statement fails.
- types = `string`: Strings lose numeric semantics and cost storage. Use decimal for amounts.

C2-L3 Lakehouse, Warehouse or SQL analytics endpoint

C2-L3 exercise 1

Hints. (1) The endpoint is described as read-only. Read-only for what? (2) It blocks changes to table data, not every kind of object. (3) Views, functions and stored procedures that read are allowed.

Answer. b) You can create a view over lakehouse tables in it. Because: r1) Views are T-SQL objects that read data. The endpoint allows objects that read, not statements that change table data.

The endpoint is read-only for table data, but you can create views, functions and stored procedures in it. Fabric creates it with the lakehouse, and it reads the same Delta tables in OneLake.

Common wrong answers.

- `a`: The endpoint is read-only for table data. Write with Spark in the lakehouse, or move the table to a warehouse.
- `c`: Fabric creates the endpoint automatically with every lakehouse.
- `d`: It reads the same Delta tables in OneLake. There is no second copy. It syncs metadata, not data.
- `b|r2`: Right choice, wrong reason. A view is a T-SQL object in the endpoint. The Delta tables stay unchanged.

C2-L3 exercise 2

Hints. (1) Ask who writes the data, and in which language. (2) Spark writers and files: one item. T-SQL writers: another. (3) T-SQL readers of lakehouse data need no new item at all.

Answer.

- Raw files and PySpark: **Lakehouse**. Spark writes, and files sit next to tables.
- T-SQL writes across several tables: **Warehouse**, for DML and multi-table transactions.
- T-SQL reads of lakehouse data, plus views: **SQL analytics endpoint**. No copy needed.

Spark writers use a lakehouse. T-SQL writers use a warehouse. T-SQL readers of lakehouse data use the endpoint that comes with the lakehouse.

Common wrong answers.

- finance = `lakehouse`: The finance team writes with T-SQL. A lakehouse accepts writes from Spark, and its endpoint is read-only. Use a warehouse.
- finance = `endpoint`: The endpoint cannot change table data, so the stored procedures would fail on their first UPDATE or INSERT.
- raw = `warehouse`: A warehouse does not hold raw files, and the team writes with PySpark. Use a lakehouse.
- adhoc = `warehouse`: Copying the lakehouse tables into a warehouse makes a second copy to keep in sync. The endpoint reads them where they are.

C2-L3 exercise 3

Hints. (1) Which statements change table data? (2) The endpoint accepts reads and objects that read. (3) UPDATE on a lakehouse table fails through the endpoint.

Answer. The bug is on line 7.

Fixed version:

```
-- Connected to the SQL analytics endpoint of lh_research
CREATE VIEW dbo.v_wave_reach AS
SELECT wave_id, SUM(weighted_reach) / SUM(weighted_base) AS reach_pct
FROM dbo.nat_metrics
GROUP BY wave_id;
GO
-- Fix wave W00 in the lakehouse with Spark, not here.
```

Line 7 is an `UPDATE` on a lakehouse table, which the read-only endpoint rejects. Make that correction in a Spark notebook against the lakehouse, and keep only the view here.

Common wrong answers.

- Line 2: Creating a view is allowed on the endpoint. Look for a statement that changes table data.
- Line 3: The division could hit zero, but that is not what stops this script. Look for a write.

C2-L3 exercise 4

Hints. (1) Aggregate both sums per wave, then divide. (2) `NULLIF(x, 0)` returns NULL when x is 0, and dividing by NULL gives NULL. (3) `SUM(weighted_reach) / NULLIF(SUM(weighted_base), 0) AS reach_pct`, then `GROUP BY wave_id`.

Answer.

```
CREATE VIEW dbo.v_wave_reach AS
SELECT
    wave_id,
    SUM(weighted_reach) / NULLIF(SUM(weighted_base), 0) AS reach_pct
FROM dbo.nat_metrics
GROUP BY wave_id;
```

Sum both columns per wave and divide by `NULLIF(SUM(weighted_base), 0)`. A zero base becomes NULL, so the division returns NULL instead of an error.

C2-L3 exercise 5

Hints. (1) Three-part names are item.schema.table. (2) The first part names the item that holds the table. (3) The table sits in the lakehouse `lh_research`.

Answer.

```
INSERT INTO dbo.wave_summary (wave_id, weighted_base)
SELECT wave_id, SUM(weighted_base)
FROM lh_research.dbo.nat_metrics
GROUP BY wave_id;
```

A warehouse query reads a lakehouse table in the same workspace with `lh_research.dbo.nat_metrics`. The data stays in one place.

Common wrong answers.

- g1 = `wh_finance`: `wh_finance` is the warehouse you are in. The table lives in the lakehouse, so name the lakehouse.
- g1 = `OneLake`: There is no database called OneLake. The first part of a three-part name is the item name: the lakehouse.

C2-L3 exercise 6

Hints. (1) The query failed first and worked later, with no change in between. (2) How does the endpoint learn that a new table exists? (3) Look at the metadata sync between the lakehouse and its endpoint.

Answer. b) The endpoint had not yet synced the new table's metadata. Because: r1) The endpoint discovers new Delta tables through an automatic sync that usually takes under a minute but has no fixed limit.

The endpoint syncs lakehouse metadata automatically, usually in under a minute, but the lag varies. When a later step depends on a new table, add a metadata refresh step (portal, REST API) or a retry before the query. Check the current refresh options on Microsoft Learn.

Common wrong answers.

- `a`: The query worked an hour later without any change to the table, so the format was fine.
- `c`: The sync runs automatically. A manual refresh speeds it up, but it is not required every time.
- `d`: Permissions do not change on a timer. The delay points to the metadata sync.
- `b|r2`: Right choice, wrong reason. The endpoint does not copy data. It syncs metadata about the Delta tables.

C3-L1 OneLake shortcuts, and the delete that goes through

C3-L1 exercise 1

Hints. (1) Is the shortcut the data or a pointer? (2) What does removing a pointer do? (3) Contrast it with deleting inside the pointer.

Answer. b) It stays. Deleting a shortcut deletes only the shortcut object. Because: r1) Shortcuts do not cascade deletes. The shortcut is an independent object.

The shortcut goes, the data stays. Deleting a file or folder inside the shortcut is different.

Common wrong answers.

- `a`: Removing the shortcut removes only the link. Deleting files inside it is what reaches the target.
- `c`: There is no copy to recycle. The data was never in OneLake.
- `d`: Nothing is copied when you delete a shortcut.
- `b|r2`: Right answer, wrong reason. Deleting files inside a shortcut can delete ADLS data.

C3-L1 exercise 2

Hints. (1) Where do the files inside a shortcut live? (2) In the target. (3) Your permissions there decide.

Answer. b) The 2023 folder is deleted in the target storage account. Because: r1) Deleting a file or folder within a shortcut deletes it in the target when you have permission there.

Inside a shortcut you are working on the target. Deletes go through.

Common wrong answers.

- `a`: There is no hidden view. The folder is the target folder.
- `c`: Shortcuts are not read-only when your identity can write to the target.
- `d`: The shortcut still works. It just points at data that no longer includes 2023.
- `b|r2`: Right answer, wrong reason. The delete happens immediately, not on a schedule.

C3-L1 exercise 3

Hints. (1) Which area holds managed tables? (2) Which format does the endpoint read? (3) Tables and Delta.

Answer.

```
1 Create the shortcut under Tables, pointing at data in Delta format.
```

A shortcut under Tables that points at Delta data is recognised as a table.

Common wrong answers.

- g1 = `Files`: Shortcuts under Files can hold any format, but they do not become tables.

C3-L1 exercise 4

Hints. (1) Check shortcuts first and refuse early. (2) Compare against root + "/" so partner does not match partner_old. (3) Allowlist owned prefixes with the trailing slash.

Answer.

```python
OWNED_PREFIXES = ("Files/landing/", "Files/tmp/")


def safe_to_delete(path: str, shortcut_roots: set[str]) -> bool:
    """True only for owned paths that are not under a shortcut.

    Args:
        path: Lakehouse-relative path, such as Files/landing/w14.csv.
        shortcut_roots: Paths of every shortcut in the lakehouse.

    Returns:
        Whether a clean-up job may delete the path.
    """
    for root in shortcut_roots:
        base = root.rstrip("/")
        if path == base or path.startswith(base + "/"):
            return False
    return path.startswith(OWNED_PREFIXES)
```

Refusing anything under a shortcut, and allowing only named folders, makes a clean-up job unable to reach someone else's data.

C3-L1 exercise 5

Hints. (1) Copy or point? (2) Where do shortcuts become tables? (3) What stops a delete from going through?

Answer.

- A shortcut avoids a second copy and a pipeline to maintain.
- Under Tables, Delta data becomes a table for SQL and Direct Lake.
- Caching cuts egress costs for repeated reads from S3.
- A read-only identity means no delete inside the shortcut can reach the bucket.

Point, place it where it becomes a table, cache cross-cloud reads, and connect read-only.

Common wrong answers.

- access = `copy`: A copy doubles storage and needs a pipeline and monitoring. A shortcut reads in place.
- writes = `owner`: With write access, deleting inside the shortcut deletes marketing's data.
- location = `files`: Under Files it will not be recognised as a table.

C3-L2 Mirroring operational databases into OneLake

C3-L2 exercise 1

Hints. (1) What do shortcuts point at? (2) Is an Azure SQL database a folder of Delta files? (3) It must be replicated.

Answer. b) Mirroring: it replicates the database into OneLake continuously. Because: r1) Azure SQL stores data in a proprietary format, so Fabric must replicate it, which is what mirroring does.

Databases in proprietary formats come in through mirroring. Open-format data in storage comes in through shortcuts.

Common wrong answers.

- `a`: Shortcuts point at storage (OneLake, ADLS, S3 and similar), not at database engines.
- `c`: A nightly export is a hand-built copy with lag and maintenance. Mirroring does it continuously.
- `d`: There is no storage folder of Delta files to point at. The database must be replicated.
- `b|r2`: Right answer, wrong reason. Shortcuts copy nothing. Mirroring is needed because the source is a database.

C3-L2 exercise 2

Hints. (1) How many CUs does the capacity have? (2) 1 TB free per CU. (3) 45 - 32.

Answer.

```
1 32 13
```

32 TB are free on an F32. The remaining 13 TB of mirrored storage is billed as OneLake storage.

Common wrong answers.

- `64 0`: The example on Learn uses F64. This capacity has 32 CUs, so 32 TB are free.

C3-L2 exercise 3

Hints. (1) The standard catalog views schema. (2) Tables, not views. (3) `INFORMATION_SCHEMA` and `BASE TABLE`.

Answer.

```
SELECT TABLE_SCHEMA, TABLE_NAME
FROM INFORMATION_SCHEMA.TABLES
WHERE TABLE_TYPE = 'BASE TABLE';
```

`INFORMATION_SCHEMA.TABLES` lists the tables the endpoint exposes. Compare it with the source list.

Common wrong answers.

- g2 = `VIEW`: Views are not mirrored tables. Filter on BASE TABLE.

C3-L2 exercise 4

Hints. (1) Loop over the source tables. (2) mirror.get(table) is None when absent, so one comparison covers both cases. (3) Sort the result.

Answer.

```
def missing_from_mirror(source: dict[str, int], mirror: dict[str, int]) -
> list[str]:
  """Source tables that are absent from the mirror or differ in row
count.

  Args:
      source: schema.table to row count in the source database.
      mirror: schema.table to row count on the mirror's endpoint.

  Returns:
      Sorted table names that need attention.
  """
  return sorted(t for t, n in source.items() if mirror.get(t) != n)
```

One comparison, `mirror.get(t) != n`, catches both missing tables and partial replication.

C3-L2 exercise 5

Hints. (1) Is there a table limit? (2) Which key types block mirroring? (3) How would you notice a missing table?

Answer.

- Mirror what reporting needs, within the 1,000-table limit.
- Tables with datetime2(7) keys cannot be mirrored. Plan for them explicitly.
- Reconcile daily, because ineligible tables fail quietly.

Plan within the documented limits, then verify with data, not with status pages.

Common wrong answers.

- scope = `all`: Azure SQL Database mirroring supports at most 1,000 tables.
- keys = `ignore`: Those tables are not eligible. They will simply be missing.
- check = `trust`: A table that was never eligible does not show as failing. Compare lists and counts.

C3-L3 Mirroring Azure Databricks Unity Catalog into Fabric

C3-L3 exercise 1

Hints. (1) What format are Unity Catalog managed tables stored in? (2) Can Fabric read that format directly? (3) Metadata plus shortcuts.

Answer. b) Only the catalog structure. Data is read through shortcuts. Because: r1) Unity Catalog tables are already open Delta files, so Fabric mirrors metadata and reads the files in place.

Only the catalog structure is mirrored. The data stays in the Databricks storage account.

Common wrong answers.

- `a`: That is database mirroring for proprietary sources. Unity Catalog mirroring has no data movement.
- `c`: There are no snapshots. Fabric reads the current Delta files through shortcuts.
- `d`: Fabric engines read the Delta files directly. No Databricks compute runs the queries.
- `b|r2`: Right answer, wrong reason. It is about format: Delta can be read in place.

C3-L3 exercise 2

Hints. (1) Check the list of unsupported table types. (2) RLS/CLM policies. (3) The table is left out.

Answer. c) It is not mirrored: tables with RLS or CLM policies are unsupported. Because: r1) The limitations page lists tables with RLS/CLM policies as unsupported.

Filtered and masked tables are excluded. Decide what to publish for Fabric users instead.

Common wrong answers.

- `a`: Unity Catalog policies are not carried into Fabric, and such tables are not supported.
- `b`: Fabric does not quietly drop the filter. It does not mirror the table.
- `d`: Views are themselves unsupported for this mirroring.
- `c|r2`: Right answer, wrong reason. The table type is unsupported, not read-only.

C3-L3 exercise 3

Hints. (1) Scope before you connect. (2) The connection's rights cap what is readable. (3) Access, then verify.

Answer.

1. Choose tables safe for the whole Fabric audience, in a dedicated schema
2. Create a connection identity with SELECT on that schema only
3. Create the mirrored Azure Databricks item for that schema
4. Grant Fabric access with workspace roles, SQL permissions or model RLS
5. Compare table lists and row counts with Databricks

Scope, least-privilege connection, mirror, Fabric access, then a data check.

Common wrong answers.

- Mirroring first exposes whatever the connection can read. Decide the scope first.
- Set up access before users arrive. Verification then confirms what they will see.

C3-L3 exercise 4

Hints. (1) Allow a fixed set of table types rather than excluding a list. (2) Also exclude tables with policies. (3) Sort the names.

Answer.

```python
MIRRORABLE_TYPES = frozenset({"MANAGED", "EXTERNAL"})


def mirror_eligible(tables: list[dict[str, object]]) -> list[str]:
    """Unity Catalog tables that Fabric can mirror.

    Args:
        tables: Dicts with name, table_type and has_row_filter_or_mask.

    Returns:
        Sorted names of eligible tables.
    """
    return sorted(
        str(t["name"])
        for t in tables
        if t["table_type"] in MIRRORABLE_TYPES and not
    t["has_row_filter_or_mask"]
     )
```

An allowlist of table types plus the policy check predicts which tables will appear, so a missing table is expected rather than a surprise.

C3-L3 exercise 5

Hints. (1) Where do Unity Catalog rules stop applying? (2) Whose credential reads the data? (3) Where does the RLS live?

Answer.

- The filtered table cannot be mirrored, so rebuild the regional rule where users consume the data.
- Scope the connection, because it reads every mirrored table for everyone.
- Keep managers on the report. Direct endpoint access would bypass the model's RLS.

Rebuild the rule where users are, scope the credential, and keep users on the governed path.

Common wrong answers.

- source = `unfiltered_all`: Every manager would see every region. The rule must be rebuilt in Fabric.
- connection = `broad`: The connection's credential serves all queries. Grant it only what the mirror needs.
- users = `viewer`: Viewers can query the endpoint with T-SQL and read rows the model's RLS would hide.

C4-L1 Choosing how data gets into Fabric

C4-L1 exercise 1

Hints. (1) Which layer is this? (2) Which tool tracks incremental state itself? (3) Copy job.

Answer. b) Copy job Because: r1) Copy job supports incremental copy and scheduling natively for bronze ingestion.

Bronze ingestion starts with Copy job: incremental copy without building a pipeline.

Common wrong answers.

- `a`: Mirroring replicates databases and catalogs. This is file ingestion into bronze.
- `c`: Dataflow Gen2 is for low-code transformation. Nothing needs transforming yet.
- `d`: It works, but you build and maintain the incremental state that Copy job tracks for you.
- `b|r2`: Right answer, wrong reason. Pipelines and shortcuts read ADLS too.

C4-L1 exercise 2

Hints. (1) Replicate, stream, transform or orchestrate? (2) Which tool is event-driven? (3) Which tool coordinates other tools?

Answer.

- Curated database tables: mirroring.
- Low-latency events: Eventstream.
- Heavy transformation: Spark.
- Ordering, branching and alerts across steps: a pipeline.

Each tool has a lane. Stay in it and there is less to build and maintain.

Common wrong answers.

- erp = `copyjob`: A Copy job works, but mirroring replicates the whole database with free replication compute and no schedule to manage.
- iot = `copyjob`: Copy jobs run on schedules. Seconds-level events need Eventstream.
- orchestrate = `spark`: A notebook can call others, but retries, branches and alerts across steps are what pipelines are for.

C4-L1 exercise 3

Hints. (1) Movement first. (2) Transformation next. (3) Orchestration wraps the rest.

Answer.

1. Replicate a whole database or catalog?
2. Copy tables or files into bronze?
3. Events that must arrive within seconds?
4. Transformation needed, and by whom?
5. Several steps that need ordering and alerts?

Choose movement tools, then transformation, then wrap them in orchestration only where needed.

Common wrong answers.

- Starting with a pipeline leads to building copy logic inside it. Choose the movement tools first, then orchestrate them.
- Decide how data arrives before how it is transformed.

C4-L1 exercise 4

Hints. (1) A dictionary from need to tool. (2) Check membership before returning. (3) Put the need in the error message.

Answer.

```python
TOOL_FOR_NEED: dict[str, str] = {
    "replicate_database": "Mirroring",
    "bronze_copy": "Copy job",
    "low_latency_events": "Eventstream",
    "code_transform": "Spark notebook",
    "lowcode_transform": "Dataflow Gen2",
    "orchestrate": "Pipeline",
}


def recommend(need: str) -> str:
    """Fabric tool for a data movement or processing need.

    Raises:
        ValueError: If the need is not in the table.
    """
    if need not in TOOL_FOR_NEED:
        raise ValueError(f"unknown need: {need}")
    return TOOL_FOR_NEED[need]
```

An explicit table makes the team's convention reviewable, and a clear error beats a silent default.

C4-L1 exercise 5

Hints. (1) Where does the bug live? (2) In hand-written state tracking. (3) Which tool does that for you?

Answer. b) Move the tables to Copy jobs with incremental copy, which track state themselves. Because: r1) Copy job manages the state of the last successful run, so there is no hand-written watermark logic to get wrong.

Removing hand-built incremental state removes the bugs it causes. Copy job tracks the last successful run itself.

Common wrong answers.

- `a`: Retries repeat the same logic, including its watermark bug.
- `c`: More runs repeat the off-by-one error more often.
- `d`: That moves the hand-built state into another tool without removing it.
- `b|r2`: Right answer, wrong reason. Pipelines can copy incrementally, but you build the state tracking.

C4-L2 Incremental copy with a watermark, or with CDC

C4-L2 exercise 1

Hints. (1) What does a watermark look for? (2) Does a deleted row have a new value? (3) CDC captures deletes.

Answer. b) Nothing. Watermark-based incremental copy cannot detect deletes. Because: r1) A watermark only finds rows with a larger value. A deleted row has no value to find.

Watermarks only see rows that exist with a larger value. Use CDC when deletes matter.

Common wrong answers.

- `a`: That needs CDC, which captures delete events.
- `c`: Soft deletes come from CDC with SCD Type 2. A watermark sees nothing.
- `d`: The job succeeds. The deleted row just stays in the destination.
- `b|r2`: Right answer, wrong reason. CDC-based incremental copy does replicate deletes.

C4-L2 exercise 2

Hints. (1) Which rows have rv above 105? (2) Where does row 3 fall? (3) Below the watermark.

Answer.

```
[4]
```

Only row 4 is copied. Row 3 is lost for good unless you reset to a full copy, which is why commit-order columns such as ROWVERSION, or CDC, are safer.

Common wrong answers.

- `[3, 4]`: Row 3 has rv 104, below the watermark 105, so it is never copied. That is the late-row blind spot.

C4-L2 exercise 3

Hints. (1) Which method captures deletes? (2) Which update method keeps one row per key? (3) `CDC` and `Merge`.

Answer.

```
Incremental method: CDC
Update method: Merge
```

CDC replicates inserts, updates and deletes. Merge keeps one row per key.

Common wrong answers.

- g2 = `Append`: Append adds each changed row again, so a key appears many times.
- g1 = `Watermark`: A watermark cannot see deletes.

C4-L2 exercise 4

Hints. (1) Handle last_wm None first. (2) Strictly greater than, to avoid re-copying the boundary. (3) Return ([], last_wm) when nothing is new.

Answer.

```python
Row = dict[str, int]


def next_batch(rows: list[Row], last_wm: int | None) -> tuple[list[Row], int | None]:
    """Rows above the watermark, and the new watermark.

    Args:
        rows: Source rows, each with an integer rv.
        last_wm: Largest rv copied so far, or None before the first run.

    Returns:
        The rows to copy and the watermark to store.
    """
    batch = [r for r in rows if last_wm is None or r["rv"] > last_wm]
    if not batch:
        return [], last_wm
```

```
    return batch, max(r["rv"] for r in batch)
```

Strictly greater avoids duplicates, and the empty-run guard keeps the watermark stable. The late-row gap remains, which is why CDC or ROWVERSION matters.

C4-L2 exercise 5

Hints. (1) Deletes matter here. (2) One row per key. (3) How do you recover missed changes?

Answer.

- CDC captures inserts, updates and deletes, with no watermark blind spots.
- Merge keeps one current row per order.
- Reset to a full copy for the table when you know changes were missed.

CDC plus Merge keeps silver in step with the source, and a per-table reset repairs known gaps.

Common wrong answers.

- method = `wm_modified`: A watermark misses deletes and late rows.
- update = `append`: Append adds each changed order again, breaking one row per key.
- recovery = `wait`: Changes below the stored state are not picked up by later runs.

C4-L3 Pipelines that fail fast, retry sensibly and alert

C4-L3 exercise 1

Hints. (1) It is much longer than most jobs. (2) Half a day. (3) 12 hours.

Answer. c) 12 hours Because: r1) The documented default timeout is 12 hours. Seven days is the maximum you can set.

12 hours by default. Set each activity's timeout close to its real run time, with some headroom.

Common wrong answers.

- `a`: 30 seconds is the default retry interval. The default timeout is 12 hours.
- `d`: Seven days is the maximum, not the default.
- `b`: The default is 12 hours.
- `c|r2`: Right answer, wrong reason. Timeouts are unrelated to capacity pausing.

C4-L3 exercise 2

Hints. (1) Four branches: success, failure, completion, skip. (2) Completion is not a failure. (3) Either outcome.

Answer. c) Whether the notebook succeeds or fails Because: r1) Upon Completion runs after success or failure. Upon Failure is the failure-only branch.

A failure alert on Upon Completion fires every night. Use Upon Failure.

Common wrong answers.

- `a`: That is Upon Failure.
- `b`: That is Upon Success.
- `d`: That is Upon Skip.
- `c|r2`: Right answer, wrong reason. It refers to this activity finishing, either way.

C4-L3 exercise 3

Hints. (1) Inside a string, expressions are wrapped in @{ }. (2) A whole value that must stay a number starts with @ alone. (3) `@{` and `@`.

Answer.

```
Folder path: landing/@{pipeline().parameters.wave}/
Max rows:    @pipeline().parameters.max_rows
```

`@{...}` inside text interpolates to a string. A bare `@...` expression keeps the value's type.

Common wrong answers.

- g2 = `@{`: String interpolation always returns a string. For a number, use the bare @ expression.

C4-L3 exercise 4

Hints. (1) A regular expression with fullmatch validates the shape. (2) Check each field's range. (3) Cap the total at 7 × 24 × 3600 seconds.

Answer.

```python
import re

TIMEOUT_PATTERN = re.compile(r"(\d+)\.(\d{2}):(\d{2}):(\d{2})")
MAX_SECONDS = 7 * 24 * 3600


def parse_timeout(value: str) -> int:
    """Seconds in a pipeline activity timeout written as D.HH:MM:SS.

```

```
    Raises:
        ValueError: On a bad format, out-of-range fields, or more than
    seven days.
    """
    match = TIMEOUT_PATTERN.fullmatch(value)
    if match is None:
        raise ValueError(f"timeout must be D.HH:MM:SS, got {value!r}")
    days, hours, minutes, seconds = (int(g) for g in match.groups())
    if hours > 23 or minutes > 59 or seconds > 59:
        raise ValueError(f"field out of range in {value!r}")
    total = days * 86400 + hours * 3600 + minutes * 60 + seconds
    if total > MAX_SECONDS:
        raise ValueError("timeout exceeds the seven-day maximum")
    return total
```

A linter that parses timeouts can flag every activity still on the 12-hour default, or set far above its normal run time.

C4-L3 exercise 5

Hints. (1) Which branch lets gold run after a failure? (2) Completion means either outcome. (3) Use Upon Success.

Answer. The bug is on line 3.

Fixed version:

```
Activity: nb_silver_orders
  Timeout: 0.00:30:00
Activity: nb_gold_sales  depends on nb_silver_orders: Upon Success
Activity: email_on_failure  depends on nb_silver_orders: Upon Failure
```

Upon Completion runs gold after a failed silver, publishing stale or partial data. Chain data steps on Upon Success.

Common wrong answers.

- Line 4: The alert is correctly on Upon Failure.
- Line 2: A 30-minute timeout is sensible for a 10-minute notebook.

C5-L1 From Databricks notebooks to Fabric notebooks

C5-L1 exercise 1

Hints. (1) Where do Fabric secrets live? (2) Which utility module handles credentials? (3) notebookutils.credentials.

Answer. b) `notebookutils.credentials.getSecret("https://kv-sales.vault.azure.net/", "sql-password")` Because: r1) Fabric reads secrets from Azure Key Vault through notebookutils.credentials.

`getSecret(vault_url, name)` reads from Azure Key Vault, with your identity's Get permission.

Common wrong answers.

- `a`: dbutils is Databricks-only. Fabric has no secret scopes.
- `c`: Workspace settings do not hold secrets as environment variables. Use Key Vault.
- `d`: Spark configuration is not a secret store.
- `b|r2`: Right answer, wrong reason. It reads from Key Vault at runtime.

C5-L1 exercise 2

Hints. (1) Look at run()'s signature. (2) timeout_seconds has a default. (3) 90.

Answer. b) It fails after 90 seconds, the default timeout. Because: r1) run() has timeout_seconds with a default of 90.

Pass `timeout_seconds` sized to the child's real run time with headroom.

Common wrong answers.

- `a`: Only if you pass a longer timeout_seconds.
- `c`: 12 hours is the pipeline activity default. notebook.run defaults to 90 seconds.
- `d`: run() waits for the child and returns its exit value.
- `b|r2`: Right answer, wrong reason. The limit is time, not job count.

C5-L1 exercise 3

Hints. (1) The Fabric utility module. (2) The name of run()'s timeout parameter. (3) `notebookutils` and `timeout_seconds`.

Answer.

```
files = notebookutils.fs.ls("Files/landing")
value = notebookutils.notebook.run("nb_child", timeout_seconds=600)
```

`notebookutils` replaces dbutils, and `timeout_seconds` sets the child's time limit.

Common wrong answers.

- g1 = `dbutils`: dbutils does not exist in Fabric.
- g1 = `mssparkutils`: mssparkutils still works but is being retired. Use notebookutils.

C5-L1 exercise 4

Hints. (1) Check for dbutils.secrets. first and raise. (2) Replace the specific prefixes, including the trailing dot. (3) Leave anything else alone.

Answer.

```python
REWRITES: tuple[tuple[str, str], ...] = (
    ("dbutils.fs.", "notebookutils.fs."),
    ("dbutils.notebook.", "notebookutils.notebook."),
    ("mssparkutils.", "notebookutils."),
)


def port_line(line: str) -> str:
    """Rewrite Databricks and legacy Fabric utility calls to notebookutils.

    Raises:
        ValueError: For secret lookups, which must move to Azure Key Vault by hand.
    """
    if "dbutils.secrets." in line:
        raise ValueError("secrets need a manual change: use notebookutils.credentials.getSecret with Azure Key Vault")
    for old, new in REWRITES:
        line = line.replace(old, new)
    return line
```

Mechanical rewrites are safe for fs and notebook calls. Secrets need a design change, so the tool stops and says so.

C5-L1 exercise 5

Hints. (1) What differs between an interactive session and a pipeline run? (2) Inline commands. (3) Move the library to an environment.

Answer. The bug is on line 1.

Fixed version:

```python
# pandera==0.33.1 is installed in the environment attached to this notebook.
import pandera.pandas as pa
df = spark.read.table("lh_sales.silver.orders").toPandas()
SCHEMA.validate(df, lazy=True)
```

Inline installs are off by default in pipeline runs. Put the library in an environment (Full mode for pipelines), which also pins the version reproducibly.

Common wrong answers.

- Line 2: The import fails only because the install on line 1 did not run.

- Line 3: Reading the table is fine.

C5-L2 Environments, resource profiles and V-Order

C5-L2 exercise 1

Hints. (1) Which resource profile do new workspaces use? (2) What does it do to V-Order? (3) Off, for faster writes.

Answer. b) No. New workspaces default to the writeHeavy profile, with V-Order off. Because: r1) spark.sql.parquet.vorder.default is false in new workspaces to favour write performance.

New workspaces favour writes. Turn V-Order on for tables that are read far more than written.

Common wrong answers.

- `a`: That was true for older workspaces. New ones default to writeHeavy, with V-Order off.
- `c`: Files holds non-table data. V-Order applies to Delta table writes.
- `d`: There is no size threshold. It depends on settings.
- `b|r2`: Right answer, wrong reason. V-Order still exists. You turn it on where reads matter.

C5-L2 exercise 2

Hints. (1) Which option is reproducible? (2) Which mode does Microsoft recommend for pipelines? (3) Full mode.

Answer. b) An environment in Full mode, attached to the notebooks or set as workspace default Because: r1) Full mode builds a reproducible snapshot, and Microsoft recommends it for pipelines and scheduled runs.

A Full mode environment pins the dependency snapshot for every scheduled run.

Common wrong answers.

- `a`: Inline installs are turned off by default in pipeline runs, and resolve dependencies differently from run to run.
- `c`: Quick mode installs at every session start, adding minutes, and suits development more than scheduled runs.
- `d`: That is an inline install by another route: session-scoped and hard to reproduce.

- `b|r2`: Right answer, wrong reason. Both modes install the same packages. Full mode is reproducible.

C5-L2 exercise 3

Hints. (1) The table-level property starts with delta. (2) delta.parquet.vorder.enabled. (3) Set it to true.

Answer.

```sql
ALTER TABLE lh.gold.sales_by_region
SET TBLPROPERTIES ("delta.parquet.vorder.enabled" = "true");
```

The table property applies V-Order to future writes to this table only.

Common wrong answers.

- g1 = `spark.sql.parquet.vorder.enable`: That old session setting is removed in Runtime 1.3 and later, and it is not a table property.

C5-L2 exercise 4

Hints. (1) The option belongs on df.write. (2) Its name is parquet.vorder.enabled. (3) Chain mode, option, saveAsTable.

Answer.

```python
from pyspark.sql import DataFrame


def write_gold(df: DataFrame, table: str) -> None:
    """Overwrite a read-heavy gold table with V-Order for this write only.

    Args:
        df: Gold rows.
        table: Target table, such as lh.gold.sales_by_region.
    """
    df.write.mode("overwrite").option("parquet.vorder.enabled",
    "true").saveAsTable(table)
```

A writer option scopes V-Order to one write, so ingestion elsewhere in the session keeps the fast default.

C5-L2 exercise 5

Hints. (1) Reproducible libraries. (2) Most writes are ingestion. (3) Where are reads heavy?

Answer.

- A Full mode default environment gives every scheduled notebook the same pinned libraries.
- writeHeavy suits ingestion, which is most of the Spark work.

- V-Order on gold tables speeds up the reads Power BI makes, without slowing ingestion.

Pin libraries once, optimise for writes by default, and pay for read layout only where reads happen.

Common wrong answers.

- libs = `inline`: Inline installs are off in pipeline runs and are not reproducible.
- session = `read_pbi`: That applies V-Order to every write, including bronze and silver, which pay the write cost for no reader.
- gold = `none_vorder`: Direct Lake reads gold constantly. V-Order is worth its write cost there.

C5-L3 Delta tables shared between Databricks and Fabric

C5-L3 exercise 1

Hints. (1) Which experiences list TIMESTAMP_NTZ as supported? (2) Spark does. (3) SQL and Direct Lake do not.

Answer. c) Fabric Spark Runtime 2.0 Because: r1) The interoperability matrix lists TIMESTAMP_NTZ as supported in Spark but not in the SQL endpoint or Direct Lake.

Use `TIMESTAMP` for columns that SQL or Power BI users need.

Common wrong answers.

- `a`: The SQL analytics endpoint does not support TIMESTAMP_NTZ.
- `b`: Direct Lake does not support TIMESTAMP_NTZ.
- `d`: Support differs by experience. Check the matrix.
- `c|r2`: Right answer, wrong reason. Every listed experience reads Delta. This feature is the issue.

C5-L3 exercise 2

Hints. (1) Which table feature does Runtime 2.0 add by default? (2) Deletion vectors. (3) Which reader lacks them?

Answer. b) A Python notebook (non-Spark) using the default Delta reader Because: r1) Runtime 2.0 writes deletion vectors by default, and Python notebooks do not support deletion vectors.

Runtime 2.0 writes reader 3, writer 7 with deletion vectors. Python notebooks do not support them.

Common wrong answers.

- `a`: The SQL analytics endpoint supports deletion vectors.
- `c`: Direct Lake supports deletion vectors.
- `d`: The writer's own runtime supports everything it writes.
- `b|r2`: Right answer, wrong reason. Python notebooks read OneLake. Deletion vectors are the problem.

C5-L3 exercise 3

Hints. (1) The Delta command that returns one row of table details. (2) DESCRIBE ... (3) `DESCRIBE`.

Answer.

```
DESCRIBE DETAIL gold.events;
```

`DESCRIBE DETAIL` returns minReaderVersion, minWriterVersion and the table features.

Common wrong answers.

- g1 = `SHOW`: SHOW TBLPROPERTIES lists properties. DESCRIBE DETAIL shows protocol versions and table features.

C5-L3 exercise 4

Hints. (1) Store each reader's supported set. (2) Set difference gives what is missing. (3) Index the dict so unknown readers raise KeyError.

Answer.

```
ALL_FEATURES = frozenset(
    {"deletion_vectors", "column_mapping_name", "column_mapping_id",
   "timestamp_ntz", "liquid_clustering"}
)
SUPPORTED: dict[str, frozenset[str]] = {
    "spark_2_0": ALL_FEATURES,
    "sql_endpoint": ALL_FEATURES - {"column_mapping_id", "timestamp_ntz"},
    "direct_lake": ALL_FEATURES - {"timestamp_ntz"},
    "python_notebook": frozenset({"timestamp_ntz"}),
}


def unsupported(reader: str, features: set[str]) -> list[str]:
    """Table features a Fabric reader does not support.

    Raises:
        KeyError: For an unknown reader.
```

```
    """
    return sorted(features - SUPPORTED[reader])
```

Encoding the matrix lets a release check refuse to publish a gold table with features its readers cannot handle.

C5-L3 exercise 5

Hints. (1) Which readers need this table? (2) Which features do they lack? (3) Check before release.

Answer.

- TIMESTAMP is supported by both readers. TIMESTAMP_NTZ is not.
- Column mapping in id mode is not supported by the SQL endpoint.
- A feature check before release catches changes a Databricks upgrade introduces.

Design for the readers that matter and check features on every release.

Common wrong answers.

- time = `ntz`: Neither the SQL endpoint nor Direct Lake supports TIMESTAMP_NTZ.
- mapping = `id_mode`: The SQL analytics endpoint does not support column mapping in id mode.
- gate = `trust`: Open format does not mean every reader supports every feature.

C6-L1 Star schemas for semantic models

C6-L1 exercise 1

Hints. (1) What does one row mean? (2) Which columns say which entities it concerns? (3) The dimension keys.

Answer. b) The dimension keys it holds, and their values Because: r1) Dimension key columns set the dimensionality, and their values set the granularity.

One row per combination of the dimension keys: that is the grain.

Common wrong answers.

- `a`: Measures are the numbers at that grain. They do not define it.
- `c`: Row count follows from the grain and the data. It does not define it.
- `d`: Import, DirectQuery or Direct Lake changes how data is read, not the grain.

- `b|r2`: Right answer, wrong reason. Grain comes from the data design.

C6-L1 exercise 2

Hints. (1) Which way should filters flow in a star? (2) Dimension to fact. (3) Bi-directional breaks that.

Answer. b) Minimise them. They can hurt performance and confuse report users. Because: r1) Bi-directional filters can slow queries and filter in ways users do not expect.

Single-direction one-to-many relationships keep filtering predictable.

Common wrong answers.

- `a`: The guidance is the opposite: minimise them.
- `c`: Date tables filter facts one way, like other dimensions.
- `d`: The concern is behaviour and performance, not support.
- `b|r2`: Right answer, wrong reason. They still exist. Use them sparingly and on purpose.

C6-L1 exercise 3

Hints. (1) What must be decided first? (2) Dimensions provide keys to facts. (3) Relationships before measures.

Answer.

1. State the fact table's grain
2. Build dimension tables with unique keys in gold
3. Build the fact table with dimension keys and measure columns
4. Create one-to-many, single-direction relationships
5. Write DAX measures

Grain, dimensions, facts, relationships, then measures.

Common wrong answers.

- Decide the grain before you build the fact table, or you will rebuild it.
- Measures depend on relationships to filter correctly.

C6-L1 exercise 4

Hints. (1) Loop over relationships and build a name. (2) Two independent checks, not elif. (3) Append in the order given.

Answer.

```
1 def model_warnings(relationships: list[dict[str, str]]) -> list[str]:
2     """Warnings for relationships that break star-schema guidance.
```

```
3
4       Args:
5           relationships: Dicts with from_table, to_table, cardinality and
        direction.
6
7       Returns:
8           Warning strings in input order.
9       """
10      warnings = []
11      for rel in relationships:
12          name = f"{rel['from_table']}-{rel['to_table']}"
13          if rel["cardinality"] == "many_to_many":
14              warnings.append(f"{name}: many-to-many")
15          if rel["direction"] == "both":
16              warnings.append(f"{name}: bi-directional")
17      return warnings
```

Independent checks report every problem. A review script like this runs on models exported with tools such as Tabular Editor.

C6-L1 exercise 5

Hints. (1) What does Direct Lake on SQL support? (2) What does one row mean? (3) Which way should filters flow?

Answer.

- Compute age_band upstream: Direct Lake on SQL endpoints does not support calculated columns.
- One row per respondent, brand and wave gives a clean fact grain.
- Single-direction relationships keep filters predictable.

Do the modelling work in gold, keep the grain clean, and filter one way.

Common wrong answers.

- where = `calc_col`: Calculated columns are not supported in Direct Lake on SQL analytics endpoints.
- grain = `resp`: A list column mixes grains and cannot be filtered by brand.
- rel = `both`: Bi-directional filtering is rarely needed and can inflate results.

C6-L2 DAX measures that state their filter context

C6-L2 exercise 1

Hints. (1) What was DIVIDE designed for? (2) What is returned without an alternate result? (3) BLANK.

Answer. b) BLANK Because: r1) DIVIDE returns BLANK when the denominator is zero or BLANK, unless an alternate result is given.

BLANK keeps empty cells empty in visuals.

Common wrong answers.

- `a`: DIVIDE was designed to handle division by zero without an error.
- `c`: Only if you pass 0 as the alternate result.
- `d`: That is what the / operator can return. DIVIDE returns BLANK.
- `b|r2`: Right answer, wrong reason. The / operator does divide by zero. DIVIDE handles it.

C6-L2 exercise 2

Hints. (1) Which table does ALL name? (2) Filters from dimensions reach the fact table. (3) Remove only the brand filter.

Answer. b) Every filter on fact_awareness, including wave and region
Because: r1) ALL on the fact table removes all its filters, including those that reach it through dimensions.

Use `ALL(dim_brand)` to compare against all brands within the same wave and region.

Common wrong answers.

- `a`: That is ALL(dim_brand). ALL(fact_awareness) removes much more.
- `c`: ALL changes filter context for every filter on the table, not just slicers.
- `d`: Filter context comes from the visual and the page. ALL removes the table's filters wherever they come from.
- `b|r2`: Right answer, wrong reason. ALL removes filters from the table or column you give it.

C6-L2 exercise 3

Hints. (1) The DAX keyword for a variable. (2) The safe division function. (3) `VAR` and `DIVIDE`.

Answer.

```
1 Weighted aware % =
2 VAR aware_w = SUMX ( fact_awareness, fact_awareness[aware] *
     fact_awareness[weight] )
3 VAR base_w = SUM ( fact_awareness[weight] )
4 RETURN DIVIDE ( aware_w, base_w )
```

Variables name each part. DIVIDE handles a zero base.

Common wrong answers.

- g1 = `LET`: DAX variables use VAR ... RETURN.

C6-L2 exercise 4

Hints. (1) Two variables, then RETURN. (2) ALL on the brand dimension only. (3) DIVIDE the two.

Answer.

```
Share of aware across brands =
VAR this_brand = [Weighted aware %]
VAR all_brands = CALCULATE ( [Weighted aware %], ALL ( dim_brand ) )
RETURN DIVIDE ( this_brand, all_brands )
```

Removing only the brand filter keeps the comparison inside the same wave and region, and DIVIDE handles an empty denominator.

C6-L2 exercise 5

Hints. (1) Does the value depend on the visual's filters? (2) Is it a property of one row? (3) Calculated columns are not an option here.

Answer.

- Age band is a property of each respondent: compute it in gold.
- Percentages depend on the visual's filters: measures.
- The base count also depends on filters: a measure, shown beside every percentage.

Row properties in gold, filter-dependent arithmetic in measures.

Common wrong answers.

- ageband = `calc_col`: Direct Lake on SQL analytics endpoints does not support calculated columns.
- share = `gold`: A stored percentage cannot re-aggregate correctly when users filter or group differently.
- base = `gold`: The base changes with every filter. It must be computed at query time.

C6-L3 Direct Lake, framing and the fallback you did not plan

C6-L3 exercise 1

Hints. (1) Does Direct Lake copy data? (2) What does it need to know about the tables? (3) Which version to read.

Answer. b) Frames the model: records which Delta table versions queries will use. Because: r1) Framing copies metadata only, takes seconds, and sets the point in time queries see.

Framing takes seconds and fixes the table versions queries see until the next frame.

Common wrong answers.

- `a`: Import copies data. Direct Lake refresh copies metadata only.
- `c`: The model reads Delta tables. It never writes them.
- `d`: Refresh does not change storage mode.
- `b|r2`: Right answer, wrong reason. They do refresh. The refresh is framing.

C6-L3 exercise 2

Hints. (1) Can a view be read as Delta files? (2) What does Automatic do when direct loading fails? (3) Fallback.

Answer. b) The query falls back to DirectQuery and runs more slowly. Because: r1) Views cannot be loaded directly from Delta, and Automatic falls back silently.

Automatic keeps the report working through DirectQuery, silently. Replace views with Delta tables.

Common wrong answers.

- `a`: That is DirectLakeOnly, or Direct Lake on OneLake, which never falls back.
- `c`: Fabric does not materialise views for you. Write a Delta table instead.
- `d`: A view has no Delta files to read.
- `b|r2`: Right answer, wrong reason. Fallback is slower.

C6-L3 exercise 3

Hints. (1) Which option errors when Direct Lake cannot be used? (2) Which is the default? (3) `DirectLakeOnly` and `Automatic`.

Answer.

```
1 Development model: Direct Lake behavior = DirectLakeOnly
2 Production model:  Direct Lake behavior = Automatic
```

DirectLakeOnly exposes fallback causes during development. Automatic protects users in production.

Common wrong answers.

- g1 = `DirectQueryOnly`: DirectQueryOnly always uses DirectQuery. It measures fallback cost but does not reveal where fallback happens.

C6-L3 exercise 4

Hints. (1) A dict from SKU to (files, rows). (2) Strictly greater than the limit is a problem. (3) Index the dict so unknown SKUs raise KeyError.

Answer.

```python
SMALL = (1000, 300)
GUARDRAILS: dict[str, tuple[int, float]] = {
    "F2": SMALL, "F4": SMALL, "F8": SMALL, "F16": SMALL, "F32": SMALL,
    "F64": (5000, 1500), "F128": (5000, 3000), "F256": (5000, 6000),
}


def guardrail_problems(sku: str, parquet_files: int, rows_millions:
   float) -> list[str]:
    """Direct Lake per-table guardrails a table exceeds on a SKU.

    Raises:
        KeyError: For a SKU not in the table.
    """
    max_files, max_rows = GUARDRAILS[sku]
    problems = []
    if parquet_files > max_files:
        problems.append("files")
    if rows_millions > max_rows:
        problems.append("rows")
    return problems
```

A nightly check against the guardrails catches small-file build-up before Direct Lake falls back or framing fails.

C6-L3 exercise 5

Hints. (1) How many files does each append add? (2) What merges small files? (3) OPTIMIZE after the write.

Answer. The bug is on line 3.

Fixed version:

```python
from pyspark.sql import DataFrame


def publish_hourly(df: DataFrame, table: str) -> None:
    """Append the latest hour, then compact small files for Direct
   Lake."""
    df.repartition(1).write.mode("append").saveAsTable(table)
    df.sparkSession.sql(f"OPTIMIZE {table}")
```

```
    return None
```

Hourly appends add a file each hour. OPTIMIZE compacts them, keeping the table under the guardrail. On busy tables, run it on a schedule rather than every hour.

Common wrong answers.

- Line 2: The docstring is fine.
- Line 4: Returning None is harmless.

C7-L1 Eventstream, events in, routed out

C7-L1 exercise 1

Hints. (1) Which format does a lakehouse use for tables? (2) Delta. (3) Eventhouse is a different destination.

Answer. b) Delta Lake tables Because: r1) The lakehouse destination converts real-time events to Delta Lake format in lakehouse tables.

Lakehouse destinations write Delta tables. Eventhouse destinations write KQL tables.

Common wrong answers.

- `a`: Events become Delta tables, which Spark, SQL and Direct Lake can read.
- `c`: Events become Delta tables.
- `d`: KQL tables live in an eventhouse, a different destination.
- `b|r2`: Right answer, wrong reason. Eventhouse destinations store data in KQL databases.

C7-L1 exercise 2

Hints. (1) Writing to an item needs which role? (2) Lesson C1-L3. (3) Contributor.

Answer. b) Contributor or higher Because: r1) Microsoft Learn requires Contributor or higher on the destination workspace for eventhouse, lakehouse and Activator destinations.

Contributor or higher on the destination workspace.

Common wrong answers.

- `a`: Viewers cannot write to items (lesson C1-L3).
- `c`: Admin is more than needed.
- `d`: Access to the destination workspace is required.

- `b|r2`: Right answer, wrong reason. Contributor applies to the workspace where the role is granted.

C7-L1 exercise 3

Hints. (1) Source first. (2) Protect the raw data before filtering. (3) Verify at the end.

Answer.

1. Connect the source
2. Route every event to a raw table
3. Create derived streams with filters for each consumer
4. Connect each derived stream to its destination
5. Compare event counts at source and destinations

Source, raw landing, derived streams, destinations, then a count check.

Common wrong answers.

- Store the raw stream first, so nothing is lost while you design filters.
- Counts can only be compared once destinations are connected and receiving events.

C7-L1 exercise 4

Hints. (1) Start with telemetry_raw for every event. (2) Check for a missing key before reading it. (3) Add fault destinations after the raw one.

Answer.

```python
def route(event: dict[str, object]) -> list[str]:
    """Destinations for one telemetry event. Every event is kept raw.

    Args:
        event: Parsed event.

    Returns:
        Destination names in routing order.
    """
    destinations = ["telemetry_raw"]
    if "event_type" not in event:
        destinations.append("quarantine")
    elif event["event_type"] == "fault":
        destinations += ["activator_faults", "bronze.vehicle_faults"]
    return destinations
```

Raw first, always. Malformed events are kept and flagged, not lost.

C7-L1 exercise 5

Hints. (1) Which source captures every change? (2) Which store answers queries in seconds? (3) Which store does Spark read?

Answer.

- The CDC source streams inserts, updates and deletes after an initial snapshot.
- An eventhouse serves low-latency KQL queries for dashboards.
- A lakehouse destination writes Delta tables for Spark.

One CDC source, two destinations, each suited to its consumer.

Common wrong answers.

- source = `poll`: Polling misses deletes and adds load. CDC streams every change.
- live = `lakehouse`: Lakehouse tables suit batch analysis. Live dashboards query an eventhouse with KQL.
- models = `custom`: Spark reads Delta tables in a lakehouse directly.

C7-L2 Eventhouse and KQL for time-series questions

C7-L2 exercise 1

Hints. (1) Does take sort? (2) Is its result consistent? (3) Use top for ordered results.

Answer. c) 10 arbitrary rows, possibly different on each run Because: r1) take guarantees nothing about which records it returns unless the data is sorted.

take is for browsing. top is for answers.

Common wrong answers.

- `a`: Use top 10 by event_time desc for the most recent rows.
- `b`: Ingestion order is not guaranteed either.
- `d`: take runs without error. It just promises nothing about which rows.
- `c|r2`: Right answer, wrong reason. There is no fixed seed. Results can change between runs.

C7-L2 exercise 2

Hints. (1) Which rows does summarize group? (2) Are there rows in those bins? (3) No.

Answer. b) No rows for those bins Because: r1) summarize groups existing rows. Bins with no rows do not appear. make-series fills them.

Charts draw a line straight across the gap. Use make-series with a default to show the outage.

Common wrong answers.

- `a`: That is make-series with default = 0. summarize only returns groups that exist.
- `c`: No rows are created at all.
- `d`: Empty bins are not an error.
- `b|r2`: Right answer, wrong reason. There are no rows to count, so no group is formed.

C7-L2 exercise 3

Hints. (1) A function for 'one hour before now'. (2) The operator that sorts and limits. (3) `ago` and `top`.

Answer.

```
1 telemetry_raw
2 | where event_time > ago(1h)
3 | top 10 by event_time desc
```

`ago(1h)` sets the window. `top 10 by event_time desc` returns the latest ten.

Common wrong answers.

- g2 = `take`: take does not accept a sort and returns arbitrary rows.

C7-L2 exercise 4

Hints. (1) Filter with where and ago first. (2) make-series fills missing steps with the default. (3) ... default = 0 on event_time step 1h by vehicle_id.

Answer.

```
1 telemetry_raw
2 | where event_time > ago(24h)
3 | make-series avg_rpm = avg(rpm) default = 0 on event_time step 1h by
    vehicle_id
```

make-series returns one array per vehicle with every hour present, so gaps show as zeros.

C7-L2 exercise 5

Hints. (1) Who needs the raw data, and for how long? (2) Which period is actually queried? (3) Aggregates are small.

Answer.

- Keep raw data as long as audits need, and no longer: 90 days.
- Cache the 30 days people query, to balance speed and cost.
- Keep small aggregates for years, where trends need them.

Retention follows need, cache follows query patterns, and aggregates outlive raw data.

Common wrong answers.

- raw_ret = `keep_default`: Ten years of raw telemetry costs storage nobody uses. Set retention from the need.
- raw_ret = `d7`: Audits need 90 days. Data past retention is removed automatically.
- raw_cache = `c90`: Caching data nobody queries raises cost without speeding up real queries.

C7-L3 Activator, rules that act on events

C7-L3 exercise 1

Hints. (1) Is this a change or a state condition? (2) How many times does the value cross 100 from below? (3) Once.

Answer. a) Once Because: r1) Increases above is a change condition. It fires when the value crosses the threshold from below.

Change conditions fire on transitions. State conditions fire on every matching event.

Common wrong answers.

- `b`: That is Is greater than, a state condition that fires on each matching event.
- `c`: The crossing from below is the trigger. No decrease is needed.
- `d`: Change conditions do not repeat on a timer.
- `a|r2`: Right answer, wrong reason. There is no daily limit of one. The rule fires once because there is one crossing.

C7-L3 exercise 2

Hints. (1) What arrives from a silent sensor? (2) Nothing. Which condition works on absence? (3) Heartbeat.

Answer. b) No presence of data Because: r1) No presence of data fires when no new events arrive within a set time. Other conditions need an event to evaluate.

Every value-based condition needs an event. Only the heartbeat condition sees silence.

Common wrong answers.

- `a`: No event arrives, so there is no value to compare.
- `c`: Becomes false needs a new event with the value false.
- `d`: Changes needs a new event with a different value.
- `b|r2`: Right answer, wrong reason. A silent sensor sends nothing at all.

C7-L3 exercise 3

Hints. (1) Which column identifies one freezer? (2) The occurrence option reads 'When it has been ... for'. (3) `freezer_id` and `true`.

Answer.

```
Object: freezer, keyed by freezer_id
Condition: temperature Is greater than -15
Occurrence: When it has been true for 10 minutes
```

Key by the object. The occurrence turns a state condition into one alert for a sustained problem.

Common wrong answers.

- g1 = `event_id`: Every event has its own ID, so no state carries across readings. Key by the freezer.

C7-L3 exercise 4

Hints. (1) Keep a dict from object_id to 'was above'. (2) Fire when the value is above and the object was not. (3) Update the object's state after every event.

Answer.

```python
def activations(events: list[tuple[str, float]], threshold: float) -> list[int]:
    """Indexes of events where an object's value crosses above a threshold.

    Args:
        events: (object_id, value) pairs in arrival order.
        threshold: Value that must be exceeded.

    Returns:
        Event indexes where the object moved from at-or-below to above.
```

```
    """
    above: dict[str, bool] = {}
    fired: list[int] = []
    for index, (object_id, value) in enumerate(events):
        is_above = value > threshold
        if is_above and not above.get(object_id, False):
            fired.append(index)
        above[object_id] = is_above
    return fired
```

Per-object state is what makes the rule mean "this oven crossed 80", not "some reading somewhere crossed 80".

C7-L3 exercise 5

Hints. (1) Which ID identifies one van? (2) How many readings arrive per minute? (3) Which condition works without events?

Answer.

- Key by van_id, so each van keeps its own state.
- A sustained state condition sends one message per incident, well under the 30 per rule per recipient per hour limit.
- A heartbeat rule detects silence, which no value-based condition sees.

Per-object keys, sustained conditions for noisy streams, and a heartbeat for silence.

Common wrong answers.

- object = `fleet`: One object mixes 400 vans. A cool reading from one van clears a hot one.
- hot = `every`: At one reading every 5 seconds, a hot van sends 12 messages a minute and hits the 30-per-hour limit in under 3 minutes.
- silent = `speed_zero`: A parked van reports speed 0. A silent van reports nothing.

C8-L1 Copilot in Fabric, where it runs and what it costs

C8-L1 exercise 1

Hints. (1) Copilot consumes CUs. Where do CUs come from? (2) Paid, not trial. (3) F2 or P1 and up.

Answer. b) A paid F2 or higher, or P1 or higher, in a supported region Because: r1) Microsoft Learn lists a paid F2+ or P1+ capacity in a supported region, with tenant settings enabled.

Paid F2+ or P1+, a supported region, and tenant settings switched on.

Common wrong answers.

- `a`: Copilot stops working when a trial expires.
- `c`: F64 matters for free-licence viewers (lesson C1-L1). Copilot starts at F2.
- `d`: Copilot consumes capacity units from the workspace's capacity.
- `b|r2`: Right answer, wrong reason. Copilot depends on the capacity and tenant settings, not a Pro licence.

C8-L1 exercise 2

Hints. (1) Is Copilot one feature or many? (2) Check the release-status page per workload. (3) Record the date: status changes.

Answer. b) Notebooks preview, Data Factory GA, with the date you checked Because: r1) Microsoft Learn's release-status page lists each Copilot experience separately.

Status is per experience. Record each one with the date you checked.

Common wrong answers.

- `a`: Copilot in notebooks is preview (Learn, September 2026).
- `c`: Copilot in Data Factory is GA (Learn, September 2026).
- `d`: Status differs by workload and changes over time. Record each one with a date.
- `b|r2`: Right answer, wrong reason. Status is per feature. Some Data Factory features are preview.

C8-L1 exercise 3

Hints. (1) Output and input rates differ. (2) Output is four times the input rate. (3) 400 and 400.

Answer.

```
input:  1,000 tokens x 100 / 1,000 = 100 CU seconds
output: 1,000 tokens x 400 / 1,000 = 400 CU seconds
total: 500 CU seconds
```

Output costs 400 CU seconds per 1,000 tokens. Long answers cost more than long questions.

Common wrong answers.

- g1 = `100`: Output tokens cost more than input: 400 CU seconds per 1,000.

C8-L1 exercise 4

Hints. (1) Three rates: 100, 10 and 400 per 1,000 tokens. (2) Uncached input = input_tokens - cached_input_tokens. (3) Validate before you compute.

Answer.

```python
INPUT_RATE = 100
CACHED_INPUT_RATE = 10
OUTPUT_RATE = 400
TOKENS_PER_UNIT = 1000


def copilot_cu_seconds(
    input_tokens: int, output_tokens: int, cached_input_tokens: int = 0
) -> float:
    """CU seconds for one Copilot request at the published token rates.

    Args:
        input_tokens: All input tokens, cached or not.
        output_tokens: Output tokens.
        cached_input_tokens: Input tokens served from the prompt cache.

    Returns:
        Capacity unit seconds consumed.

    Raises:
        ValueError: If a count is negative or cached exceeds input.
    """
    if min(input_tokens, output_tokens, cached_input_tokens) < 0:
        raise ValueError("token counts must not be negative")
    if cached_input_tokens > input_tokens:
        raise ValueError("cached tokens cannot exceed input tokens")
    uncached = input_tokens - cached_input_tokens
    total = (
        uncached * INPUT_RATE
        + cached_input_tokens * CACHED_INPUT_RATE
        + output_tokens * OUTPUT_RATE
    )
    return total / TOKENS_PER_UNIT
```

Charge each token once, at its own rate. Put the rates in named constants so you update one place when prices change.

C8-L1 exercise 5

Hints. (1) Tenant settings take security groups. (2) Is Copilot one feature? (3) Who owns production logic?

Answer.

- Scope the tenant setting to the security group that needs Copilot.

- Record each experience with its status and date, since status differs by workload.
- Treat Copilot output as a draft: a person reviews, tests and owns it.

Scoped access, a dated status record, and human ownership of every output.

Common wrong answers.

- scope = `everyone`: Scope tenant settings to the groups that need them. Every request draws on the F16.
- record = `copilot_yes`: Status is per experience. 'Yes' tells an auditor nothing.
- prod = `direct`: Running without error is not correctness. Lesson C8-L3 shows how to check.

C8-L2 Data agents, questions in plain language

C8-L2 exercise 1

Hints. (1) Which query types does the agent generate? (2) Only read queries. (3) Read-only holds for Admins too.

Answer. b) The agent generates only read queries, so it does not delete data Because: r1) Microsoft Learn states the data agent generates only read queries and keeps read-only connections.

Data agents are read-only by design.

Common wrong answers.

- `a`: The agent never generates create, update or delete queries.
- `c`: Read-only applies to every user, whatever their role.
- `d`: There is no approval flow. The agent does not write.
- `b|r2`: Right answer, wrong reason. The agent uses the user's permissions. It is read-only by design, whatever those permissions allow.

C8-L2 exercise 2

Hints. (1) Whose credentials does the agent use? (2) The person asking. (3) Ana cannot read EU sales.

Answer. b) Answers only from data Ana has permission to read Because: r1) The agent uses the requesting user's credentials and permissions.

The agent never widens access. Grant permissions on the data, not on the agent.

Common wrong answers.

- `a`: The agent runs as the person asking, not as its owner.
- `c`: The agent applies the user's permissions. It does not add its own masking.
- `d`: The agent queries data. It does not invent estimates for data it cannot read.
- `b|r2`: Right answer, wrong reason. There is no automatic country filter. Ana's own permissions decide what she reaches.

C8-L2 exercise 3

Hints. (1) What do examples get validated against? (2) Context before testing. (3) Test before publishing.

Answer.

1. Select only the tables and columns the questions need
2. Write instructions that define business terms
3. Add example question and SQL pairs
4. Run the ground-truth question set and record accuracy
5. Publish and share the agent

Schema, instructions, examples, evaluation, then publish.

Common wrong answers.

- Measure accuracy before users rely on the agent.
- Example queries are validated against the selected schema. Select the tables first.

C8-L2 exercise 4

Hints. (1) Use `answer is None` to detect a missing number. (2) `math.isclose` handles the tolerance. (3) Divide by `len(expected)`.

Answer.

```python
import math


def evaluate(
    expected: dict[str, float], actual: dict[str, float | None], rel_tol: float = 0.001
) -> tuple[float, list[str]]:
    """Accuracy of a data agent against a ground-truth set.

    Args:
        expected: Question id to ground-truth answer.
```

```
        actual: Question id to the agent's answer, None if it gave no
    number.
        rel_tol: Relative tolerance for a match.

    Returns:
        Share of expected questions answered correctly, and sorted failed
    ids.

    Raises:
        ValueError: If expected is empty.
    """
    if not expected:
        raise ValueError("ground-truth set is empty")
    failed = []
    for question, truth in expected.items():
        answer = actual.get(question)
        if answer is None or not math.isclose(answer, truth,
rel_tol=rel_tol):
            failed.append(question)
    return (len(expected) - len(failed)) / len(expected), sorted(failed)
```

The denominator is the ground-truth set. A zero is an answer. Both mistakes inflate or deflate accuracy without an error.

C8-L2 exercise 5

Hints. (1) Which file types are supported? (2) Which source types accept example queries? (3) Two sources, same numbers: how does the agent choose?

Answer.

- Data agents do not support PDF, Word or text files.
- Example queries work on lakehouse, warehouse and KQL sources, not semantic models.
- Two sources hold the same numbers, so tell the agent which to use for which question.

Supported sources only, examples where they work, and routing instructions where sources overlap.

Common wrong answers.

- pdfs = `include`: Unstructured files such as PDFs are not supported data agent sources.
- examples = `model`: Semantic model sources do not support example queries.
- routing = `no_instr`: With two sources of the same numbers, the agent needs an instruction to route questions.

C8-L3 Verify what Copilot writes

C8-L3 exercise 1

Hints. (1) What does the engine check before running? (2) Does it know what answer you expected? (3) Syntax and names only.

Answer. b) The syntax parses and the referenced names exist Because: r1) An engine checks syntax and names before it runs a query. It does not know what answer you wanted.

A clean run is the start of verification, not the end.

Common wrong answers.

- `a`: An average of averages runs cleanly and returns a wrong number.
- `c`: A query that scans a whole table runs without error too.
- `d`: The engine does not know the grain you intended.
- `b|r2`: Right answer, wrong reason. Microsoft says to review all generated code. The engine did the checking, not Copilot.

C8-L3 exercise 2

Hints. (1) Does every store weigh the same in the result? (2) Should it? (3) Total sales over total baskets.

Answer. The bug is on line 2.

Fixed version:

```
1 SELECT r.region,
2        SUM(s.total_sales) / NULLIF(SUM(s.basket_count), 0) AS avg_basket
3 FROM dbo.store_summary AS s
4 LEFT JOIN dbo.dim_region AS r ON s.store_id = r.store_id
5 GROUP BY r.region;
```

Recompute ratios from their totals at the level you report.

Common wrong answers.

- Line 4: A left join keeps stores without a region. That line is fine.
- Line 5: Grouping by region matches the requirement.

C8-L3 exercise 3

Hints. (1) Cheapest check first. (2) Which check gives you a known right answer? (3) Keep what you built.

Answer.

1. Read every line: names, grain, join types
2. Run it on a small dataset with a hand-worked answer

3. Compare the result with a second method on real data
4. Keep the known-data and comparison checks as tests

Read, known data, second method, then keep the checks.

Common wrong answers.

- Read first. A wrong join or invented column is cheaper to catch on paper.
- Known data first: it tells you which answer is right when the two methods disagree.

C8-L3 exercise 4

Hints. (1) Wrap only the candidate call in try/except. (2) Handle None before math.isclose, which rejects None. (3) `expected is not got` is True when one is None and the other is not.

Answer.

```python
import math
from collections.abc import Callable


def mismatches(
    candidate: Callable[[list[float]], float | None],
    reference: Callable[[list[float]], float | None],
    cases: list[list[float]],
) -> list[int]:
    """Indexes of cases where a candidate function disagrees with a reference.

    Args:
        candidate: Function under test, such as Copilot-generated code.
        reference: Trusted implementation.
        cases: Inputs to run through both.

    Returns:
        Case indexes where the candidate raised or returned a different result.
    """
    failed: list[int] = []
    for index, case in enumerate(cases):
        expected = reference(case)
        try:
            got = candidate(case)
        except Exception:
            failed.append(index)
            continue
        if expected is None or got is None:
            if expected is not got:
                failed.append(index)
        elif not math.isclose(got, expected):
```

```
            failed.append(index)
    return failed
```

A comparison harness turns "looks right" into a list of failing cases. Keep it with the pipeline tests.

C8-L3 exercise 5

Hints. (1) Which dataset has an answer you know? (2) Is the second method independent? (3) What checks the next change?

Answer.

- Hand-built rows cover the cases that break code quietly. A random sample rarely contains them.
- A second, independent method on the same rows catches logic errors.
- A committed test checks every later change, including the next Copilot edit.

Known awkward data, an independent second method, and a committed test.

Common wrong answers.

- data = `sample`: You have no hand-worked answer for a random sample, and it seldom contains the awkward cases.
- second = `rerun`: The same code gives the same wrong answer twice. The second method must be independent.
- keep = `note`: A note does not run. A test fails when a later change breaks the logic.

C9-L1 Domains and endorsement, finding the right data

C9-L1 exercise 1

Hints. (1) What are domains for? (2) Finding data. (3) Access stays with roles and permissions.

Answer. b) Nothing about access. The workspace's items show under Finance in the OneLake catalog Because: r1) Domain assignment does not affect item visibility or access. Workspace roles and item permissions do.

Domains organise discovery. Roles and permissions control access.

Common wrong answers.

- `a`: Domains do not restrict access.
- `c`: Domains do not grant access either.

- `d`: Access requests go through workspace roles and item permissions, not domains.
- `b|r2`: Right answer, wrong reason. Every item in the workspace inherits the domain, not only Power BI items.

C9-L1 exercise 2

Hints. (1) Promoted and Certified differ in who applies them. (2) Who names the certifiers? (3) A Fabric admin.

Answer. b) They can request certification. Only users a Fabric admin specifies can certify Because: r1) Any user can request certification, but only users specified by a Fabric administrator can certify.

Promotion is self-service. Certification is an authorised review.

Common wrong answers.

- `a`: Contributors can promote. Certifying needs a name on the Fabric admin's list.
- `c`: Certification is a human review. No test applies it.
- `d`: Any item except a Power BI dashboard can be certified.
- `b|r2`: Right answer, wrong reason. The workspace Admin role does not grant certification rights. The Fabric admin's list does.

C9-L1 exercise 3

Hints. (1) Tenant-level role first. (2) The domain role for workspace admins. (3) `admin` and `contributor`.

Answer.

```
1 Creates domains and names domain admins: Fabric admin
2 Assigns their own workspace to a domain: domain contributor, who must be a
    workspace admin
```

Fabric admins create domains. Domain contributors, who are workspace admins, assign their own workspaces.

Common wrong answers.

- g2 = `admin`: Domain admins associate workspaces too, but the role for workspace admins assigning their own workspaces is domain contributor.

C9-L1 exercise 4

Hints. (1) Handle the dashboard first. (2) Start from promoted and certified. (3) Add master_data only for data item types.

Answer.

```python
DATA_ITEM_TYPES = frozenset({"lakehouse", "warehouse", "semantic_model",
    "kql_database"})
NO_BADGE_TYPES = frozenset({"dashboard"})


def allowed_badges(item_type: str) -> set[str]:
    """Endorsement badges an item type can receive.

    Args:
        item_type: Fabric item type, such as "lakehouse" or "report".

    Returns:
        Allowed badge names.
    """
    if item_type in NO_BADGE_TYPES:
        return set()
    badges = {"promoted", "certified"}
    if item_type in DATA_ITEM_TYPES:
        badges.add("master_data")
    return badges
```

Encode the rules as constants and small branches, so a rule change touches one line.

C9-L1 exercise 5

Hints. (1) Which feature gives catalog filtering? (2) Which badge marks the single source of truth? (3) What controls access?

Answer.

- A domain gives filtering in the OneLake catalog and a home for delegated settings.
- One master data copy, with shortcuts for other uses (lesson C3-L1).
- Access comes from roles, never from domains.

Domains for discovery, master data for the authoritative copy, roles for access.

Common wrong answers.

- domain = `naming`: A prefix helps a little. A domain adds catalog filtering, delegated settings and named owners.
- customer = `promote_all`: Three promoted copies tell users nothing about which one to trust.
- restrict = `by_domain`: Domain assignment does not affect access.

C9-L2 OneLake security, roles down to rows and columns

C9-L2 exercise 1

Hints. (1) Within a role: intersection. Across roles? (2) Union. (3) Row filters combine with OR.

Answer. a) North and South rows Because: r1) Roles combine as a union. Row filters from different roles combine with OR.

Across roles, access adds up. Plan roles so their union is what you intend.

Common wrong answers.

- `b`: Across roles, filters combine with OR, not AND.
- `c`: Role order does not matter. Access is the union.
- `d`: Only rows allowed by at least one role.
- `a|r2`: Right answer, wrong reason. OneLake unions the roles. It does not pick one.

C9-L2 exercise 2

Hints. (1) Which users does OneLake security govern? (2) Viewers and Read-permission users. (3) Contributors have write access already.

Answer. b) All rows: Contributors already have data access through their workspace role Because: r1) OneLake security governs Viewers and users with Read permission. Admins, Members and Contributors already have data access.

Restrict readers by making them Viewers first.

Common wrong answers.

- `a`: Contributors are not restricted by OneLake security roles.
- `c`: The workspace role gives them full data access.
- `d`: The read succeeds with all rows.
- `b|r2`: Right answer, wrong reason. Spark enforces OneLake security (GA). The Contributor role is why it does not apply here.

C9-L2 exercise 3

Hints. (1) The engines behave differently. (2) Which one is stricter? (3) The SQL endpoint.

Answer. a) Spark returns the allowed columns. The endpoint returns an error Because: r1) Microsoft Learn documents that CLS behaves differently: Spark shows only allowed columns, the endpoint blocks the query.

List columns explicitly in SQL queries that users with CLS will run.

Common wrong answers.

- `b`: The SQL analytics endpoint blocks queries that touch hidden columns.
- `c`: Spark succeeds and omits the hidden column.
- `d`: Hidden columns are removed or blocked, not nulled.
- `a|r2`: Right answer, wrong reason. Spark enforces CLS by dropping the hidden column.

C9-L2 exercise 4

Hints. (1) Collect the column sets from roles that include the table. (2) No grants: return an empty set. (3) `set().union(*grants)` for Spark, `set.intersection(*grants)` for the endpoint.

Answer.

```python
from typing import Literal


def visible_columns(
    roles: list[dict[str, set[str]]],
    table: str,
    engine: Literal["spark", "sql_endpoint"],
) -> set[str]:
    """Columns a user sees in one table across their OneLake security roles.

    Args:
        roles: Each role maps table names to the columns it allows.
        table: Table being queried.
        engine: "spark" (union across roles) or "sql_endpoint" (intersection).

    Returns:
        Visible column names. Empty if no role includes the table.

    Raises:
        ValueError: If the engine is not recognised.
    """
    if engine not in ("spark", "sql_endpoint"):
        raise ValueError(f"unknown engine: {engine}")
    grants = [role[table] for role in roles if table in role]
    if not grants:
        return set()
    if engine == "spark":
```

```
        return set().union(*grants)
    return set.intersection(*grants)
```

The same roles give different column sets in different engines. Test the engine your users query.

C9-L2 exercise 5

Hints. (1) Who does OneLake security govern? (2) Where do related row and column rules go? (3) Which Direct Lake mode enforces OneLake security?

Answer.

- Viewers are governed by OneLake security. Contributors are not.
- Row and column rules for the same table belong in one role. Split roles with different columns and a row filter are rejected.
- Direct Lake on OneLake enforces OneLake security (GA).
- DefaultReader grants full reads, and roles combine by union, so it overrides your restrictions.

Viewers, one role per audience, a supported engine mode, and no leftover default grants.

Common wrong answers.

- ws_role = `contributor`: Contributors already have data access. OneLake roles do not restrict them.
- roles = `split`: Two roles with different columns where one filters rows is a combination OneLake does not support. Keep them in one role.
- model = `import`: Direct Lake on OneLake is on the list of engines that enforce OneLake security. An import model holds its own copy and needs its own security rules.
- default = `keep`: Roles combine as a union. DefaultReader's full access wins over your filters.

C9-L3 Sensitivity labels, Purview and lineage

C9-L3 exercise 1

Hints. (1) Was the report's label set by hand? (2) Can inheritance downgrade? (3) No to both.

Answer. b) It stays Highly Confidential Because: r1) Downstream inheritance never overwrites a manually applied label, and never downgrades a label.

Inheritance only raises or fills in labels. It never lowers them or overrides a person.

Common wrong answers.

- `a`: Inheritance never replaces a label with a less restrictive one, and never touches a manual label.
- `c`: The report's label was set by hand, so inheritance leaves it alone.
- `d`: Inheritance does not remove labels.
- `b|r2`: Right answer, wrong reason. Reports do inherit labels from their semantic model. This one was set manually.

C9-L3 exercise 2

Hints. (1) Which feature follows dependencies? (2) It starts from lineage. (3) Impact analysis.

Answer. a) Impact analysis Because: r1) Impact analysis shows affected downstream items and workspaces and can notify their contacts.

Lineage shows the graph. Impact analysis reads it from one item downwards.

Common wrong answers.

- `b`: Domains group items for discovery. They do not track dependencies.
- `c`: Labels classify sensitivity, not dependencies.
- `d`: The Metrics app shows capacity use (lesson C10-L3).
- `a|r2`: Right answer, wrong reason. Impact analysis informs and notifies. It does not block changes.

C9-L3 exercise 3

Hints. (1) Named users keep what they already have. (2) Everyone else? (3) `keeps` and `blocked`.

Answer.

```
1 Item labelled Highly Confidential, policy allows the HR group.
2 HR group member with Viewer access: keeps their Viewer access.
3 Finance user with Viewer access: blocked.
```

A protection policy never adds permissions. It removes access from everyone it does not name.

Common wrong answers.

- g2 = `keeps`: Users not named in the policy are blocked, even with a workspace role.

C9-L3 exercise 4

Hints. (1) Validate both labels against order first. (2) Unlabelled takes upstream. Manual keeps current. (3) Compare `order.index` values.

Answer.

```python
def inherit_label(
    current: str | None, manual: bool, upstream: str, order: list[str]
) -> str | None:
    """Downstream label after the upstream label changes.

    Args:
        current: The downstream item's label, or None.
        manual: Whether a person applied the current label.
        upstream: The upstream item's new label.
        order: Labels from least to most restrictive.

    Returns:
        The downstream item's label after inheritance.

    Raises:
        ValueError: If a label is not in order.
    """
    for label in (current, upstream):
        if label is not None and label not in order:
            raise ValueError(f"unknown label: {label}")
    if current is None:
        return upstream
    if manual:
        return current
    if order.index(upstream) > order.index(current):
        return upstream
    return current
```

Encoding the rule makes its asymmetry visible: labels move up by inheritance, never down.

C9-L3 exercise 5

Hints. (1) What travels with an Excel export? (2) What blocks a person with a link? (3) What lists downstream items?

Answer.

- A label on the model flows to its reports and stays on Excel exports.
- A protection policy blocks anyone outside the HR group, whatever they were sent.
- Impact analysis names what breaks before it breaks.

Label, protect, and check impact before change.

Common wrong answers.

- label = `unlabelled`: Workspace restrictions stop at the workspace. Labels travel with exports.

- policy = `trust`: A protection policy blocks access even when a link reaches the wrong person.
- change = `rename`: Fixing after the break costs a morning of errors across workspaces. Check first.

C9-L4 Audit evidence, Purview scans, label coverage and the audit log

C9-L4 exercise 1

Hints. (1) The question is about people's actions. (2) Which source records activities? (3) The audit log.

Answer. a) The unified audit log in Microsoft Purview

Fabric records user activities in the Purview unified audit log.

Common wrong answers.

- `b`: A scan lists items and lineage. It records no user activity.
- `c`: Coverage shows which items carry labels, not who opened them.
- `d`: Lineage shows how items connect, not who used them.

C9-L4 exercise 2

Hints. (1) How long does Audit (Standard) keep records? (2) Is that longer or shorter than the period? (3) Export before records expire.

Answer. a) Export audit records on a schedule into storage you control Because: r1) Audit (Standard) keeps records for 180 days, so the first months would expire before the end.

Records older than 180 days expire under Audit (Standard), so export on a schedule.

Common wrong answers.

- `b`: By then, records older than 180 days have expired.
- `c`: Audit (Standard) keeps 180 days. A year needs Premium or your own exports.
- `d`: A scan records items, not who accessed them.
- `a|r2`: Right action, wrong reason. Seven days is only the default search range. Retention is the limit.

C9-L4 exercise 3

Hints. (1) Inventory comes from scanning. (2) Protection is measured as a share of items labelled. (3) `scan`, `coverage`, `audit`.

Answer.

```
1 What do you hold?          a Purview Data Map scan of the Fabric tenant
2 How is it protected?       label coverage in the OneLake catalog Govern
     tab
3 Who touched it?            the unified audit log
```

A scan for inventory, label coverage for protection, the audit log for access.

Common wrong answers.

- g1 = `label`: The scan lists items and lineage. It does not label Fabric data.

C9-L4 exercise 4

Hints. (1) Keep a [labelled, total] pair per workspace. (2) Check `"workspace" in item` before you use it. (3) Divide only for workspaces that have items, so an empty list returns `{}`.

Answer.

```python
def label_coverage(items: list[dict]) -> dict[str, float]:
    """Share of items with a sensitivity label, per workspace.

    Args:
        items: Dicts with "workspace" and "label", where label is a name or None.

    Returns:
        Workspace to the share of its items that carry a label, from 0.0 to 1.0.

    Raises:
        ValueError: If an item has no "workspace" key.
    """
    counts: dict[str, list[int]] = {}
    for item in items:
        if "workspace" not in item:
            raise ValueError(f"item without a workspace: {item}")
        tally = counts.setdefault(item["workspace"], [0, 0])
        tally[1] += 1
        if item.get("label") is not None:
            tally[0] += 1
    return {workspace: labelled / total for workspace, (labelled, total) in counts.items()}
```

Count per workspace, so the gaps show where to act. A workspace exists only once it has items, so no division is by zero.

C9-L4 exercise 5

Hints. (1) Which option gives a date and a count? (2) How long do audit records last? (3) Whose permissions apply after the scan?

Answer.

- A scheduled, scoped scan gives a dated inventory an auditor can repeat.
- Monthly exports keep records past the 180-day retention of Audit (Standard).
- Purview permissions, not Fabric roles, govern scanned metadata, so restrict the collection.

Scheduled scans, monthly audit exports and a restricted Purview collection make evidence you can show.

Common wrong answers.

- inventory = `list`: A manual list is a statement, not system evidence.
- access = `later`: Records older than 180 days will have expired.
- metadata = `inherit`: Once in Purview, metadata follows Purview permissions. Fabric roles do not carry over.

C10-L1 Git integration for Fabric workspaces

C10-L1 exercise 1

Hints. (1) Does Git store data or definitions? (2) Definitions. (3) Tables are not tracked.

Answer. b) Metadata only, such as its name, shortcuts and OneLake security roles Because: r1) Microsoft Learn: only lakehouse metadata is tracked. Git operations never overwrite table or file data.

Git versions your items. Pipelines rebuild your data.

Common wrong answers.

- `a`: Tables and files are not tracked in Git.
- `c`: Lakehouses are supported. Only their metadata is tracked.
- `d`: Git is not a data backup. Plan data recovery separately.
- `b|r2`: Right answer, wrong reason. Git integration tracks item definitions by design, not data.

C10-L1 exercise 2

Hints. (1) Both sides changed. (2) What does the source control panel show then? (3) A conflict.

Answer. a) A conflict you must resolve Because: r1) An item changed in both the workspace and the branch is a conflict, shown in the source control panel.

Commit often, and work in your own workspace, so conflicts stay rare.

Common wrong answers.

- `b`: Changes on both sides are flagged as a conflict first.
- `c`: The workspace does not win by default.
- `d`: There is no automatic merge. You resolve the conflict.
- `a|r2`: Right answer, wrong reason. Fabric does not merge notebooks. You choose how to resolve.

C10-L1 exercise 3

Hints. (1) Connecting changes the workspace itself. (2) Lesson C1-L3. (3) Admin.

Answer. a) Admin Because: r1) Connect and disconnect are Admin operations. Members and Contributors commit and update.

Admins connect. Members and Contributors commit and update, within their Git permissions.

Common wrong answers.

- `b`: Members commit and update. Connecting is for Admins (lesson C1-L3).
- `c`: Contributors commit and update. Connecting is for Admins.
- `d`: Viewers see no Git information.
- `a|r2`: Right answer, wrong reason. Git permissions matter for everyone. The workspace role decides who connects.

C10-L1 exercise 4

Hints. (1) Collect the set of statuses present. (2) Validate against the constant first. (3) Check in priority order: conflict, uncommitted, update_required.

Answer.

```
ALLOWED_STATUSES = frozenset({"synced", "uncommitted", "update_required",
    "conflict"})


def next_step(statuses: dict[str, str]) -> str:
    """Next source control action for a workspace.

    Args:
        statuses: Item name to source control status.
```

```
 9
10      Returns:
11          "resolve", "commit", "update" or "none".
12
13      Raises:
14          ValueError: If a status is not recognised.
15      """
16      present = set(statuses.values())
17      unknown = present - ALLOWED_STATUSES
18      if unknown:
19          raise ValueError(f"unknown statuses: {sorted(unknown)}")
20      if "conflict" in present:
21          return "resolve"
22      if "uncommitted" in present:
23          return "commit"
24      if "update_required" in present:
25          return "update"
26      return "none"
```

Priority order encodes the safe sequence: never let an update overwrite work you have not committed.

C10-L1 exercise 5

Hints. (1) Where does work in progress live? (2) How do changes reach main? (3) What does Git hold for a lakehouse?

Answer.

- One workspace and branch per engineer isolates work in progress.
- Pull requests put a review between each change and main.
- Git holds lakehouse metadata only, so data comes back through pipelines.

Isolated workspaces, reviewed merges, and pipelines that rebuild data.

Common wrong answers.

- workspaces = `shared`: A shared workspace is a shared runtime. One person's update overwrites another's uncommitted work.
- merge = `direct`: Direct commits skip review.
- data = `nothing`: Tables and files are not tracked in Git.

C10-L2 Deployment pipelines, dev to test to production

C10-L2 exercise 1

Hints. (1) What does a deployment copy? (2) Items and metadata. (3) Not data.

Answer. b) Refresh the model in Production, because data is not copied between stages Because: r1) Microsoft Learn: a semantic model refresh is required because data isn't copied from one stage to another.

Deploy, then refresh or reload in the target stage.

Common wrong answers.

- `a`: Data is not copied between stages.
- `c`: Deployment does not touch permissions. On later deployments, IDs, URLs and permissions stay the same.
- `d`: Deployment pipelines and Git connections are separate.
- `b|r2`: Right answer, wrong reason. Deployment does not clear data. It never copied any.

C10-L2 exercise 2

Hints. (1) Which stage needs the different value? (2) The target. (3) Default lakehouse rule.

Answer. a) In the Production stage, on the notebook, as a default lakehouse rule Because: r1) Rules are defined in the target stage on the item, and apply from the next deployment into that stage.

The target stage owns its settings.

Common wrong answers.

- `b`: Rules belong to the target stage.
- `c`: Branching on workspace names hard-codes environments into code. Use a deployment rule.
- `d`: Deployment rules are pipeline configuration, set per stage.
- `a|r2`: Right answer, wrong reason. Rules are per stage and per item, not global.

C10-L2 exercise 3

Hints. (1) Rules must exist before deployment. (2) Compare before deploy. (3) Data after deploy, checks last.

Answer.

1. Check the deployment rules in the Production stage
2. Compare Test with Production and review differences
3. Deploy Test to Production
4. Refresh semantic models and run load pipelines in Production
5. Check row counts and a known report figure in Production

Rules, compare, deploy, reload, verify.

Common wrong answers.

- Rules apply at deployment. Check them before you deploy.
- Verify after the data is loaded, or you check an empty model.

C10-L2 exercise 4

Hints. (1) `rules.get(stage, {})` handles stages without rules. (2) Compare rule keys with source keys. (3) `{**source, **stage_rules}` builds a new dict.

Answer.

```python
def deployed_settings(
    source: dict[str, str], rules: dict[str, dict[str, str]], stage: str
) -> dict[str, str]:
    """Item settings in a target stage after its deployment rules apply.

    Args:
        source: Settings in the source stage.
        rules: Stage name to the settings its rules override.
        stage: Target stage.

    Returns:
        A new dict of settings for the target stage.

    Raises:
        ValueError: If a rule names a setting the item does not have.
    """
    stage_rules = rules.get(stage, {})
    unknown = set(stage_rules) - set(source)
    if unknown:
        raise ValueError(f"rules for unknown settings: {sorted(unknown)}")
    return {**source, **stage_rules}
```

Settings per stage, validated, with the source left intact: the same contract deployment rules give you.

C10-L2 exercise 5

Hints. (1) How do changes reach Development? (2) Which tool keeps IDs and applies rules? (3) What does a deployment not copy?

Answer.

- Git and pull requests control what reaches Development.
- Deployment pipelines promote with per-stage rules and a compare step.
- Deployments copy items, not data, so reload and check.

Git for development, deployment pipelines for promotion, and a reload and check after each release.

Common wrong answers.

- dev = `edits`: Direct edits skip review and conflict with Git updates.
- promote = `copy`: Manual copies lose rules, compare and autobinding, and produce new IDs and URLs.
- post = `done`: Data is not copied between stages.

C10-L3 Capacity, smoothing and throttling

C10-L3 exercise 1

Hints. (1) Background or interactive? (2) Background jobs are long and heavy. (3) 24 hours.

Answer. a) 24 hours Because: r1) Microsoft Learn: background operations are smoothed over a 24-hour period.

Background work is spread over a day, so one heavy job raises every timepoint a little.

Common wrong answers.

- `b`: Five minutes is the minimum for interactive operations.
- `c`: Smoothing spreads cost beyond the job's run time.
- `d`: Background operations are smoothed, over 24 hours.
- `a|r2`: Right answer, wrong reason. The smoothing window is fixed. It does not depend on job length.

C10-L3 exercise 2

Hints. (1) Which band is 90 minutes in? (2) 60 minutes to 24 hours. (3) Interactive rejection.

Answer. a) The report is rejected. The pipeline still runs Because: r1) Between 60 minutes and 24 hours, interactive jobs are rejected. Background rejection starts only above 24 hours.

Users feel throttling first. Background jobs, often the cause, keep running.

Common wrong answers.

- `b`: The 20-second delay applies between 10 and 60 minutes.
- `c`: Background jobs are rejected only above 24 hours of future use.
- `d`: Above 60 minutes, interactive jobs are rejected.
- `a|r2`: Right answer, wrong reason. Background jobs have a later threshold, not priority.

C10-L3 exercise 3

Hints. (1) CUs times hours in a day. (2) Divide the job by the daily total. (3) 96 CU hours, about 2.1%.

Answer.

```
F4 capacity per day: 4 CUs x 24 hours = 96 CU hours
Share of every timepoint for 24 hours: 2 / 96 = about 2.1%
```

2 / 96 is about 2.1%, the same share as the F2 example with half the job.

Common wrong answers.

- g1 = `4`: Multiply CUs by 24 hours for the daily CU hours.

C10-L3 exercise 4

Hints. (1) Three constants: 10, 60 and 1,440 minutes. (2) Use `<=` for each limit. (3) Validate first.

Answer.

```python
OVERAGE_PROTECTION_MINUTES = 10
INTERACTIVE_DELAY_MINUTES = 60
INTERACTIVE_REJECTION_MINUTES = 24 * 60


def throttle_stage(future_minutes: float) -> str:
    """Fabric throttling stage for the future capacity already consumed.

    Args:
        future_minutes: Minutes of future capacity used by smoothed operations.

    Returns:
        "none", "interactive_delay", "interactive_rejection" or "background_rejection".

    Raises:
        ValueError: If future_minutes is negative.
    """
    if future_minutes < 0:
        raise ValueError("future_minutes must not be negative")
    if future_minutes <= OVERAGE_PROTECTION_MINUTES:
        return "none"
    if future_minutes <= INTERACTIVE_DELAY_MINUTES:
        return "interactive_delay"
    if future_minutes <= INTERACTIVE_REJECTION_MINUTES:
        return "interactive_rejection"
    return "background_rejection"
```

Boundaries are where bugs hide. Test each limit exactly.

C10-L3 exercise 5

Hints. (1) Where does a 06:00 job's cost land? (2) Who shares the capacity? (3) Which setting protects interactive users?

Answer.

- Smoothing spreads a 06:00 job over the next 24 hours, including the morning peak. Less work, earlier, helps.
- Separate capacities stop experiments from consuming production CUs.
- Surge protection rejects background work before users are rejected. Resize only if the Compute page still shows a shortfall.

Reduce and move background work, isolate experiments, then protect users with surge protection.

Common wrong answers.

- refresh = `keep`: Smoothing does not remove cost. It spreads it across the morning peak.
- ds = `same`: All-day notebooks add to every timepoint on the production capacity.
- protect = `resize`: Fix the workload first. A larger SKU doubles the bill and hides the cause.